ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 407

Final Report

TARAS: Agricultural Analysis & Simulation

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Literature Review.....5
1.1. Introduction.....5
1.1.1. Background.....5
1.1.2. Problem Definition.....5
1.1.3. Scope of the Study.....6
1.1.4. Aim and Contributions.....7
1.2. IoT Sensor Node Architecture.....8
1.2.1. Microcontroller Selection: Fully Wireless ESP32.....8
1.2.2. Sensing Technologies: Capacitive vs. Resistive.....9
1.2.3. Sensor Calibration.....9
1.3. Cloud Computing and Data Management.....9
1.3.1. Structured Database Design (PostgreSQL).....9
1.4. Irrigation Recommendation.....10
1.5. Artificial Intelligence and Machine Learning.....10
1.5.1. Computer Vision for Disease Detection (CNNs).....10
1.5.2. Large Language Models.....11
1.6. Visualization and Digital Twins.....11
1.6.1. Reducing Cognitive Load with Digital Twins.....11
1.7. Mobile Application.....12
1.7.1. Mobile Applications in Agriculture.....12
1.7.2. Mobile App in TARAS.....12
1.7.3. Technology Stack.....12
1.8. Ecological Impact.....13
1.8.1. Water Consumption.....13
1.8.2. Carbon Footprint.....13
1.9. Related Work.....14
1.9.1. Single-Purpose Smart Irrigation Systems.....14
1.9.2. Standalone Disease Detection Applications.....15
1.9.3. High-End Digital Twins.....15
1.10. Conclusion and Research Gap.....16
1.11. References.....16
2. SRS.....18
2.1. Introduction.....18
2.1.1. Purpose.....18
2.1.2. Scope.....19
2.2. General Description.....19
2.2.1. Glossary.....19
2.2.2. User Characteristics.....22
2.2.2.1. Primary User: Farmer.....22
2.2.2.2. External Party: Stakeholders.....23
2.2.3. Overview of Functional Requirements.....23
2.2.4. General Constraints and Assumptions.....24
2.2.4.1. Constraints.....24

2.2.4.2. Assumptions..... 25
2.3. Specific Requirements.....26
2.3.1. Interface Requirements..... 26
2.3.1.1. Hardware Interface Requirements.....26
2.3.1.2. Communication Requirements.....27
2.3.1.3. System and Backend Requirements.....28
2.3.1.4. Software Interface Requirements..... 29
2.3.1.5. User Interface Requirements.....30
2.3.2. Detailed Description of Functional Requirements.....31
● Microcontroller - Sensor Interface.....31
● Gateway Hardware Interface Functional Requirement.....32
● Power Interface Functional Requirement..... 33
● Environmental / Enclosure Interface Functional Requirement.....34
● Wi-Fi Uplink Communication – Functional Requirement..... 34
● Store-and-Forward Communication Reliability Functional Requirement..... 35
● Node-to-Cloud Communication – Functional Requirement..... 36
● Sensor Telemetry Storage and Management Functional Requirement..... 37
● Disease Detection Database Recording Functional Requirement.....38
● Digital Twin Data Assembly Functional Requirement..... 39
● Disease Detection Functional Requirement..... 39
● Irrigation Prediction Request & Response Handling Functional Requirement..... 41
● Role-Based Access Control & IU Authorization Functional Requirement.....42
● Accessibility & Inclusive UI Support Functional Requirement..... 42
● Mobile Application Framework & Navigation Functional Requirement..... 43
● Carbon Footprint Calculation & Display.....44
2.3.3. Data Preprocessing..... 45
2.3.3.1. Irrigation Data Preprocessing..... 45
2.3.3.1.1. Purpose and Data Sources.....45
2.3.3.1.2. Timestamp Standardization and Time Alignment.....46
2.3.3.1.3. Sensor Quality Control (Noise, Drift, Spikes).....46
2.3.3.1.4. Missing Data Analysis and Handling.....46
2.3.3.1.5. Context-Aware Outlier Detection.....46
2.3.3.1.6. Crop Coefficient (Kc) Profile Preparation.....47
2.3.3.1.7. Derived Features for ET_0 + Kc Irrigation Logic.....47
2.3.3.2. Image Data Preprocessing for Disease Detection.....47
2.3.3.2.1. Raw Image Validation..... 47
2.3.3.2.2. Image Resizing and Aspect-Ratio Normalization.....47
2.3.3.2.3. Color Space Standardization..... 48
2.3.3.2.4. Noise Reduction.....48
2.3.3.2.5. Leaf Region Isolation..... 48
2.3.3.2.6. Data Augmentation for Model Generalization..... 48
2.3.3.2.7. Image Quality Assessment Checks..... 48
2.3.3.2.8. Dataset Class Distribution Documentation..... 48
2.3.3.2.9. Normalization for Inference.....49

2.3.4. Non-Functional Requirements....49
2.3.4.1. Performance Requirements....49
2.3.4.2. Reliability Requirements....49
2.3.4.3. Availability Requirements....49
2.3.4.4. Scalability Requirements....50
2.3.4.5. Security Requirements....50
2.3.4.6. Usability Requirements....50
2.3.4.7. Maintainability Requirements....51
2.3.4.8. Portability Requirements....51
2.4. Analysis - UML....52
2.4.1. Use Cases....52
2.4.1.1. Use Case Diagram....52
2.4.1.2. Description of Use Cases....53
2.4.2. Functional Modelling(DFD)....67
2.4.2.1. DFD Diagrams....67
2.4.2.1.1. Level 0 - Context Diagram....67
2.4.2.1.2. Level 1 DFD....68
2.5. Conclusion....68
2.6. References....68
3. System Architecture & Methodology....69
3.1. Introduction....69
3.1.1. Purpose....69
3.1.2. Definitions and Acronyms....70
3.2. System Overview....72
3.3. System Architecture....72
3.3.1. Architectural Design....72
3.3.1.1. Layered Architecture....72
3.3.1.1.1. Presentation Layer....72
3.3.1.1.2. Application & Analytics Layer....73
3.3.1.1.3. Data & Cloud Services Layer....73
3.3.1.1.4. Device & Sensing Layer....73
3.3.1.1.5. Communication Layer....73
3.3.1.2. Client–Server Architecture....73
3.3.2. Decomposition Description....73
3.3.2.1. Field Sensing & Data Acquisition Subsystem....73
3.3.2.2. Analytics & Decision Support Subsystem....74
3.3.2.3. Visualization & User Interaction Subsystem....74
3.3.2.4. Class Diagram....74
3.3.3. System Modeling....75
3.3.3.1. Activity Diagrams....75
3.3.3.2. Sequence Diagrams....92
3.4. Methodology....101
3.4.1. Irrigation Prediction....101
3.4.1.1. Problem Formulation....101

3.4.1.2. Data Acquisition and Preprocessing..... 101
3.4.1.3. Algorithmic Approach..... 102
3.4.1.4. System Evaluation Metrics..... 102
3.4.1.5. Implementation Details..... 102
3.4.2. Disease Detection..... 103
3.4.2.1. Problem Formulation..... 103
3.4.2.2. Data Acquisition and Preprocessing..... 103
3.4.2.2.1. Data Acquisition..... 103
3.4.2.2.2. Preprocessing..... 103
3.4.2.3. Algorithmic Approach..... 103
3.4.2.4. Model Selection and Comparative Evaluation..... 104
3.4.2.5. System Evaluation Metrics..... 105
3.4.2.6. Implementation Details..... 105
3.4.3. LLM-Based Advisory & Decision Support..... 106
3.4.3.1. Problem Formulation..... 106
3.4.3.2. Input Representation and Context Construction..... 106
3.4.3.3. Response Generation and Explainability..... 107
3.4.3.4. System Evaluation and Limitations..... 107
3.4.3.5. Implementation Details..... 107
3.5. Data Design..... 108
3.5.1. Entity-Relationship Diagram..... 108
3.5.2. Data Pipeline..... 108
3.6. User Interface Design..... 109
3.6.1. Interface Hierarchy..... 109
3.6.2. Visual Presentation..... 109
3.6.3. Accessibility and Responsivity..... 110
3.7. Conclusion..... 111
3.8. References..... 111

1. Literature Review

1.1. Introduction

Modern agriculture is increasingly affected by climate variability, declining freshwater resources, and sustainability challenges. Many farmers still make irrigation and crop-management decisions based on intuition rather than systematic measurements, which reduces decision accuracy and leads to inefficient water use [1]. Research also shows that improper irrigation practices can cause water loss and yield reduction, making data-driven decision support essential for future agricultural productivity [2], [3].

Recent advances in digital agriculture offer viable solutions. IoT-based sensing systems provide continuous measurements of soil and environmental conditions [4], while machine-learning algorithms can analyze these data to generate accurate irrigation recommendations [4]. Digital twin technologies enable virtual representations of agricultural fields for real-time simulation and visualization [5], and later studies show that digital twins can significantly enhance field monitoring under dynamic conditions [6]. In parallel, computer-vision models have demonstrated strong performance in diagnosing plant diseases directly from leaf images [7].

TARAS platform brings these technologies together into a unified, farmer-oriented system that offers real-time field monitoring, irrigation prediction, disease assessment, and digital-twin visualization. The project aligns with Türkiye's Twelfth Development Plan, which emphasizes AI-supported digital agriculture [8], and also contributes to global sustainability objectives such as SDG 12, SDG 13, and SDG 15 [9].

1.1.1. Background

Agriculture has progressed through four major transformations: the emergence of settled farming, the mechanization introduced during the Industrial Revolution, the productivity gains of the Green Revolution, and today's Agriculture 4.0 era driven by digital technologies. Precision Agriculture, a key component of this era, relies on monitoring and responding to spatial and temporal field variability to optimize resource use [4]. IoT sensors enable real-time environmental monitoring, digital twins provide interactive virtual models of fields for prediction and analysis [5], and computer-vision systems support early disease recognition through image-based diagnosis [7]. TARAS builds on this technological landscape to create a unified platform tailored for real agricultural environments.

1.1.2. Problem Definition

Traditional Farming Limitations: One of the major challenges in modern agriculture is that farmers struggle to make reliable, data-based decisions. Many still rely on intuition rather than systematic and evidence-based methods, which

limits the accuracy of field management [1]. Although intuition has supported farming for generations, it is becoming less effective as climate patterns shift and soil conditions change. As a result, crops often fail to reach their full yield potential when decisions depend solely on historical experience.

Resource Scarcity: Water scarcity is one of the most critical constraints in agriculture today. Traditional irrigation methods frequently lead to over-watering, which wastes water and damages root systems, or under-watering, which stunts plant growth. Studies show that inefficient or poorly timed irrigation can cause significant water loss and reduce productivity [2], [3]. Because soil moisture varies across fields and changes over time, static rule-based irrigation schedules cannot meet crop-specific requirements, leading to substantial inefficiencies in both water and energy use.

Disease Management Challenges: Farmers commonly notice unusual visual changes in plants but cannot determine whether these differences indicate disease or harmless variation. This uncertainty can delay intervention or lead to unnecessary pesticide use. When early symptoms are subtle or unfamiliar, problems may be ignored until they become severe, or pesticides may be applied without need. Both outcomes increase production costs and negatively affect food safety and environmental health. Image-based decision-support systems can help address this uncertainty by allowing farmers to upload leaf images and receive automated analysis, improving early assessment and supporting timely, informed action [7].

Data Quality & System Reliability Issues: Even when farmers adopt technological tools, field data are often incomplete or inconsistent due to sensor drift, hardware faults, or communication failures. Soil-moisture probes may lose calibration over time, wireless connections may drop, and environmental fluctuations can distort sensor readings. Without proper validation, calibration, and anomaly-detection mechanisms, inaccurate data can mislead decision-making and become as harmful as intuition-based irrigation.

Lack of Accessible, Integrated Decision Support: Many farms rely on fragmented digital tools, such as separate systems for sensors, irrigation controllers, weather forecasts, and crop health information. Because these systems do not communicate with each other, farmers, especially small-scale producers, struggle to benefit from advanced analytics. This fragmentation highlights the need for an integrated platform that combines real-time field monitoring, intelligent recommendations, visual explanations, and sustainability metrics within a single accessible system.

1.1.3. Scope of the Study

The scope of TARAS covers the development of an end-to-end precision-agriculture platform. The system includes IoT-based soil-moisture and temperature sensing using ESP32 microcontrollers with integrated battery management, and direct WiFi connectivity for field-to-cloud data transfer. It incorporates a secure cloud database for storing real-time and historical records, physics-based irrigation prediction algorithms following FAO guidelines, and computer-vision algorithms for disease identification.

The project further includes the design of a mobile application that provides farmers with moisture visualization through a digital twin using Mapbox-based SVG overlays, image-based disease assessment with weather-integrated risk prediction, and an AI-assisted decision interface. The system focuses on decision support rather than full automation, ensuring that farmers remain central to the decision-making process. It is intended to be low-cost, portable, and scalable across various field configurations.

1.1.4. Aim and Contributions

The aim of TARAS is to provide farmers with an intelligent, real-time agricultural assistant that can analyze environmental conditions, predict irrigation requirements, detect plant diseases from leaf images, and visualize moisture distribution through digital twin technology. By bringing together IoT, machine learning, computer vision and cloud computing, the system offers a highly adaptive and user-centered approach that supports precision farming and sustainable resource management.

TARAS aims to optimize irrigation efficiency, reduce water and energy waste, enhance crop health monitoring, improve sustainability through carbon footprint tracking and provide farmers with actionable, explainable, and accessible insights

Contributions: Recent advancements in smart agriculture highlight the value of combining IoT-based sensing with machine-learning-driven decision systems to improve irrigation efficiency and field management. Studies show that ML algorithms using soil moisture, temperature, humidity, and soil-type data can generate highly precise irrigation recommendations and adapt to changing field and weather conditions. TARAS builds on these findings by pairing such predictive models with real-time digital-twin visualizations, enabling farmers to interpret moisture patterns through intuitive heatmaps, observe changes over time, and identify areas requiring intervention more clearly.

Beyond irrigation, TARAS incorporates computer-vision-based disease detection that allows farmers to upload leaf images through the mobile application. These images are processed using CNN models that classify diseases or stress symptoms and provide actionable assessments, reducing the need for expert diagnosis. The platform also addresses sustainability by integrating carbon-footprint metrics, estimating emissions based on pump energy usage, fertilizer inputs and water savings.

By combining these capabilities into a single, cohesive platform, TARAS contributes to the development of intelligent, adaptive and sustainability-focused digital farming systems that make advanced decision support accessible to farmers across different scales.

1.2. IoT Sensor Node Architecture

The foundation of any smart farming system is the ability to collect accurate environmental data from the field [10]. The literature emphasizes that hardware in agriculture must be low-power, durable, and cost-effective.

1.2.1. Microcontroller Selection: Fully Wireless ESP32

In the realm of agricultural IoT, the primary engineering challenge is maximizing the operational lifespan of battery-powered nodes. Zhang and Meng (2021) highlight that energy efficiency is the most critical factor for the economic viability of precision agriculture, as frequent battery replacements increase labor costs significantly [11], and using wired technology introduces complexity most fields are not suited to deal with.

While earlier projects successfully utilized standard ESP32 or ESP8266 modules, recent literature indicates a shift toward more specialized architectures. Recent studies in wireless sensor networks have begun favoring efficient architectures for their superior power-to-performance ratio. For example, a study by Irga et al. (2023) implemented agricultural monitoring nodes noting a significant reduction in standby power consumption compared to traditional controllers [12].

Building on these findings, the TARAS project utilizes the ESP32 microcontroller, which offers excellent power efficiency, and by integrating a battery, allows for a fully wireless solution. This integrated approach simplifies the hardware design while ensuring reliable long-term operation in field conditions. Mohd Khalid et al. (2024) highlighted that these chips, when paired with renewable energy sources, allow nodes to survive extended periods without going offline, a critical factor for continuous year-round field monitoring [13].

Another advantage of using ESP32 is its support of the I²C (Inter-Integrated Circuit) Communication Interface, which is a widely used two-wire communication interface designed for low-power embedded systems. I²C enables multiple digital sensors to communicate with the ESP32 microcontroller over a shared SDA (data) and SCL (clock) bus, reducing wiring complexity and supporting scalable node designs. Its address-based structure allows several sensors to operate simultaneously, making it particularly suitable for multi-parameter agricultural monitoring. The protocol's low latency and

efficient power consumption make it ideal for continuous field measurements where reliability and simplicity are critical.[14]

1.2.2. Sensing Technologies: Capacitive vs. Resistive

Accurate irrigation depends entirely on accurate soil moisture readings. Historic literature often cited "resistive" sensors, which measure moisture by passing an electric current through the soil [15]. However, engineering studies have extensively documented the failure of these sensors due to **electrolysis**, which causes the metal probes to corrode (rust) within weeks [16].

To solve this, modern research advocates for **Capacitive Soil Moisture Sensors**. These measure the dielectric permittivity of the soil, basically how much charge the soil can hold, without any exposed metal touching the soil. This prevents corrosion and ensures reliable data over long periods [17]. Similarly, for air metrics, digital sensors like the **SHT Series sensors** are preferred over analog thermistors because they provide calibrated digital output, eliminating errors caused by electrical noise in the wires [18].

1.2.3. Sensor Calibration

Proper calibration of soil moisture sensors is essential for accurate readings. The TARAS system employs a simple min-max calibration procedure that can be performed in the field. Using two reference samples, completely dry soil and fully saturated (muddy) soil, the sensor's raw analog values are mapped to the 0-100% moisture range. This calibration data is stored and applied to all subsequent readings. This approach ensures that sensors remain accurate across different soil types and conditions without requiring complex laboratory calibration procedures.

1.3. Cloud Computing and Data Management

Handling the influx of data from the field requires a scalable and organized backend system.

1.3.1. Structured Database Design (PostgreSQL)

Agricultural data is primarily "Time-Series Data" (e.g., specific moisture values recorded at specific times). While flexible NoSQL databases are popular for web apps, scientific applications usually prefer Relational Databases like **PostgreSQL** for their strict structure and reliability. PostgreSQL is ACID-compliant, meaning it guarantees data validity even in the event of errors or power failures [24]. Furthermore, the system utilizes a "Log-Based" architecture. As recommended in foundational database literature, this approach involves never deleting or overwriting old records, but simply appending new ones [24]. This creates an

unchangeable history of the field, which is essential for training accurate machine learning models later.

1.4. Irrigation Recommendation

Determining when to irrigate crops is primarily a decision-making problem that depends on accurately interpreting soil-moisture dynamics and environmental conditions. Traditional irrigation controllers often rely on fixed schedules or simple threshold-based rules, which can lead to inefficient water use when field conditions vary over time.

In this system, irrigation decisions are generated using a calibrated rule-based approach that evaluates time-series soil-moisture trends together with environmental conditions and field-specific parameters. By analyzing moisture-level thresholds and rate-of-change behavior, the method adapts irrigation timing and duration without relying on large historical datasets or complex predictive models.

Compared to machine-learning-based solutions, this rule-based method offers improved transparency, lower computational complexity and greater robustness in environments with limited data availability or intermittent connectivity. Its explainable structure allows irrigation recommendations to be easily interpreted and adjusted, making it suitable for practical field deployment.

1.5. Artificial Intelligence and Machine Learning

Collecting data is only useful if it leads to better decisions. Artificial Intelligence is used to process raw numbers into actionable advice [25].

1.5.1. Computer Vision for Disease Detection (CNNs)

Identifying plant diseases early can prevent significant yield losses; however, expert agronomists are not always readily available [7]. To address this challenge, we employ Convolutional Neural Networks (CNNs), which are deep learning models specifically designed for image analysis. By training a CNN on large-scale datasets containing images of both healthy and diseased plant leaves, the model learns to identify discriminative visual patterns such as color variations, texture irregularities, and lesion structures associated with different plant diseases.

In this study, the **EfficientNet architecture** is selected due to its ability to achieve high classification accuracy with significantly fewer parameters compared to conventional convolutional neural network (CNN) models. EfficientNet introduces a compound scaling strategy that uniformly scales network depth, width, and input resolution, leading to improved performance while preserving computational efficiency. This balanced

scaling approach has been shown to outperform traditional CNN architectures across various image classification benchmarks while requiring fewer computational resources.[19]

1.5.2. Large Language Models

To translate complex analytical outputs into actionable guidance for farmers, TARAS integrates a **Large Language Model (LLM)** within an interactive chatbot interface. This component acts as the system's communication layer, interpreting sensor data, historical irrigation records, and computer vision findings to generate clear, context-aware explanations.

Research by Wang et al. (2024) indicates that LLMs significantly bridge the gap between technical data and farmer understanding by providing natural language explanations for agronomic advice [20]. Instead of presenting users with raw numerical values, the LLM-equipped assistant provides meaningful insights, for example, why irrigation is recommended on a specific day or how a detected disease might impact the crop. Additionally, the LLM supports "what-if" scenario analysis by simulating alternative decisions (such as delaying irrigation or applying preventive treatments) and explaining their potential outcomes. Through domain-adapted fine-tuning or retrieval-augmented generation (RAG), the model ensures high accuracy, transforming TARAS into a transparent, reliable, and user-friendly agricultural advisor.

1.6. Visualization and Digital Twins

The interface between the human and the machine is just as important as the technology itself.

1.6.1. Reducing Cognitive Load with Digital Twins

"Cognitive Load" refers to how much mental effort is needed to understand information. Research indicates that standard spreadsheets impose a high cognitive load, whereas visual maps reduce this load significantly [21]. The concept of the **Digital Twin** involves creating a 3D virtual replica of the physical field. Research suggests that visualizing data spatially, such as a 3D heat map where dry areas glow red, allows farmers to identify problem zones much faster than reading lists of numbers [22]. To make this accessible, **React Native** is used to build a cross-platform mobile app, ensuring the tool works on the smartphones farmers already own.

1.7. Mobile Application

Mobile applications have emerged as one of the most widely adopted interfaces in smart agriculture systems. Studies highlight that mobile platforms make complex IoT and AI infrastructures accessible by presenting field information directly to farmers in real time. They support continuous monitoring, rapid decision making, and convenient data entry, which makes them a practical tool for field operations where desktop interfaces are rarely available [23].

1.7.1. Mobile Applications in Agriculture

Recent smart-agriculture research emphasizes the role of mobile applications as the primary interaction layer between farmers and digital farming systems. Mobile tools frequently act as gateways for sensor networks, cloud platforms, and machine-learning models. They are used for monitoring field conditions, visualizing environmental data, delivering agronomic recommendations, and providing remote diagnostic services. Literature also notes that mobile interfaces significantly increase technology adoption among farmers, especially when the design is simple and the recommendations are easy to interpret [23].

1.7.2. Mobile App in TARAS

Within this literature landscape, the TARAS mobile application follows the same principles identified in prior studies. It functions as the primary interface through which farmers receive sensor readings, irrigation recommendations, and disease analysis results. The app also includes a 3D digital-twin visualization that represents moisture distribution spatially, a feature that extends the capabilities of many existing mobile agricultural systems. By combining monitoring, prediction, disease assessment, and spatial visualization into a single platform, TARAS addresses the fragmentation noted in current literature and provides a unified decision-support tool aligned with modern precision-agriculture practices.

1.7.3. Technology Stack

The technological choices used in the TARAS mobile application reflect practices recommended in mobile and IoT literature. **React Native** is widely adopted for agricultural applications because it enables cross-platform deployment on Android and iOS, which is essential given the diversity of devices in rural areas [24]. Expo simplifies development and maintenance processes by managing native modules and supporting incremental updates, which is useful for systems deployed in field environments with limited technical infrastructure [24].

For visualization, **React Three Fiber** allows efficient rendering of 3D elements on mobile devices, supporting interactive digital-twin

displays similar to those explored in recent spatial agriculture studies [22]. The data exchange between the mobile client and the cloud backend uses a simple request–response API model, which is frequently adopted in smart farming architectures for real-time synchronization.

1.8. Ecological Impact

Technology must also serve environmental goals. Modern agriculture is under pressure to reduce its environmental impact, in line with UN Sustainable Development Goals 12 and 13 [9]. Literature suggests that IoT systems are uniquely positioned to automate sustainability tracking. By monitoring exactly how long water pumps run and how much resource is consumed, the system can calculate a real-time **Carbon Footprint**. This moves sustainability from a vague concept to a measurable metric [25].

1.8.1. Water Consumption

The TARAS system promotes responsible resource use by ensuring that irrigation is conducted only when and where it is needed, directly supporting the principles of Goal 12. By replacing uniform, schedule-based watering with sensor-driven and Algorithm-optimized recommendations, TARAS minimizes unnecessary water consumption, reduces nutrient runoff, and lowers the overall environmental footprint of agricultural production. This precision in irrigation not only conserves water but also decreases the energy required for pumping, thereby reducing operational emissions and contributing to Goal 13 on climate action. Through its Digital Twin visualizations and adaptive decision-support engine, the system enables farmers to monitor consumption patterns, identify inefficiencies, and implement more sustainable long-term practices. In doing so, TARAS strengthens both resource responsibility and climate resilience within modern agriculture.

1.8.2. Carbon Footprint

Agriculture is one of the most significant contributors to global greenhouse gas (GHG) emissions, accounting for approximately one-fifth of total anthropogenic emissions [26]. The main sources include fertilizer production and application, energy consumption for irrigation and machinery, livestock management, and soil-based emissions such as nitrous oxide (N_2O) from nitrogen fertilizers and methane (CH_4) from organic decomposition. These emissions, when expressed as carbon dioxide equivalents (CO_2e), constitute the carbon footprint of agricultural production. Understanding and reducing this footprint has become a major focus in achieving the United Nations Sustainable Development Goals (SDGs), particularly Goal 12 (Responsible Consumption and Production) and Goal 13 (Climate Action).

Recent studies indicate that digital and IoT-based farming systems can play a transformative role in mitigating agricultural greenhouse gas emissions. Precision irrigation systems that utilize real-time soil-moisture and climate sensing enable farmers to reduce unnecessary water use and the associated energy required for pumping, thereby decreasing CO_2 emissions [27]. Similarly, IoT-supported fertilization and nutrient-management approaches help align nitrogen application with actual crop demand, preventing overuse and limiting nitrous oxide (N_2O) emissions, one of the most potent agricultural greenhouse gases [28]. Overall, these digital and sensor-driven strategies improve resource efficiency and contribute to more sustainable agricultural production [29].

ML and decision-support systems further enhance the ability to estimate and monitor carbon footprints dynamically. By combining environmental sensor data (soil moisture, temperature) with user-provided activity data (fertilizer use, diesel consumption, electricity usage), these systems can calculate real-time carbon emission scores at the farm or greenhouse level.

Integrating this functionality into a mobile application significantly increases accessibility and user engagement. Farmers can view their estimated emissions, identify the main contributing factors. Within the TARAS system, this allows users to enter their input data and instantly calculate their farm's carbon footprint. The module supports sustainability awareness and provides actionable insights for emission reduction strategies, reinforcing the system's contribution to climate action (Goal 13) and responsible production (Goal 12).

1.9. Related Work

To design the TARAS platform, we analyzed several existing systems and research projects. While many solutions exist, they typically solve only one part of the problem. This section discusses these related works and how they directly influenced our design choices.

1.9.1. Single-Purpose Smart Irrigation Systems

A significant amount of research focuses solely on automating water pumps.

- **The Study:** A prominent study by Nawandar & Satpute (2019) developed a low-cost IoT system using simple neural networks to schedule irrigation based on soil data [30].
- **Limitation:** While effective at saving water, this system operates in a vacuum. It assumes the plant is healthy. If a plant is attacked by a fungus, it might need less water or a chemical treatment, not

just standard irrigation. The system had no way of "seeing" the plant's health.

- **Effect on TARAS:** This influenced us to build a hybrid system. TARAS combines soil sensors with camera-based disease detection so decisions are made based on the whole picture, not just the soil moisture level.

1.9.2. Standalone Disease Detection Applications

On the other side of the spectrum, there is extensive research on using AI to spot diseases.

- **The Study:** The most famous work in this field is by Mohanty et al. (2016), who trained Deep Learning models on the PlantVillage dataset [7].
- **Limitation:** While scientifically impressive, this approach was purely diagnostic. It tells the farmer "You have Apple Scab," but it doesn't link that information to the farm's environmental history. Was the disease caused by over-watering? The app couldn't say because it didn't have access to soil data.
- **Effect on TARAS:** This taught us that context matters. In TARAS, when a disease is detected, the system can cross-reference it with historical humidity logs to help explain why the outbreak happened.

1.9.3. High-End Digital Twins

Finally, we looked at existing "Digital Twin" concepts.

- **The Study:** Pylianidis et al. (2021) explored creating digital twins for high-tech greenhouses. These systems create incredible 3D simulations of crop growth and climate control [31].
- **Limitation:** These systems are extremely complex, expensive, and require powerful desktop computers. They are designed for industrial agronomists, not for a typical farmer standing in a field with a phone.
- **Effect on TARAS:** This defined our Mobile-First strategy. We chose technologies like React Native and low-poly 3D rendering to ensure our Digital Twin works smoothly on a standard smartphone, making the technology accessible to regular farmers.

1.10. Conclusion and Research Gap

However, a significant gap remains. Most current solutions function in isolation, offering either irrigation control or disease detection, and often lack user-friendly visualization tools for small-scale farmers. Furthermore, few systems actively track sustainability metrics like carbon footprints alongside agronomic data. Consequently, there is a need for a unified, low-cost platform that combines real-time sensing, AI-driven predictions and visual analytics into a single accessible mobile system. The proposed TARAS project aims to address this gap by developing an end-to-end solution tailored for these needs.

1.11. References

[1] A. Araujo-McCole, K. D. Ilyes, and J. M. S. da Silva, "Exploring Farmers' Decision-Making Practices in Precision Agriculture," *Agricultural Systems*, vol. 204, p. 103554, 2023.

[2] S. Ali et al., "Effects of Improper Irrigation Practices on Water Use Efficiency," *Irrigation Science*, vol. 42, no. 1, pp. 45–58, 2024.

[3] Y. An et al., "Impact of Irrigation Scheduling on Crop Productivity and Water Savings," *Agricultural Water Management*, vol. 265, p. 107530, 2022.

[4] A. Soussi, E. Zero, R. Sacile, D. Trinchero, and M. Fossa, "Smart Sensors and Smart Data for Precision Agriculture: A Review," *Sensors*, vol. 24, no. 8, p. 2647, 2024.

[5] R. Zhang, Y. Wang, and L. Chen, "A Comprehensive Review of Digital Twin Technology in Agriculture," *Agriculture*, vol. 15, no. 2, p. 301, 2025.

[6] T. Y. Melesse, J. Kang, and P. Nam, "Digital Twin-Based Applications in Crop Monitoring," *Computers and Electronics in Agriculture*, vol. 216, p. 108488, 2025.

[7] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[8] Republic of Türkiye, "Twelfth Development Plan (2024–2028)," Presidency of Strategy and Budget, Ankara, 2023.

[9] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," A/RES/70/1, 2015.

[10] T. Ojha, S. Misra, and N. S. Raghuwanshi, "Wireless sensor networks for agriculture: The state-of-the-art in practice and future challenges," *Computers and Electronics in Agriculture*, vol. 118, pp. 66–84, 2015.

[11] B. Zhang and L. Meng, "Energy Efficiency Analysis of Wireless Sensor Networks in Precision Agriculture Economy," *Scientific Programming*, vol. 2021, Article ID 8346708, 2021.

[12] Irga, F. R. Ikhwannul, and A. Zuchriadi, "Analysis of Power Consumption Efficiency of ESP Microcontroller Operation in the Implementation of IoT-Based Agricultural Monitoring Systems," in 2023 10th International Conference on ICT for Smart Society (ICISS), Bandung, Indonesia, 2023, pp. 1–5.

[13] N. K. Mohd Khalid, N. Jamal, F. Mustafa, N. A.[1] Zambri, and M. S. R. Abdullah, "Solar Power IoT Based Smart Agriculture System Using NodeMCU ESP32," Progress in Engineering Application and Technology (PEAT), vol. 5, no. 1, pp. 1–10, 2024.

[14] Espressif Systems, "Inter-Integrated Circuit (I2C)," ESP-IDF API Reference. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2c.html

[15] E. T. Engman, "Soil Moisture," in *Remote Sensing in Hydrology and Water Management*, G. A. Schultz and E. T. Engman, Eds. Berlin, Heidelberg: Springer, 2000, pp. 197–216.

[16] R. Rowe, *Soil Moisture Measurement Lab Manual*, Biosystems Engineering, Auburn University, 2018.

[17] S. K. Sahoo, "Capacitive Soil Moisture Sensor for Smart Irrigation: A Review," *IEEE Sensors Journal*, vol. 19, no. 14, pp. 5374-5383, 2019.

[18] Sensirion AG, "Datasheet SHT3x-DIS: Humidity and Temperature Sensors," Version 6.0, Sensirion AG, Stäfa, Switzerland, 2023.

[19] Tan, M., & Le, Q. (2019). EfficientNetV2: Smaller Models and Faster Training Proceedings of the 36th International Conference on Machine Learning (ICML) https://proceedings.mlr.press/v139/tan21a/tan21a.pdf

[20] Q. Wang, Y. Ma, K. Zhao, and Y. Tian, "A Comprehensive Survey of Large Language Models in Agriculture: Applications, Challenges, and Future Directions," *IEEE Access*, vol. 12, pp. 11200-11220, 2024.

[21] S. Kummar, F. S. Al-Aani, H. Kahtan, M. J. Darr, and H. Al-Bashiri, "Data Visualisation for Smart Farming Using Mobile Application," *International Journal of Computer Science and Network Security*, vol. 19, no. 11, pp. 174-180, 2019.

[22] C. Verdouw et al., "Digital twins in smart farming," *Agricultural Systems*, vol. 189, p. 103046, 2021.

[23] A. Sharma et al., "Mobile Apps for Farmers: Adoption and Usability in Smart Agriculture," *Computers and Electronics in Agriculture*, vol. 179, p. 105845, 2021.

[24] M. Wójcik and P. Brudło, "Rapid Mobile Prototyping for Agriculture Using Expo and React Native," *SoftwareX*, vol. 18, p. 101088, 2022.

[25] S. Laha, B. K. Pattanayak, and S. Pattnaik, "IoT-Based Real-Time Carbon Monitoring in Urban and Agricultural areas," in *Advances in Environmental Engineering and Green Technologies*, IGI Global, 2022.

[26] A. Mahato, "Greenhouse gas emission from agriculture and their mitigation strategies," *International Journal of Agriculture Extension and Social Development*, vol. 7, no. 7, pp. 188–198, 2024.

[27] E. A. Abioye et al., "Precision irrigation management using machine learning and digital farming solutions," *AgriEngineering*, vol. 4, no. 1, pp. 70–103, 2022.

[28] D. Neocleous, G. Kotsiras, and G. K. Georgiou, "Applying IoT sensor-based ferti-irrigation practices to enhance water and nutrient use efficiency in intensive potato cropping systems," *Frontiers in Agronomy*, vol. 7, 2025.

[29] M. A. Khan and M. S. Khan, "Smart farming and IoT-based agricultural solutions: A review," *IEEE Access*, vol. 9, pp. 125297–125314, 2021.

[30] N. K. Nawandar and V. R. Satpute, "IoT based low cost and intelligent module for smart irrigation system," *Computers and Electronics in Agriculture*, vol. 162, pp. 979–990, 2019.

[31] C. Pylianidis, S. Osinga, and S. Athanasiadis, "Introducing digital twins to agriculture," *Computers and Electronics in Agriculture*, vol. 184, p. 105942, 2021.

2. SRS

2.1. Introduction

2.1.1. Purpose

The purpose of this Software Requirements Specification is to define the objectives and functional boundaries of the TARAS system. TARAS is designed to support farmers in making accurate, data-driven decisions about irrigation, plant health and overall field management. The system integrates distributed IoT sensors to monitor soil and environmental conditions, applies ML models to generate zone-specific irrigation recommendations and incorporates computer-vision-based disease assessment to help with the early detection and prevention of crop loss.

This document specifies the requirements for the TARAS mobile application, which provides the primary user interface for real-time field visualization through a digital twin, personalized notifications, decision support and sustainability tools such as carbon-footprint estimation. The purpose of this SRS is to define the software's goals, intended functions,

and constraints so that all stakeholders share a common understanding of the system to be developed.

2.1.2. Scope

The scope of this project includes developing an end-to-end precision-agriculture system that collects soil and environmental data through IoT sensor nodes and transmits them directly to a cloud backend via Wi-Fi. The collected data are stored in a cloud database and processed using Real-world based algorithmic calculations for irrigation prediction, computer-vision algorithms for disease detection and a large language model for decision support. A mobile app provides farmers with digital-twin visualizations of soil moisture levels, real-time alerts, recommendations and carbon-footprint estimation.

2.2. General Description

2.2.1. Glossary

Term	**Definition**
Farmer	The primary user of TARAS who manages the agricultural field, monitors irrigation, and responds to alerts using the mobile application.
Stakeholder	An organization or representative entity such as an agricultural cooperative, farmer association, or advisory body that uses aggregated and anonymized TARAS outputs for monitoring field performance, supporting farmers, and improving collective decision-making.
Digital Twin	A virtual representation of the physical agricultural field that integrates real-time sensor data, spatial information, and analytical models to simulate field conditions and support decision-making.
Field	The physical agricultural land managed by the farmer, where crops are cultivated and monitored using TARAS.
Field Zone	A distributed embedded device deployed in the field that collects data and transmits it wirelessly to the gateway.
IoT Sensor Node	A central device that aggregates data from multiple IoT sensor nodes and forwards it to the backend system via the internet.
ESP32	A low-power microcontroller with integrated Wi-Fi and Bluetooth capabilities, used as the main controller in IoT sensor nodes for data acquisition and communication.
SHT	The name of the series of digital temperature and humidity sensor products that provides high-accuracy environmental measurements via an I²C communication interface.
Capacitive Soil	A sensor that measures soil moisture levels by detecting changes

Moisture Sensor	in capacitance caused by water content in the soil, offering improved durability over resistive sensors.
Telemetry	The automated process of collecting measurement data from remote sensors and transmitting it to a central system for monitoring and analysis.
Time-Series Data	Data collected sequentially over time intervals, such as sensor readings, used to analyze trends, patterns, and temporal changes in field conditions.
Irrigation Recommendation Engine	An algorithm based component that analyzes sensor data, historical records, and crop requirements to generate optimized irrigation suggestions.
Disease Detection Module	A computer-vision component that analyzes leaf images using CNN models to identify plant diseases, estimate severity, and provide early alerts to the farmer.
LLM-based Advisory System	A decision-support component that uses a Large Language Model to generate natural-language guidance for farmers by interpreting sensor data, irrigation recommendations, and disease detection results.
Carbon Footprint	A calculated metric representing the estimated greenhouse gas emissions associated with agricultural activities such as irrigation, fertilizer use, and energy consumption.
REST API	A stateless application programming interface that enables communication between system components using standard HTTP methods and structured data formats such as JSON.
Backend	The server-side components of the TARAS system responsible for data storage, processing, analytics, and integration with machine-learning models.
Frontend	The user-facing components of the TARAS system, including the mobile application interface through which users view data, receive alerts, and interact with recommendations.
PostgreSQL	An open-source, ACID-compliant relational database management system that supports advanced features such as JSONB, extensions, strong indexing, and complex queries, making it suitable for scalable and data-intensive applications.
TimescaleDB	A PostgreSQL extension optimized for time-series data. It uses hypertables to efficiently store, query, and compress IoT sensor readings.
PostGIS	A spatial database extension for PostgreSQL that adds support for geographic objects, enabling storage and analysis of field zones, polygons, sensor locations, and other geospatial data used in the digital twin of the farm.
CNN(Convolutional	A deep-learning model designed for image analysis, used in this

Neural Network)	project to detect plant diseases by classifying leaf images and identifying visual patterns of stress or infection.
ML Model	A trained computational model that learns patterns from data and generates predictions or classifications, such as irrigation needs or disease likelihood.
Inference	The process of applying a trained machine-learning model to new input data to generate predictions or outputs.
Store-and-Forward	A communication strategy in which data is temporarily stored locally when connectivity is unavailable and transmitted once the connection is restored.
Deep Sleep Mode	A low-power operating state of microcontrollers in which most system functions are disabled to conserve energy between data collection cycles.
JWT(JSON Web Token)	A secure, compact token format used for authentication and authorization between the frontend and backend components of the system.
Telemetry Packet	A structured data unit containing sensor readings, timestamps, and device identifiers transmitted from sensor nodes to the gateway or backend.
Threshold	A predefined limit or value that, when exceeded or violated, triggers system alerts or recommendations.
Normal Operating Conditions	Conditions in which the system operates with stable hardware, expected network connectivity, and no major external disruptions such as prolonged power outages or hardware failures.
Guidance Session	A short onboarding or instructional interaction designed to help users understand basic system functionality.
Evapotranspiration(ET_0)	ET_0 is a standardized measure of atmospheric water demand, representing the evapotranspiration rate from a hypothetical reference surface under given meteorological conditions. It reflects the effect of weather conditions on water loss independently of crop type.
Crop Coefficients(Kc)	Kc is a dimensionless coefficient used to adjust reference evapotranspiration (ET_0) to account for crop type and growth-stage characteristics, enabling estimation of crop-specific water demand.
ACID	A set of four fundamental properties—Atomicity, Consistency, Isolation, and Durability—that ensure database transactions are processed reliably and maintain data integrity, even in the presence of errors or system failures.
I^2C	A synchronous serial communication protocol that uses two lines (Serial Data Line (SDA) and Serial Clock Line(SCL)) to connect a microcontroller with multiple peripheral devices, enabling simple, low-power data exchange over short distances in embedded

	systems.
ADC	A hardware component that converts continuous analog signals (such as voltage levels from a soil-moisture sensor) into discrete digital values that can be processed by a microcontroller.
DAP	A time-based index that represents the number of days elapsed since a crop was planted, commonly used in agriculture to track crop growth stages, schedule irrigation or fertilization, and map crop coefficient (Kc) profiles to specific development periods.

2.2.2. User Characteristics

The system is designed with two key user groups in mind: the primary user (farmer), and external institutional stakeholders (such as cooperatives or environmental reporting bodies). Each group has distinct abilities, responsibilities and interaction needs, which shape the system's interface and functionality.

2.2.2.1. Primary User: Farmer

Profile: Small-to-medium-scale farmers who rely on their experience for regular irrigation and crop-health decisions. They interact with the system mostly in outdoor field conditions.

Age Range: Typically 20–65, though many may be older and accustomed to traditional farming practices.

Technical Skills: Low-to-moderate comfort with digital tools. Interfaces shall be simple, intuitive and optimized for mobile use, with minimal steps and clear visual cues.

Work Environment: Outdoor, field-based conditions with limited connectivity and variable lighting.

Key Characteristics:

- Relies heavily on intuitive decision-making
- Faces challenges like water scarcity, plant disease uncertainty
- Needs clear, non-technical explanations rather than complex analytics
- Requires solutions that are low-cost, reliable and field-tolerant

2.2.2.2. External Party: Stakeholders

Profile: Stakeholders are organizations that have access to the system but with highly restricted, read-only permissions. They do not participate in daily decision-making or sensor management. Their role is limited to viewing summaries and sustainability-related metrics for monitoring or reporting purposes.

Examples:

- Agricultural cooperatives
- Sustainability and environmental monitoring institutions
- Government programs that track water usage, efficiency, or carbon impact

Interaction Model: Stakeholders log into a summary-focused dashboard that displays high-level insights such as water consumption trends, irrigation efficiency metrics and carbon-footprint estimates. They don't upload data, adjust system settings or interact with field-level features such as disease detection or digital-twin visualizations. Their interaction is passive and observational.

2.2.3. Overview of Functional Requirements

TARAS provides an integrated set of functional capabilities to support data-driven decision-making in smart agriculture. At its core, the system continuously collects soil and environmental data from distributed IoT sensor nodes deployed in the field. These data are transmitted directly to the cloud backend via Wi-Fi connectivity, where they are stored as time-series records and made available for analysis, visualization, and decision-support processes.

Based on the collected data, TARAS performs analytical and decision-support functions, including machine-learning–based irrigation recommendations and computer-vision–based plant disease detection.

The mobile app serves as the primary interaction tool for all users to visualize real-time and historical field conditions through a digital twin, receive recommendations, upload plant images and interact with advisory features. In addition, the system supports sustainability-related functions such as carbon footprint calculation when farmers input necessary information.

Together, these functional requirements define an end-to-end system that integrates sensing, communication, data processing, analytics, visualization and user interaction to deliver reliable and actionable agricultural decision support.

2.2.4. General Constraints and Assumptions

2.2.4.1. Constraints

- Hardware Constraints:

  The sensor nodes operate on ESP32 microcontrollers with limited RAM and processing capability, which restricts complex on-device computation. As a result, most data processing, analytics, and decision-making tasks are offloaded to cloud-based services accessed directly via Wi-Fi connectivity.

  All soil-moisture, temperature, and humidity sensors must operate within strict voltage ranges because of the ESP32's output limitations (typically 3.3 V), limiting hardware choices[1]. Since all sensor nodes are battery-powered, energy consumption must remain minimal to ensure long-term field deployment.

- Communication Constraints:

  The system relies on Wi-Fi connectivity for direct data transmission from ESP32 sensor nodes to the cloud backend. Although Wi-Fi offers higher bandwidth compared to low-power long-range protocols, its effective range and reliability can be limited in rural agricultural environments. Variations in signal strength and intermittent internet access may lead to temporary data transmission failures, requiring retry mechanisms and buffering strategies at the node level to ensure reliable synchronization with the cloud services.

- Environmental Constraints:
  Devices that are all field-deployed must withstand harsh outdoor conditions such as humidity, temperature extremes, rainfall, dust, and direct sunlight. Variations in soil composition and microclimatic differences introduce noise into sensor readings, potentially affecting predictive model accuracy.

- Software & Data Constraints:
  Available training data for machine-learning models is limited and may not represent all crop types or environmental contexts, reducing early-stage model accuracy[2].

  Real-time inference must meet latency requirements appropriate for on-field decision-making, which imposes limits on model size and computational complexity.

- Operational Constraints:

Continuous internet connectivity cannot be guaranteed for either the gateway or the user's mobile device, requiring offline-tolerant system behavior. The mobile application must support a wide range of user devices, specifically Android versions 7.0–16.0 and iOS versions 15.1–26.1.

- Budget & Resource Constraints:
  High-quality agricultural sensors and reliable power-management components represent a significant portion of the system cost. In addition, ensuring continuous Wi-Fi or internet connectivity in agricultural environments may require auxiliary networking equipment, such as mobile hotspots or cellular modems, which can increase deployment and operational expenses.

  Real-world field access for testing may be limited, which may reduce the frequency and duration of validation cycles.

2.2.4.2. Assumptions

- Sensor Deployment Assumptions:
  Sensors are assumed to be properly installed, calibrated, and positioned in pre-determined areas of the field. Battery-powered sensor nodes are assumed to maintain sufficient charge throughout their scheduled data collection intervals.

- Communication Assumptions:
  The sensor nodes are assumed to operate within the coverage range of a stable Wi-Fi network or an alternative internet access point. Temporary connectivity outages are expected in agricultural environments; however, it is assumed that retry and buffering mechanisms will allow buffered data to be transmitted successfully once connectivity is restored, ensuring eventual synchronization with the cloud backend without permanent data loss.

- User Behavior Assumptions:
  Farmers are assumed to regularly monitor the mobile application and take appropriate actions based on irrigation recommendations or disease analysis results. External stakeholders are assumed to require only high-level, read-only access for monitoring and reporting purposes.

- System Assumptions (Irrigation & Disease Detection):
  Soil-moisture, temperature, and humidity sensor readings are assumed to accurately represent current field conditions for irrigation decision-making.
  The rule-based irrigation logic assumes that moisture trends and threshold crossings reliably indicate crop water stress under normal operating conditions.

Images uploaded for disease detection are assumed to be clear, correctly captured, and representative of the plant's current health state.

Environmental anomalies or abnormal conditions (e.g., extreme weather, sensor faults, or equipment malfunctions) are assumed to be detectable through irregular sensor behavior or inconsistent image analysis results.

Calibration parameters and decision rules are assumed to remain valid for the defined crop type and field conditions unless manually updated.

- Mobile Application Assumptions:
  The mobile application assumes intermittent but recurring Internet connectivity to retrieve sensor data, digital twin updates, and ML-driven recommendations.

2.3. Specific Requirements

2.3.1. Interface Requirements

2.3.1.1. Hardware Interface Requirements

The TARAS system interacts with a distributed network of IoT sensor nodes that communicate directly with cloud-based services via Wi-Fi connectivity, requiring consistent electrical, communicational and operational compatibility across all hardware components. Each sensor node incorporates an ESP32 microcontroller, which provides the necessary GPIO pins, I²C bus capabilities and low-power operational modes required for continuous monitoring. The hardware interface must support stable connections to capacitive soil-moisture sensors, SHT31 temperature-humidity units and additional digital sensors through standardized I²C communication. All sensors must operate within the microcontroller's voltage range (3.0–3.6 V) and support uninterrupted periodic sampling[1].

The wireless communication interface is based on the ESP32's integrated Wi-Fi module, which enables direct data transmission from sensor nodes to the cloud backend. The system must ensure reliable Wi-Fi connectivity, appropriate network configuration and secure data exchange using standard IP-based communication protocols. Variations in signal strength or temporary connectivity interruptions in agricultural environments must be handled through

retry and buffering mechanisms at the device level to maintain data integrity.

Power interfaces are also fundamental. Each sensor node must operate reliably on 3.7 V Li-Ion or Li-Po battery systems. The hardware must support energy-saving modes and deep-sleep states. All components must be physically robust and weather-resistant to withstand outdoor conditions. The complete hardware interface architecture must guarantee stable electrical operation, reliable wireless communication and long-term durability.

2.3.1.2. Communication Requirements

The communication infrastructure of the TARAS system is designed as a scalable and reliable architecture that supports continuous IoT telemetry streaming, real-time ML inferences, disease detection and mobile app interactions. At the field level, distributed ESP32-C6 sensor nodes collect soil-moisture and environmental data from connected sensors and transmit the telemetry data directly to cloud-based services via Wi-Fi. This direct communication approach reduces system complexity while enabling timely data delivery for analysis and decision support.

The system’s data infrastructure is built around a cloud-hosted PostgreSQL database, deployed on a cloud platform such as AWS. This backend architecture supports reliable data storage, efficient querying of historical records and real-time access for analytical services[3].

TARAS's backend architecture will consist of two layers: the cloud-based main backend and the service layer that manages mobile clients. The main backend consists of API services running on a commercial cloud hosting service such as AWS. Sensor data from the nodes first reaches the API layer, where it undergoes formatting before being transferred to time series tables. The backend service provides all the necessary API endpoints for users for them to view the digital twin map, receive irrigation recommendations and upload images for disease detection. This layer will include “caching” strategies to ensure devices can operate in intermittent internet conditions by keeping any recent data that has failed to transmit. Authentication will be provided via industry standard solutions such as AWS Cognito, which ensures two-way security[4]. The data model includes a structured schema consisting of sensor nodes, time-series telemetry records, field regions, digital twin snapshots, irrigation recommendation processing history, disease detection outputs and user profiles. All tables are designed to maintain ACID integrity and relationship

constraints, identification mechanisms and data validation rules will be enforced. Image files used for disease detection will be stored in cloud storage, not directly in the database; only the file path and classification outputs will be stored in the database.

The system relies on the integrated Wi-Fi capabilities of ESP32 sensor nodes for wireless communication with the cloud backend[1]. Each node transmits collected soil-moisture and environmental telemetry directly over IP-based protocols, enabling reliable data delivery without the need for long-range radio transceivers or intermediate gateway hardware. This design simplifies the communication stack while maintaining sufficient bandwidth for periodic sensor updates and system monitoring.

To ensure continuity and data integrity in rural agricultural environments, where intermittent internet connectivity may occur, the system implements device-level retry and buffering mechanisms. Sensor nodes temporarily store measurements in local memory when transmission attempts fail and automatically resend buffered data once connectivity is restored. This approach ensures eventual consistency and prevents permanent telemetry loss without introducing additional gateway complexity.

Communication between the sensor nodes and the backend services is performed using secure HTTP APIs over Wi-Fi. Data packets are transmitted through encrypted channels such as HTTPS, ensuring confidentiality and integrity during transmission. On the cloud side, received telemetry is validated, stored and made available to analytics, ML and visualization services. This communication architecture provides a robust, secure and scalable end-to-end data flow from field-level sensors to cloud intelligence.

2.3.1.3. System and Backend Requirements

The TARAS system's data infrastructure is built on a scalable and reliable architecture designed to support continuous IoT telemetry streaming, real-time ML inferences, a disease detection module and mobile app interactions. The system's core database will be implemented using PostgreSQL, providing a robust relational foundation for structured agricultural data management[3].

To efficiently handle sensor telemetry, the architecture supports the use of the TimescaleDB extension, which may be enabled to improve performance for time-based queries, historical analysis and aggregation-heavy workloads[5]. When utilized, this extension facilitates efficient storage, partitioning and compression of sensor data. When spatial field analysis is required, the PostGIS extension can be enabled to support geospatial queries and

region-based operations, ensuring seamless integration with the digital twin component[6].

TARAS's backend architecture consists of two logical layers: an AWS-based main backend and a service layer responsible for managing mobile client interactions. The main backend includes API services deployed on AWS and a PostgreSQL database hosted on AWS. Sensor data collected from field gateways is first received by the API layer, where it undergoes validation and formatting before being persisted in the database.

The TARAS backend architecture will utilize Node.js as the server-side runtime environment for managing RESTful APIs, processing incoming sensor data and coordinating interactions between the mobile application, database services and ML modules. The backend exposes RESTful API endpoints that allow users to access digital twin maps, receive irrigation recommendations, upload plant images for disease detection and receive system notifications.

2.3.1.4. Software Interface Requirements

The TARAS software architecture relies on a set of coordinated interfaces that connect the ML components, backend services and mobile app. These interfaces ensure that irrigation prediction, disease diagnosis and the LLM-based advisory system operate consistently. The system's ML modules communicate with the backend through well-defined API contracts, enabling the exchange of structured sensor data, user input and model outputs.

The irrigation-prediction module operates through a time-series interface with the cloud database. The backend supplies the decision engine with aggregated historical sensor data, including soil moisture trends, environmental conditions and past irrigation records. Based on these inputs, the model produces a recommendation object that specifies whether irrigation is required, along with suggested timing and duration parameters and supporting indicators used for explainability.

The disease-detection component interacts with the system through an image-processing interface exposed by the backend. The mobile app uploads leaf images in compressed JPEG/PNG format and the backend forwards these images to the CNN inference engine. The model returns a classification label, confidence score and an explanation map if available.

The LLM-based advisory module operates on top of the system's analytical outputs and requires a specialized interface to retrieve the latest sensor data, irrigation predictions and disease-diagnosis

results. The backend provides the model with a context package containing environmental data, recent recommendations and user-submitted information such as irrigation history. The LLM returns a natural-language advisory response, including explanations, warnings or scenario-based comparisons. This interface allows the mobile app to request advice in real time while ensuring that the LLM's outputs remain consistent with the most recent sensor readings and model predictions.

The mobile interface uses the designated API endpoints to retrieve ML results, send image data and request advisory messages. All interfaces must maintain compatibility with the PostgreSQL-backed data layer and must handle synchronization between the nodes, cloud and mobile app without data loss. These software interfaces also ensure that the ML-driven components such as disease detection and LLM, integrate seamlessly with the user-facing mobile app and the backend.

2.3.1.5. User Interface Requirements

The TARAS mobile application serves as the primary interaction layer between the farmer and the underlying sensing, prediction and visualization ecosystem, requiring a cohesive and intuitive interface that enables efficient decision-making under varying environmental conditions. Overall, the user interface must maintain responsiveness across diverse screen sizes, uphold visual consistency aligned with the system's design identity and model outputs into actionable, farmer-friendly insights.

The user interface is structured around a persistent navigation bar that provides direct switching between tabs that will enable the user to access their desired function such as Overview, Timetable, Disease Detection and Settings. A global floating button remains visible across these primary screens to engage the AI assistant overlay. This conversational interface processes natural language queries and enables users to execute "What-if" scenario simulations.

Access to the application is managed through a role-based login interface that authenticates distinct user types, including farmers and stakeholders and adjusting interface permissions and data visibility.

The central dashboard functions as a digital twin, rendering an interactive 2D/3D visualization of the field. This model maps real-time sensor inputs, specifically soil moisture and temperature, to color-coded zones within the canvas. Users navigate this digital representation by dragging to rotate the perspective and tapping specific sectors to isolate detailed data. To support immediate

decision-making, the visualization highlights zones where readings fall outside optimal ranges, providing visual alerts for conditions such as drought stress or over-irrigation.

The disease detection module utilizes a live camera viewfinder and image upload, enabling users to capture plant imagery for on-screen classification of disease types and severity. Additionally, the specific timetable interface presents a schedule of irrigation and fertilization tasks, while also providing input fields for logging fuel, energy and fertilizer consumption to support carbon footprint calculations.

Navigation shall remain simple and consistent across Android and iOS devices, using a navigation bar to provide fast access to primary screens such as dashboard, digital twin, disease detection and settings.

2.3.2. Detailed Description of Functional Requirements

- Microcontroller - Sensor Interface

Name	**Microcontroller - Sensor Interface**
Purpose / Description	The sensor interface enables reliable acquisition of soil moisture, temperature, humidity and environmental data from IoT sensor nodes. Each ESP32-C6 node reads SHT31 sensors via I²C and the soil-moisture probe via ADC. The interface ensures proper electrical levels (3.0–3.6 V), periodic sampling and noise-resistant communication.
Inputs	● I²C readings from SHT31 temperature-humidity sensor ● ADC readings from capacitive soil-moisture sensor ● I²C/ADC configuration parameters (address, bus speed, ADC resolution, sampling rate) ● Regulated 3.0–3.6 V power input
Processing	1. Initialize I²C bus and ADC at boot 2. Perform periodic sampling of all sensors 3. Verify sensor presence via I²C acknowledgment 4. Apply calibration and scaling to raw readings 5. Filter noisy values using smoothing/averaging 6. Package processed data into Wi-Fi-ready telemetry messages 7. Timestamp each measurement for cloud synchronization
Outputs	● Validated sensor data (soil moisture, humidity and etc) ● Structured telemetry payloads for Wi-Fi transmission to the cloud backend ● Sensor status indicators (active, missing, calibrated)

Name	Microcontroller - Sensor Interface
Error Handling	● Retry I²C communication on missing acknowledgment ● Mark and suppress physically invalid readings ● Reset I²C bus after collisions or noise ● Suspend sampling under low-voltage conditions ● Log diagnostic flags locally for cloud-side monitoring and debugging

- Gateway Hardware Interface Functional Requirement

Name	Gateway Hardware Interface
Purpose / Description	The sensor node communication interface enables reliable transmission of processed sensor data from ESP32 nodes directly to the cloud backend via Wi-Fi. This interface is responsible for network initialization, secure data forwarding and temporary buffering to ensure continuity under intermittent connectivity conditions.
Inputs	● Structured sensor telemetry generated by the sensor interface ● Wi-Fi network credentials and configuration parameters ● System time for timestamping ● Device memory for temporary buffering
Processing	1. Initialize Wi-Fi connectivity at startup 2. Establish secure communication channels with cloud services 3. Package sensor data into standardized messages (e.g., JSON) 4. Timestamp and format data for cloud ingestion 5. Transmit telemetry via HTTP or MQTT over Wi-Fi 6. Buffer data locally during connectivity loss and retry transmission upon reconnection
Outputs	● Cloud-delivered sensor telemetry ● Transmission status indicators (success, retry, failure) ● Diagnostic logs for connectivity and transmission events
Error Handling	● Retry Wi-Fi connection on initialization or runtime failure ● Queue telemetry locally when transmission fails ● Discard or flag stale data after configurable retry limits ● Reset network interface if connectivity becomes unresponsive

Name	Gateway Hardware Interface
	● Report persistent failures to cloud-side monitoring services

- Power Interface Functional Requirement

Name	Power Interface
Purpose / Description	The power interface ensures stable and efficient energy delivery to ESP32 sensor nodes and connected sensors using a 3.7 V Li-Ion/Li-Po battery regulated to safe operating levels. It supports low-power modes, deep-sleep operation and voltage stability to maximize battery lifetime. The interface also prevents undervoltage, brownout condition, and power-related data loss.
Inputs	● 3.7 V Li-Ion / Li-Po battery input ● Regulated 3.0–3.6 V output for ESP32-C6 and sensors ● Power-management configuration (sleep intervals, wake-up triggers) ● Voltage and current readings from onboard regulators
Processing	1. Regulate battery voltage to component-safe operating levels 2. Manage deep-sleep and low-power operational modes of the ESP32 3. Monitor voltage stability to detect undervoltage and brownout conditions 4. Allocate power to sensors and communication subsystems based on sensing and Wi-Fi transmission cycles 5. Log power-related events (sleep, wake-up, low-voltage warnings)
Outputs	● Stable power delivery to the microcontroller and sensors ● Power-state indicators (active, sleep, low voltage) ● Logged power-performance data for system diagnostics
Error Handling	● Enter safe low-power mode under undervoltage conditions ● Suspend sensing and wireless transmission when power becomes unstable ● Trigger automatic system restart after a detected brownout event ● Log critical power faults for cloud-side monitoring and analysis

- Environmental / Enclosure Interface Functional Requirement

Name	**Environmental / Enclosure**
Purpose / Description	The environmental interface ensures that all hardware components operate reliably in outdoor conditions by providing protection against moisture, dust, UV exposure, temperature variation and mechanical stress. Weather-resistant enclosures, sealed connectors and durable materials maintain sensor and node integrity during long-term deployment.
Inputs	● Outdoor environmental exposure(rain, dust, UV, heat). ● Mechanical stress from handling or field conditions. ● Moisture and particulate ingress risks. ● Thermal load from sun exposure.
Processing	1. Seal enclosures to prevent water and dust ingress. 2. Regulate airflow or thermal dissipation to maintain safe operating temperature. 3. Shield sensors and electronics using protective casing and strain-relieved cabling. 4. Prevent corrosion through coated PCBs or protected connectors.
Outputs	● Stable device operation under outdoor conditions. ● Protected internal electronics and sensors. ● Maintained enclosure integrity and mechanical stability. ● Environmental health status indicators.
Error Handling	● Detects abnormal temperature/humidity inside enclosure. ● Trigger warnings or reduced operation if thermal or moisture limits are exceeded. ● Log environmental fault conditions for maintenance. ● Enter protective mode or shut down to prevent hardware damage.

- Wi-Fi Uplink Communication – Functional Requirement

Name	**Wi-Fi Uplink Communication**
Purpose / Description	Provides direct wireless data transmission from ESP32 sensor nodes to the cloud backend using Wi-Fi. This interface ensures

Name	Wi-Fi Uplink Communication
	reliable IP-based communication for periodic and event-driven telemetry delivery.
Inputs	● Sensor payloads generated by ESP32 sensor interface ● Wi-Fi network credentials and configuration parameters ● Node identifier and message metadata ● System time for timestamping
Processing	1. Initialize Wi-Fi interface at node startup 2. Establish secure network connection to the backend services 3. Package sensor data into standardized message formats (e.g., JSON) 4. Transmit telemetry periodically or event-driven via HTTP or MQTT 5. Monitor connection status and transmission acknowledgments 6. Buffer outgoing data locally during temporary connectivity loss
Outputs	● Successfully transmitted sensor data received by the cloud backend ● Transmission status indicators (success, retry, failure) ● Structured telemetry payloads available for storage and analytics
Error Handling	● Retry Wi-Fi connection on network or authentication failure ● Reattempt data transmission upon timeout or backend unavailability ● Temporarily buffer telemetry when transmission fails ● Discard or flag stale data after configurable retry limits ● Log communication and connectivity errors for cloud-side diagnostics

- Store-and-Forward Communication Reliability Functional Requirement

Name	Store-and-Forward Communication Reliability
Purpose / Description	Ensures reliable data delivery in environments with intermittent internet connectivity by implementing local caching at the ESP3 sensor node level. When Wi-Fi or cloud transmission fails, sensor data is temporarily stored on the device and

Name	Store-and-Forward Communication Reliability
	automatically synchronized with the cloud backend once connectivity is restored, preventing telemetry loss.
Inputs	● Sensor data awaiting transmission ● Wi-Fi connectivity status of the sensor node ● Cloud backend availability status ● Local storage resources on the ESP32 (RAM/flash) ● Packet metadata (timestamps, sequence numbers)
Processing	1. Detect failed Wi-Fi or cloud transmission attempts 2. Store unsent data in local buffers on the sensor node 3. Periodically check network availability for retransmission 4. Synchronize buffered packets in chronological order once connectivity returns 5. Verify successful cloud acknowledgment before deleting local copies 6. Filter duplicate packets and ensure correct ordering using timestamps or sequence identifiers
Outputs	● Reliable, lossless delivery of sensor telemetry to the cloud backend ● Updated synchronization logs indicating queued, transmitted, and retried packets ● Continuous data flow despite connectivity interruptions
Error Handling	● Apply retry and backoff strategies when repeated transmission failures occur ● Manage buffer overflow by discarding or overwriting the oldest data based on policy ● Log cache write/read errors for diagnostics ● Validate buffered packets before retransmission to prevent data corruption

- Node-to-Cloud Communication – Functional Requirement

Name	Node-to-Cloud Communication
Purpose / Description	Enables secure and reliable transmission of sensor data directly from ESP32 sensor nodes to the cloud backend using HTTP or MQTT protocols over Wi-Fi. This interface ensures encrypted communication, standardized payload formatting and uninterrupted data flow despite intermittent internet availability.

Name	Node-to-Cloud Communication
Inputs	● Validated sensor data generated by the ESP32 node ● Cloud endpoint URLs and authentication parameters ● Wi-Fi network connectivity status ● Protocol configuration parameters (HTTP/MQTT) ● System timestamp for message scheduling
Processing	1. Format sensor data into standardized JSON or binary payloads 2. Establish secure communication channels using TLS/HTTPS 3. Transmit data via HTTP POST/MQTT publish operations 4. Monitor backend responses (ACKs, status codes) to confirm delivery 5. Temporarily cache unsent data on the device during connectivity loss 6. Automatically retransmit buffered data once network connectivity is restored 7. Log communication events, including latency, error codes
Outputs	● Successfully transmitted sensor data stored in the cloud backend ● Diagnostic logs indicating transmission success, failures, and response codes ● Updated local buffer status reflecting synchronized and pending telemetry
Error Handling	● Retry failed HTTP or MQTT transmissions ● Cache telemetry locally when internet connectivity is unavailable ● Handle TLS or authentication failures by suspending uploads until a secure channel is re-established ● Discard/reformat invalid payloads before retransmission

- Sensor Telemetry Storage and Management Functional Requirement

Name	Sensor Telemetry Storage and Management
Purpose / Description	Defines the system's responsibility to receive, store, manage and aggregate high-frequency sensor telemetry data in a time-series-oriented data infrastructure. The system shall ensure that collected sensor measurements can be reliably accessed and processed to support irrigation prediction, digital twin visualization and historical trend analysis.
Inputs	● Incoming telemetry packets from gateway: ○ soil_moisture

Name	Sensor Telemetry Storage and Management
	○ temperature ○ humidity ○ node_id ○ timestamp
Processing	1. Validate the structure and data types of incoming telemetry. 2. Automatically partition data by time and node identifier. 3. Apply compression policies to historical data chunks. 4. Generate continuous aggregates, including: ○ hourly soil moisture averages, ○ daily moisture trends, ○ temperature–humidity correlation metrics. 5. Expose optimized query interfaces for ml modules and digital twin services.
Outputs	● Telemetry records accessible through REST APIs. ● Summarized datasets for UI rendering, ML processing. ● Historical trend datasets for analytics and visualization.
Error Handling	● If a schema mismatch occurs, the telemetry record is marked as invalid and logged. ● If the database connection is unavailable, records are temporarily queued at the sensor nodes and forwarded once connectivity is restored.

- Disease Detection Database Recording Functional Requirement

Name	Disease Detection Database Recording
Purpose / Description	Stores the output of the disease detection module; including image path, classification, severity, confidence and timestamp, inside a structured disease results table in PostgreSQL.
Inputs	● Classification result from CNN model ● Cloud storage URL of user-uploaded image ● User ID (farmer) ● Device and field metadata
Processing	1. Validate disease result schema. 2. Insert record into disease results table. 3. Update disease statistics for the field. 4. Trigger notification system if severe disease detected.

Name	Disease Detection Database Recording
Outputs	● Database record (row) created. ● Disease history accessible via API. ● Push notification sent.
Error Handling	● If the database write operation fails, the system performs an automatic retry before reporting an error. ● If the classification output is missing, the system records the entry with the label “unidentified.”

- Digital Twin Data Assembly Functional Requirement

Name	Digital Twin Data Assembly
Purpose / Description	Supports the Digital Twin by managing field geometry, zone polygons and spatial queries using PostgreSQL + PostGIS.
Inputs	● Field boundary polygons ● Zone definitions (polygons) ● Sensor-node geolocation coordinates ● Latest moisture/temperature telemetry
Processing	1. Store and index spatial data 2. Map telemetry to nearest field zone 3. Generate zone-level moisture/temperature aggregates 4. Prepare structured datasets for heatmap rendering
Outputs	● Zone-level moisture distribution ● 2D/3D Digital Twin datasets ● Spatial queries for crop/zone analysis
Error Handling	● If a geometry record is invalid, it’s rejected and logged for review. ● If telemetry data arrives without a spatial reference, it is assigned to an “unmapped” zone. ● If a spatial index becomes corrupted, the system rebuilds the index to restore proper query performance.

- Disease Detection Functional Requirement

Name	Disease Detection Functional Requirement
Purpose / Description	Enables users to upload plant leaf images and receive disease classification using the system’s CNN-based image-processing

Name	Disease Detection Functional Requirement
	module. The module analyzes images, detects disease type/severity, generates confidence scores and provides recommendations.
Inputs	● User-uploaded plant leaf image. ● Optional metadata: crop type, field zone, timestamp. ● Pretrained CNN weights and preprocessing parameters. ● Stored disease classes in the database.
Processing	1. Validate image size, format, and resolution. 2. Preprocess image (resize, normalize, color correction). 3. Run CNN inference to classify disease type & severity. 4. Store classification output into PostgreSQL (disease table) with image path (cloud storage). 5. Forward result to mobile application for visualization.

Outputs	● Disease label
	● Severity score.
	● Confidence value.
	● API response to the mobile client.
	● Database entry containing:
	  ○ image_url
	  ○ classification
	  ○ confidence
	  ○ severity
	  ○ timestamp

Error Handling	● If the image format is invalid, the system returns an “unsupported file format” error. ● If the image file is corrupted, the system rejects the upload and records it in the logs. ● If the inference engine is unavailable, the system attempts a retry and then returns a error if the issue persists. ● If inserting the result into the database fails, the system logs the failure and returns a partial-success response to the client. ● If metadata is missing, the system proceeds using default assumptions defined by the backend.

- Irrigation Prediction Request & Response Handling Functional Requirement

Name	**Irrigation Prediction Request & Response**
Purpose / Description	Generates irrigation recommendations using a rule-based decision engine that evaluates aggregated sensor data, calibrated soil-moisture dynamics and predefined irrigation rules.
Inputs	● Soil moisture (time-series) ● Temperature and humidity ● Field metadata (crop type, soil type, irrigation method) ● Calibrated thresholds (field capacity, stress threshold) ● Historical irrigation markers

Processing	1. Retrieve latest time-series sensor data from PostgreSQL
	2. Smooth and validate soil-moisture signal
	3. Detect irrigation start and finish times using moisture-derivative rules
	4. Evaluate soil-moisture state against calibrated rule thresholds
	5. Apply irrigation rules to determine:
	  ● irrigation necessity
	  ● recommended duration
	  ● next irrigation timing
	6. Assign recommendation priority based on moisture deficit trend

Outputs	JSON recommendation object including: ● irrigation decision (apply / delay / skip) ● recommended irrigation duration ● suggested next evaluation time
Error Handling	● If sensor data is partially missing, use last valid stabilized segment ● If database is unavailable, cache request ● If rule evaluation fails, notify backend and return unavailable

- Role-Based Access Control & IU Authorization Functional Requirement

Name	**Role-Based Access Control & UI Authorization**
Purpose/Description	Defines role-based access rules for the TARAS mobile and web interfaces. The system controls which screens, features and data are visible or accessible to users based on their assigned role.
Inputs	● Authenticated User Session ● User Role ● Requested UI Screen or Feature ● Navigation Events
Processing	1. Role Identification: Determine the user's role after successful authentication. 2. Access Evaluation: Check requested screen or feature against role-based permission rules. 3. UI Rendering: Display only the interface components permitted for the user's role. 4. Navigation Filtering: Hide or disable navigation items that are not allowed for the current role. 5. Session Enforcement: Enforce access rules during navigation and screen transitions.
Outputs	● Role-appropriate UI layout. ● Accessible navigation menu based on user role. ● Authorized access to data, visualizations and actions.
Error Handling	● If a user attempts to access a restricted screen, redirect to an authorized default screen. ● If role information is missing or invalid, terminate the session and require re-authentication. ● If permission rules fail to load, restrict access to read-only or safe default views.

- Accessibility & Inclusive UI Support Functional Requirement

Name	**Accessibility & Inclusive UI Support**
Purpose / Description	Defines accessibility features of the TARAS user interface to ensure that the mobile application is usable by farmers and stakeholders with varying physical, visual, and cognitive abilities. The interface prioritizes clarity, readability, and ease of interaction under field conditions.
Inputs	● System Accessibility Settings (Font Size, Contrast) ● Device Accessibility Preferences

Name	Accessibility & Inclusive UI Support
	● User Interaction Events ● UI Rendering Context
Processing	1. Font Scaling: Adapt text size according to system-level font and accessibility settings. 2. Contrast Enforcement: The system shall ensure a minimum contrast ratio of 4.5:1 between text and background for all primary information elements displayed in the interface[7]. 3. The system shall ensure that all interactive UI elements have a minimum touch target size that was stated in Web Content Accessibility Guidelines[8]. 4. Visual Simplification: The system shall limit the number of primary interactive elements displayed on a single screen to a maximum of five, prioritizing critical actions and information. 5. Consistent Feedback: The system shall provide immediate visual feedback for user interactions, including pressed, selected, loading, and error states.
Outputs	● Accessible UI layout with readable text and clear contrast. ● Enlarged and touch-friendly interactive elements. ● Simplified screens optimized for field usage.
Error Handling	● If accessibility preferences cannot be applied, load the default high-contrast and large-text layout. ● If UI rendering fails due to accessibility overrides, reset to a safe default layout. ● If device accessibility settings change during runtime, re-render the UI to reflect updated preferences.

- Mobile Application Framework & Navigation Functional Requirement

Name	Mobile Application Framework & Navigation
Purpose / Description	Defines the initialization, localization and navigation structure of the TARAS application. It establishes the primary user interface shell, ensuring consistent navigation and language support across Android and iOS devices.
Inputs	● System Language Settings (Locale) ● System Theme Preference (Light/Dark) ● User Credentials (Login/Skip) ● Touch Interaction Events
Processing	1. Localization: Detect device system language on startup apply Turkish(TR) by default or English(EN) if explicitly set. 2. Theme Application: Apply the color schema (Light/Dark) based on system settings or user override. 3. Authentication Routing: Direct user to Login Screen or load

Name	Mobile Application Framework & Navigation
	main dashboard if a valid session token exists. 4. Navigation Mounting: Render the persistent bottom navigation bar (dashboard, timetable, settings). 5. Overlay Initialization: Attach the Global Floating Action Button (FAB) for the AI Assistant to the root view.
Outputs	● Localized Main UI Shell. ● Active Navigation Stack. ● Visual Theme (Colors/Fonts).
Error Handling	● If the system language is not supported, automatically fallback to Turkish. ● If a screen fails to mount, reset the navigation stack to the dashboard.

- Carbon Footprint Calculation & Display

Name	Carbon Footprint Calculator
Purpose / Description	The system shall provide a carbon footprint calculation feature that estimates the agricultural carbon dioxide emission impact of a field based on farmer-provided inputs. The calculated footprint shall be associated with the corresponding user and field and presented to the user through the mobile application.
Inputs	● Farmer responses to predefined multiple-choice questions ● Optional numeric inputs related to energy, fuel, or fertilizer usage
Processing	1. UI Form Handling ● Display a separate “Carbon Footprint” section in the mobile app. ● Collect answers via multiple-choice and numeric fields. ● Validate that required questions are answered before submission. 2. Backend Validation ● Validate schema (required fields, allowed options, numeric ranges). ● Map multiple-choice selections to internal coefficients. 3. CO_2e Calculation ● Compute component emissions (example components): ○ Irrigation energy emissions ○ Fertilizer-related emissions ● And then produce:

Name	Carbon Footprint Calculator
	○ Total_CO2e (kg CO_2e) ○ Breakdown 4. Store a record in carbon_footprint_results table, which possible row such as: ● user_id, total_co2e, breakdown_json, timestamp 5. Return computed values to UI in a standardized JSON object for immediate display.
Outputs	● Total carbon footprint estimate (kg CO_2e) ● Component breakdown
Error Handling	● If some multiple-choice answers are missing, the UI prevents submission and highlights incomplete fields. ● If numeric inputs are out of range or invalid (negative, non-numeric), the backend rejects request with message. ● If emission-factor mapping fails, the backend defaults that component to unavailable and flags the response with a warning. ● If database write fails, the system retries automatically; on repeated failure, it returns the computed result to the user but marks it as “not saved” and logs it for admin review.

2.3.3. Data Preprocessing

2.3.3.1. Irrigation Data Preprocessing

2.3.3.1.1. Purpose and Data Sources

The irrigation recommendation module relies on consistent and time-aligned field and weather data to compute crop evapotranspiration (ETc), which represents the actual water consumption of a specific crop under current atmospheric conditions and growth stage

ETc is derived by adjusting reference evapotranspiration (ET_0)—the atmospheric water demand—using the crop coefficient (Kc), which reflects crop type and growth-stage characteristics.

Based on ETc-derived water demand and observed soil moisture dynamics, the system estimates both when irrigation should be initiated (e.g., after a certain level of water depletion or time interval) and how long irrigation should be applied, converting required water depth into pump operation time through irrigation gain parameters. To support this estimation process, the system shall execute a structured preprocessing pipeline on the following inputs before any ET_0/Kc-based calculation:

- **Weather / Meteorological Data** (for ET_0 or ET_0 drivers): temperature, relative humidity, wind speed(or alternatives).
- **Field Sensor Data**: soil moisture (SM), air temperature/humidity.
- **Crop Metadata**: crop type and crop coefficient (Kc) profile parameters.
- **Irrigation Records**: irrigation events and pump duration/flow estimations for calibration and validation.

2.3.3.1.2. Timestamp Standardization and Time Alignment

To ensure consistent irrigation calculations based on reference evapotranspiration (ET_0), which represents the atmospheric water demand, and the crop coefficient (Kc), which adjusts ET_0 according to crop growth stage, the system shall convert all timestamps to a unified timezone and format. Sensor and weather data streams shall be aligned on a common timeline using a fixed resampling interval (e.g., hourly or daily).

2.3.3.1.3. Sensor Quality Control (Noise, Drift, Spikes)

The system shall apply sensor quality controls to mitigate noise, spikes, and long-term drift in soil moisture measurements. Unrealistic spikes shall be detected and excluded, smoothing shall reduce noise without suppressing irrigation effects, and baseline drift shall be monitored to generate sensor health indicators. These controls shall be applied before missing-value handling.

2.3.3.1.4. Missing Data Analysis and Handling

The system shall compute missing-value ratios per feature and generate a missing-data report. For ET_0 and weather-critical variables, short gaps shall be interpolated, while extended gaps shall suspend ETc computation. For soil moisture data, short gaps shall be interpolated, and long gaps shall result in reduced confidence using weather-based fallback estimation.

2.3.3.1.5. Context-Aware Outlier Detection

The system shall perform context-aware outlier detection rather than generic IQR filtering. Statistical outlier detection shall be applied to meteorological variables, while soil moisture outliers shall be evaluated using irrigation-aware rules that distinguish valid irrigation-induced changes from sensor anomalies.

2.3.3.1.6. Crop Coefficient (Kc) Profile Preparation

The system shall generate a continuous Kc time series by loading crop-specific Kc profiles and mapping planting dates to a day-after-planting (DAP) index. If calibrated Kc parameters are available, versioned calibrated values shall be applied consistently.

2.3.3.1.7. Derived Features for ET_0 + Kc Irrigation Logic

The system shall compute crop evapotranspiration as **$ETc = ET_0 \times Kc$** and derive cumulative ETc over a configurable rolling window to support irrigation recommendation decisions.

2.3.3.2. Image Data Preprocessing for Disease Detection

The disease detection module relies on accurate, noise-reduced, and consistently formatted image data to ensure model stability, high inference accuracy, and reproducible visual diagnostics. To support both training and inference, the system shall apply a structured preprocessing pipeline to all plant leaf images captured/uploaded through the TARAS application. This pipeline ensures that the CNN-based disease classifier operates on standardized, clean and meaningful visual inputs.

2.3.3.2.1. Raw Image Validation

The system shall validate the integrity and format of each incoming image before performing any analytical operation. Validation includes checking file format (JPEG/PNG), verifying that file size does not exceed predefined limits, ensuring that the image is not corrupted, confirming minimum acceptable resolution for classification. Images failing any of these checks shall be rejected and logged for quality assurance.

2.3.3.2.2. Image Resizing and Aspect-Ratio Normalization

In order to maintain consistency across the CNN input pipeline, the system shall resize all images to the model's expected input dimensions, preserve aspect ratio when possible and apply center-cropping/padding to prevent distortion. This ensures that all images conform to the numerical structure required by the model.

2.3.3.2.3. Color Space Standardization

To ensure consistent lighting interpretation and feature extraction, the system shall convert all images to a standardized color space. Operations include converting raw images to RGB and normalizing pixel intensity ranges to 0–1 (depending on model specification), This enables reproducible visual feature extraction across devices.

2.3.3.2.4. Noise Reduction

Mobile field images often contain shadows, glare or compression artifacts. The preprocessing pipeline shall apply optional filters such as Gaussian blur (low-intensity smoothing), median filtering for salt-and-pepper noise and contrast normalization for uneven lighting conditions. These operations reduce noise while preserving disease-relevant texture patterns.

2.3.3.2.5. Leaf Region Isolation

The system performs leaf segmentation to isolate disease-relevant regions. Methods include color-threshold masking (green-channel analysis), background removal and contour detection. Isolating the leaf region reduces false positives caused by soil, sky, or human-made background elements.

2.3.3.2.6. Data Augmentation for Model Generalization

To improve model robustness and prevent overfitting, the training pipeline shall apply controlled augmentation techniques, such as random rotations and flips, controlled brightness/contrast adjustments, and slight zoom or crop transformations. Augmentation must not distort disease-critical features.

2.3.3.2.7. Image Quality Assessment Checks

Before inference or training, the system shall evaluate image quality using criteria such as blur detection (Laplacian variance), lighting adequacy, obstruction analysis (e.g., partially visible leaves). Images failing minimum quality thresholds shall be flagged or optionally rejected.

2.3.3.2.8. Dataset Class Distribution Documentation

To support fairness and balanced classification accuracy, the system shall compute and store the distribution of disease classes in the annotated dataset. This documentation includes number of images per disease category, identification of underrepresented diseases,

recommendations for synthetic augmentation or targeted data collection.

2.3.3.2.9. Normalization for Inference

During inference, the system shall apply the same normalization parameters used during training, specifically mean subtraction, standard deviation division, reordering to the model's expected tensor structure. Consistency between training and inference pipelines is mandatory for reliable performance.

2.3.4. Non-Functional Requirements

2.3.4.1. Performance Requirements

Under normal operating conditions, sensor data is expected to become available to the backend and mobile application within **30 seconds to 1 minute** after collection. The mobile application is expected to respond to user interactions such as tab navigation, data loading and dashboard updates within **1–2 seconds**. Irrigation analysis and recommendation generation is expected to complete within a few seconds after relevant data becomes available. Disease-detection results are expected to be delivered within **5–10 seconds** following image upload, depending on network conditions. In cases of limited or unstable connectivity, the system is expected to immediately display cached or last-known data to maintain usability.

2.3.4.2. Reliability Requirements

The system is expected to work reliably in real agricultural environments where network interruptions and temporary component issues may occur. Under normal conditions, sensor data is expected to be successfully delivered for around **95–98%** of the time and any missing data being limited to short gaps. If communication is temporarily unavailable, the system is expected to continue using the last known data for up to **10–30 minutes** until new measurements arrive. Once connectivity is restored, data synchronization between field devices and the backend is expected to resume automatically within a few minutes. Any data loss is expected to be minimal and occasional rather than continuous. In case of temporary failures in sensor nodes, gateways or backend services, the system is expected to keep operating in a reduced but stable state and return to normal operation automatically when the issue is resolved.

2.3.4.3. Availability Requirements

The system is expected to remain available for regular use throughout the agricultural season. Backend services are

expected to achieve an overall availability of around **97–99%** under normal conditions. The application is expected to be accessible whenever the user's device is operational and core features remain available even during temporary outages. Planned maintenance or short service interruptions are expected to be infrequent and limited in duration, usually not exceeding a few minutes at a time. In the event of connectivity loss, the system is expected to continue providing access to previously synchronized data. Once connectivity is restored, the system is expected to resume normal operation automatically without requiring user intervention.

2.3.4.4. Scalability Requirements

The system is expected to scale gradually as the number of fields and deployed sensor nodes increases over time. In early deployments, the platform will start with a prototype setup in a limited field area, initially using two sensor nodes. The number of sensors is not fixed and is expected to be adjusted based on collected data, performance and the quality of recommendations produced by the system. Data storage and processing components are expected to accommodate continuous growth across multiple growing seasons. Therefore, the overall data volume is expected to increase steadily as new measurements and field records are accumulated, rather than increasing abruptly.

2.3.4.5. Security Requirements

The system is expected to protect user accounts and field related data from unauthorized access. Under normal operating conditions, only authenticated users shall be able to access the system. And each user is supposed to only see the information that they are allowed. User login information and other sensitive data are expected to be protected during both transmission and storage. Data related to fields, sensor outputs and analysis results are expected to be available only to the authorized users. Images and inputs uploaded by the users and inputs are expected to be secure in order to prevent data exposure. As the number of users increase, the system is expected to maintain this level of security without affecting usability and performance.

2.3.4.6. Usability Requirements

The system is expected to be usable by farmers with limited technical experience. Under normal operating conditions, a first time user is expected to understand and perform basic tasks like viewing field data or checking irrigation recommendations. Users should access most actions in the mobile app within 2 to 4 steps. Visual elements are expected to be readable under outdoor conditions and key information shall be distinguishable within

three to five seconds of viewing. New users are expected to become familiar with the system's main features after two short tutorial sessions at most. As the system evolves, the user effort required to perform common tasks is expected to remain in the limits stated previously in this paragraph.

2.3.4.7. Maintainability Requirements

The system is expected to be maintainable with limited effort. Under normal operating conditions, common maintenance activities like configuration changes, minor bug fixes, or small updates are expected to be completed within 1 to 2 maintenance sessions with 10 to 30 minutes of down time at most. Software updates are supposed to be able to be deployed remotely. Also they shall not require more than one technician. When a system fault occurs, the cause of the problem should be identified in 30 to 60 minutes. Minor fixes are expected to be resolved in a work day, while more complex issues are expected to be resolved in 2 to 5 work days.

2.3.4.8. Portability Requirements

The system is supposed to be portable across commonly used platforms and environments. Under normal operating conditions, the mobile app is expected to run on both iOS and Android devices without platform specific changes. Backend components are expected to be deployable across different server and cloud environments with one configuration adjustment. System migration to a new environment is expected to be completed in one work day without affecting stored data or functionality.

2.4. Analysis - UML

2.4.1. Use Cases

2.4.1.1. Use Case Diagram

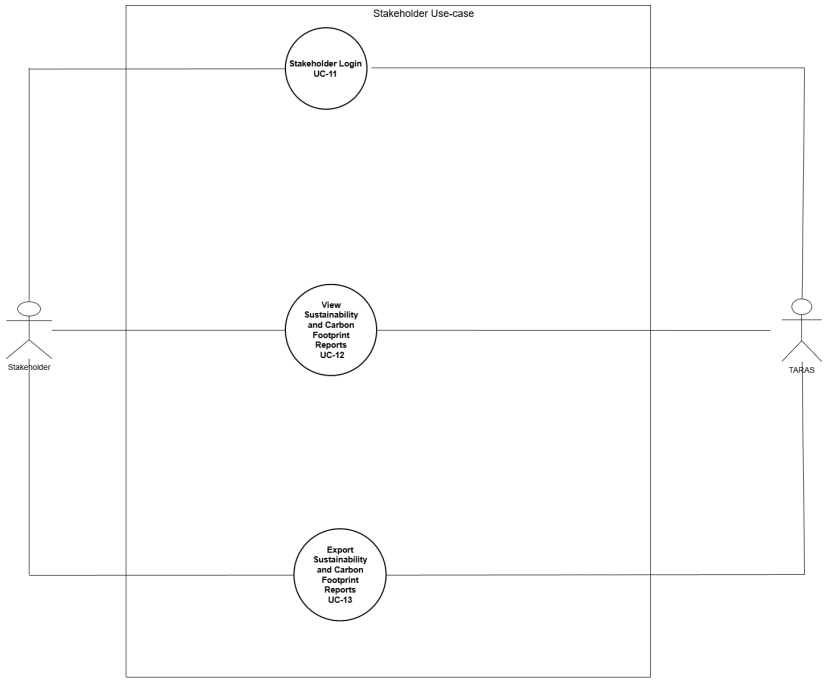


Diagram 1: Stakeholder Use Case Diagram

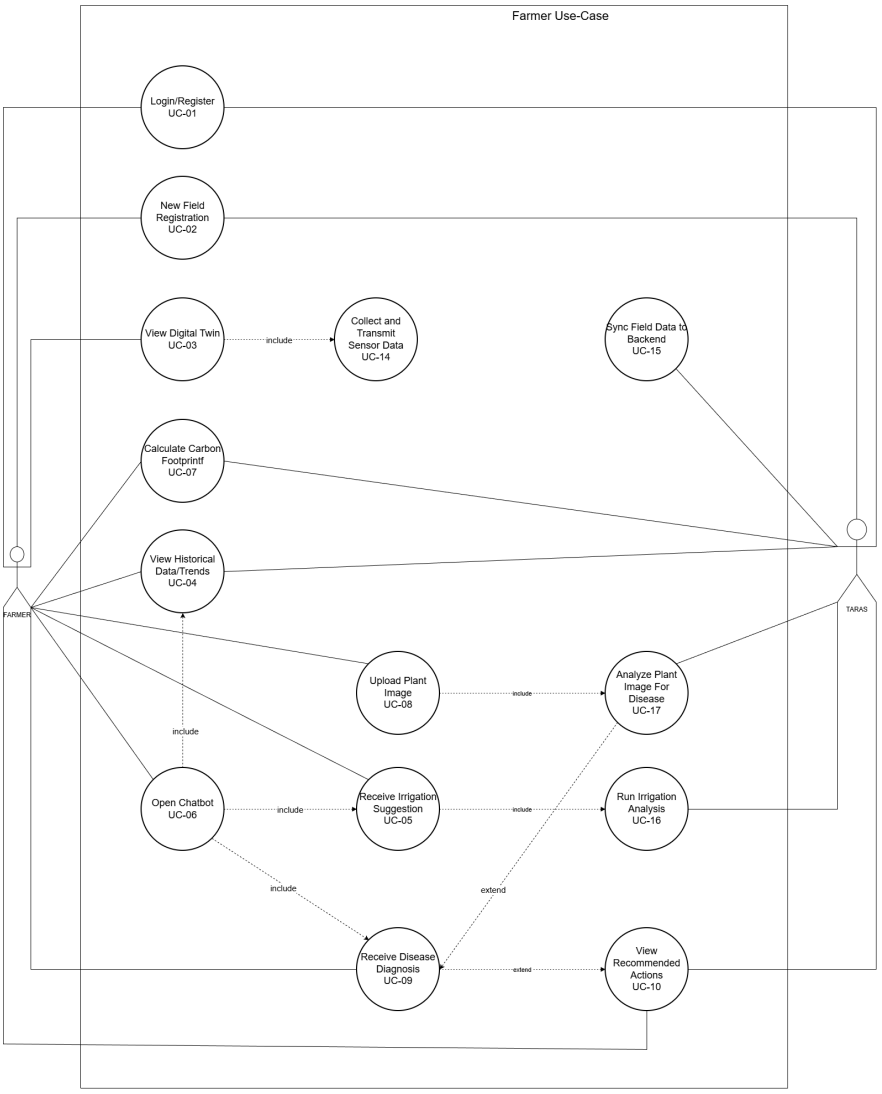


Diagram 2: Farmer Use Case Diagram

2.4.1.2. Description of Use Cases

Use Case Name	Login/Register
Use Case #	UC-01
Actors	Farmer
Description	The system manages user login and registration and grants access to the TARAS mobile application according to the user's role.
Precondition	● The user has access to the TARAS application installed. ● The user has a stable internet connection for the login/registration process. ● For login, the user already has a registered account.

Scenario	1. The system displays a login/register screen.
	2. User chooses Login or Registration
	3. For login:
	  a. User inputs credentials
	  b. System verifies information
	4. For registration:
	  a. User inputs required information
	  b. The system validates the inputs and creates a new user account.
	5. Upon successful authentication, the system loads the appropriate dashboard

Postcondition	● Users are authenticated and granted access to the system according to their role. ● The user session is initialized and stored for further operations.
Exceptions	● Incorrect username/password = System displays an error message. ● Weak or missing registration information = System rejects input and prompts correction. ● Network failure = System cannot authenticate and inform the user. ● Suspended or disabled account = System denies access.
Related Use Cases	

Use Case Name	**New Field Registration**
Use Case #	UC-02
Actors	Farmer
Description	The system records a new field for a registered farmer and links it to the selected sensor nodes so that incoming data can be associated with the correct field.
Precondition	● The farmer is already logged into the system. ● Sensor nodes are already in the field.
Scenario	● System scans for nearby sensor nodes/shows available node IDs. ● Farmer selects which sensor nodes belong to this field. ● The system links the selected node IDs to the new field. ● The system confirms that the field has been registered.
Postcondition	● A field record is created in the system. ● The selected sensor nodes are associated with this field, allowing incoming data to be grouped accordingly.
Exceptions	● No sensor nodes detected = System notifies the farmer. ● Invalid or duplicate IDs = System asks for correction.

	● Network issues = System cannot complete the registration.
Related Use Cases	

Use Case Name	**View Digital Twin**
Use Case #	UC-03
Actors	Farmer
Description	Users view the real-time and historical moisture/temperature distribution of the field via digital twin.
Precondition	Sensor data exists in DB.
Scenario	1. The farmer opens the digital twin page. 2. The app sends requests for the latest sensor values. 3. Backend queries DB hypertables. 4. Aggregated & real-time values returned. 5. Heatmap rendered on device.
Postcondition	Digital Twin updated with fresh data.
Exceptions	● No recent sensor data available. ● The backend cannot access DB. ● Device offline = cached data shown.
Related Use Cases	UC-14

Use Case Name	**View Historical Data/Trends**
Use Case #	UC-04
Actors	Farmer
Description	The system displays historical information for a selected field over a chosen time range, helping the farmer understand past field conditions and the effects of irrigation decisions.

Precondition	● Farmer is logged in. ● At least one field is registered and linked to sensor nodes. ● Historical data exists in the system for the selected field (or at least some past measurements are stored).
Scenario	1. Farmer chooses the history/trends option in the app. 2. System displays a time-range selector 3. Farmer selects a time range and confirms. 4. The system retrieves historical sensor and/or irrigation data for that period. 5. The system displays graphs or charts (e.g., moisture vs. time, temperature vs. time, irrigation events on the timeline). 6. Farmers may scroll, zoom, or switch between variables (moisture, temperature, irrigation). 7. Farmer closes the view or returns to the field overview.
Postcondition	● The farmer has viewed historical trends for the selected field over the chosen period.
Exceptions	● No data available for the selected period = System shows an informative “No data for this period” message and suggests another range. ● Partial data gaps = System shows available data and may mark missing intervals. ● Network/connectivity issues = System cannot fetch new history and offers cached trends if available.
Related Use Cases	UC-06

Use Case Name	**Receive Irrigation Suggestion**
Use Case #	UC-05
Actors	Farmer
Description	The system provides an irrigation recommendation for the selected field based on previously computed irrigation analysis results, including ETc-based water demand and soil moisture dynamics. The recommendation indicates when irrigation should be initiated and how long irrigation should be applied, presented in a clear and farmer-friendly format.
Precondition	● The farmer is logged into the system. ● At least one field is registered and selected. ● Irrigation analysis results are available for the field.

Scenario	1. The farmer opens the field dashboard in the mobile application. 2. The system checks whether a valid and up-to-date irrigation analysis result is available for the selected field. 3. If available, the system retrieves the most recent irrigation analysis outputs, including irrigation timing indicators and estimated irrigation duration. 4. The system interprets these outputs and converts them into a farmer-oriented recommendation format (e.g., textual guidance, highlighted indicators, or visual cues). 5. The recommendation is displayed on the dashboard, clearly indicating when irrigation should start and how long it should be applied. 6. The farmer reviews the recommendation and decides whether to act on it.
Postcondition	● The irrigation recommendation is presented to the farmer for decision-making. ● The recommendation remains accessible until a new analysis result replaces it.
Exceptions	● No irrigation analysis result available: The system informs the farmer that a recommendation cannot currently be provided. ● Network or backend failure: The system displays the last successfully retrieved recommendation, if available.
Related Use Cases	UC-06,UC-16

Use Case Name	**AI Decision-Support Chatbot**
Use Case #	UC-06
Actors	Farmer
Description	The farmer interacts with a conversational AI assistant that uses a Large Language Model (LLM) to interpret sensor data, irrigation recommendations, historical trends and disease-detection results. The chatbot provides natural-language explanations guidance, and actionable decision-support suggestions tailored to the field's current conditions.
Precondition	● The user is authenticated and has access to the chatbot interface. ● Recent sensor data, irrigation recommendations, or disease analysis results are available. ● The LLM service is online and reachable by the backend.
Scenario	1. The user opens the AI assistant interface within the mobile application.

Use Case Name	AI Decision-Support Chatbot
	2. The user enters a question, command or request for advice (e.g., *"Should I irrigate today?"* or *"What does this disease mean for my field?"*). 3. The system retrieves relevant contextual data: ● Latest sensor readings (soil moisture, temperature, humidity) ● Irrigation recommendations generated by the recommendation service. ● Disease-detection results, if available ● Weather and historical trends (optional) 4. The backend compiles this data into a structured context package. 5. The LLM processes the user's question together with the contextual data. 6. The LLM generates a natural-language response with explanations or recommended actions. 7. The system returns the response to the user through the chat interface. 8. The user reads or follows the provided guidance.
Postcondition	● The farmer receives a personalized, context-aware advisory message. ● The conversation may be logged for future analytics or model improvement.
Exceptions	● The LLM service is unreachable or times out. ● No relevant sensor or irrigation data is available for the query. ● User query is ambiguous or cannot be interpreted. ● Backend fails to compile context data. ● Response generation error occurs (model failure).
Related Use Cases	UC-04, UC-05, UC-09

Use Case Name	Calculate Carbon Footprint
Use Case #	UC-07
Actors	Farmer
Description	The farmer requests a carbon footprint calculation for a selected field by providing basic activity-related inputs. The system uses this information together with existing field data to estimate the environmental impact and presents the result to the farmer [9].

Precondition	● Farmer is logged in. ● At least one field is registered and selected. ● The farmer has access to the carbon footprint feature.
Scenario	1. Farmer opens the “Carbon Footprint” or “Environmental Impact” section. 2. The system prompts the farmer to enter required inputs (e.g., irrigation method, energy source, usage level). 3. Farmer provides the requested inputs and confirms the calculation request. 4. The system processes the provided information together with existing field data. 5. The system calculates the estimated carbon footprint for the field. 6. The calculated result is displayed to the farmer in a clear and understandable format.
Postcondition	● A carbon footprint estimate is generated for the selected field. ● The result becomes available for viewing and comparison over time.
Exceptions	● Required inputs are missing = System asks the farmer to complete the information. ● Calculation cannot be completed = System informs the farmer that the result is temporarily unavailable.
Related Use Cases	

Use Case Name	**Upload Plant Image**
Use Case #	UC-08
Actors	Farmer
Description	The user selects or captures a plant leaf image and uploads it to the system for disease analysis.
Precondition	● The user is authenticated. ● The device has network connectivity/queued upload mode enabled.
Scenario	1. The user navigates to the disease detection page. 2. The user captures an image or selects an existing image. 3. The system validates the selected image type and size. 4. The image is uploaded to the backend for further processing. 5. The system acknowledges successful upload.
Postcondition	The image is received and stored temporarily by the backend for analysis.

Use Case Name	Upload Plant Image
Exceptions	● Invalid or unsupported file format. ● Image corrupted or unreadable. ● Network failure during upload. ● Upload exceeds size limit.
Related Use Cases	UC-17

Use Case Name	Receive Disease Diagnosis
Use Case #	UC-09
Actors	Farmer
Description	After uploading an image, the user receives the disease classification, confidence score and severity returned by the system.
Precondition	● A valid image has been uploaded. ● The inference engine is available.
Scenario	1. The system preprocesses the uploaded image. 2. The system runs the disease-detection model on the image. 3. The model generates a label, severity level and confidence score. 4. The backend sends the diagnosis result to the user's device. 5. The user views the diagnosis.
Postcondition	● Diagnosis is displayed to the user. ● The result is stored in the database
Exceptions	● Model service unavailable. ● Inference timeout. ● Disease cannot be identified with sufficient confidence. ● Backend failure returning the result.
Related Use Cases	UC-06,UC-10, UC-17

Use Case Name	View Recommended Actions
Use Case #	UC-10
Actors	Farmer
Description	The user views actionable recommendations generated based on the diagnosed disease (e.g., treatment steps, risk warnings).
Precondition	● A diagnosis has been successfully generated ● Recommendation rules or mappings exist for the detected disease.
Scenario	1. The system matches the diagnosed disease to predefined recommendation rules. 2. The system generates appropriate treatment or monitoring actions. 3. The system presents the recommended actions on the app. 4. The user reviews the suggested next steps.
Postcondition	● Users receive actionable treatment guidance.
Exceptions	● No recommendation exists for the specific disease. ● Backend fails to retrieve recommendation rules. ● User interface fails to load the recommendation section.
Related Use Cases	UC-09

Use Case Name	Stakeholder Login
Use Case #	UC-11
Actors	Stakeholder
Description	A stakeholder logs into the platform to access high-level summaries, sustainability metrics and environmental reports for the agricultural fields they have a stake in.
Precondition	● The stakeholder has a valid account and credentials. ● System authentication services are available.
Scenario	1. Stakeholder opens the dashboard login page. 2. Stakeholder enters email/username and password. 3. System verifies credentials. 4. The system loads the stakeholder dashboard with authorized field data. 5. Stakeholders gain access to summary and reporting tools.

Use Case Name	Stakeholder Login
Postcondition	● Stakeholders are successfully authenticated and connected to their dashboard.
Exceptions	● Invalid credentials. ● User account disabled or missing necessary permissions. ● Authentication service unavailable. ● Network failure.
Related Use Cases	

Use Case Name	View Sustainability and Carbon Footprint Reports
Use Case #	UC-12
Actors	Stakeholder
Description	The stakeholder views sustainability and carbon footprint for a selected time period to support monitoring and reporting.
Precondition	● Stakeholder is authenticated and logged into the system. ● Relevant historical field, irrigation, and sustainability data exists in the database.
Scenario	1. The stakeholder selects a time period (year, month, quarter, or custom range). 2. The stakeholder selects the report type: a. Field Summary b. Sustainability Indicators c. Irrigation Efficiency d. Carbon Footprint & Environmental Metrics 3. The system retrieves aggregated and processed data for the selected period and report type. 4. The system displays the report using charts, tables, and key performance indicators.
Postcondition	Selected reports are successfully displayed to the stakeholder.
Exceptions	● No data available for the selected period. ● Required metrics cannot be computed due to missing or incomplete data. ● Backend analytics or query service failure.
Related Use	UC-13

Use Case Name	View Sustainability and Carbon Footprint Reports
Cases	

Use Case Name	Export Sustainability and Carbon Footprint Reports
Use Case #	UC-13
Actors	Stakeholder
Description	The stakeholder exports sustainability and carbon footprint for a selected period in a downloadable format for reporting.
Precondition	● Stakeholder is authenticated and logged into the system. ● Relevant historical data exists in the database. ● Report data can be generated for the selected period.
Scenario	1. The stakeholder selects a time period (year, month, quarter, or custom range). 2. The stakeholder selects the report type: a. Field Summary b. Sustainability Indicators c. Irrigation Efficiency d. Carbon Footprint & Environmental Metrics 3. The stakeholder selects an export file format (CSV or Excel). 4. The system generates the requested report file. 5. The system provides the file as a downloadable resource.
Postcondition	The selected report is successfully exported and made available for download.
Exceptions	● No data available for the selected period. ● Export generation failure. ● Backend query timeout or report rendering error.
Related Use Cases	UC-12

Use Case Name	Collect and Transmit Sensor Data
Use Case #	UC-14
Actors	System
Description	Sensor nodes deployed in the field automatically collect environmental measurements and transmit them directly to the TARAS cloud backend via Wi-Fi connectivity. The received data are stored and used for field monitoring, digital-twin updates, and irrigation decision support.
Precondition	● The sensor node is powered on and calibrated. ● The node is correctly registered in the system and associated with a specific field. ● Wi-Fi or alternative internet connectivity is available. ● The sampling interval is configured.
Scenario	1. The sensor node collects environmental measurements in the field. 2. The node prepares the measurements as a structured data payload. 3. The node transmits the data directly to the backend system via Wi-Fi. 4. The backend receives the data and verifies the associated field and node identity. 5. If valid, the backend stores the new measurements in the database. 6. The backend updates the field's latest status. 7. The system automatically triggers follow-up processes (digital-twin update, trend analysis, irrigation model input).
Postcondition	● New sensor data is stored in the database and available to farmer dashboards. ● The digital twin and field status are updated based on the latest measurements.
Exceptions	● If new data cannot be received due to connectivity issues, the system continues to display the most recent available data. ● The system may notify the farmer if sensor data has not been received for an unusually long period.
Related Use Cases	UC-03

Use Case Name	Sync Field Data to Backend
Use Case #	UC-15
Actors	System
Description	Field data collected from sensor nodes and the mobile application, including sensor readings and irrigation-related records, are synchronized with the cloud backend to ensure that all field information is safely stored and kept up to date.
Precondition	● Sensor nodes and/or the mobile application have collected new field data. ● At least one field is registered in the system. ● Wi-Fi or alternative internet connectivity is available at the sensor node or mobile device.
Scenario	1. New field data becomes available on a sensor node or the mobile application. 2. The system prepares the collected data for transmission to the backend. 3. When an internet connection is available, the system transmits the data directly to the backend. 4. The backend receives the data and stores it in the database. 5. The records associated with the relevant field are updated. 6. Updated information becomes available for farmer dashboards, digital-twin visualizations, historical analysis, and recommendations.
Postcondition	● Field data stored on sensor nodes or mobile devices is synchronized with the central TARAS system. ● The backend contains the latest version of field-related information.
Exceptions	● If no internet connection is available, the system temporarily stores the data locally and retries synchronization later. ● If synchronization fails repeatedly, the system may display a warning or status notification.
Related Use Cases	

Use Case Name	Run Irrigation Analysis
Use Case #	UC-16
Actors	System
Description	The system performs irrigation analysis for a registered field by processing sensor data, weather inputs, crop metadata, and irrigation records. The analysis produces irrigation-related indicators that are later used to generate irrigation recommendations and visual feedback.
Precondition	● A field is registered and linked to active sensor nodes. ● Sensor data and supporting inputs are available. ● Irrigation analysis services are operational.
Scenario	1. The arrival of new sensor data, updated weather information or a scheduled evaluation triggers the irrigation analysis process. 2. The system gathers the required field-related data from the database. 3. Irrigation data preprocessing steps are executed to ensure temporal consistency and data quality. 4. The system performs ETc-based irrigation analysis using the preprocessed data. 5. Irrigation timing indicators and estimated irrigation duration parameters are derived. 6. The analysis results are stored in the backend and marked as available. 7. The system updates dependent modules.
Postcondition	● Updated irrigation analysis results are stored for the field. ● The results become available for recommendation generation and historical inspection.
Exceptions	● Insufficient or invalid data: The analysis is delayed or marked as unavailable. ● Processing error: The system logs the failure and follows the corresponding procedure.
Related Use Cases	UC-05

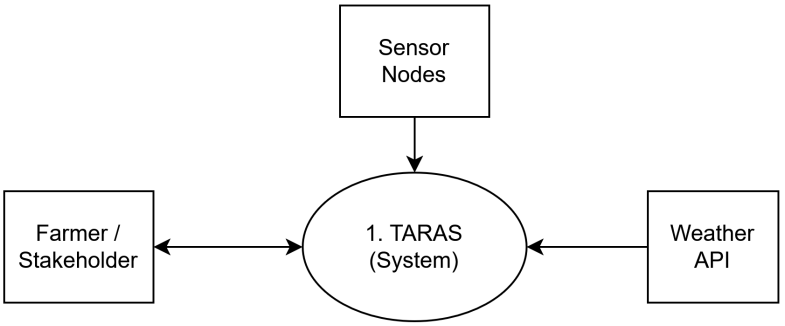
Use Case Name	Analyze Plant Image for Diseases
Use Case #	UC-17
Actors	System (Inference Engine, Backend Services)

Use Case Name	Analyze Plant Image for Diseases
Description	The system automatically processes an incoming plant image, runs the CNN model to identify potential diseases and stores the result for retrieval by the user.
Precondition	● A valid image upload request has been received from the mobile client. ● The CNN model is available and loaded on the server.
Scenario	1. The system receives the uploaded leaf image. 2. The system performs validation (format, resolution, corruption check). 3. The image is preprocessed (resize, normalization). 4. The system executes CNN inference to detect disease type/severity. 5. The system generates confidence scores and optional explanation maps. 6. The system stores the classification result in the database. 7. The system returns the structured result to the mobile application.
Postcondition	● The image analysis result is stored in the disease_results table. ● The result becomes available for display in the disease detection module.
Exceptions	● Model service unavailable or inference timeout. ● Image preprocessing failure. ● Database insertion error. ● Unrecognized disease or insufficient confidence.
Related Use Cases	UC-08, UC-09

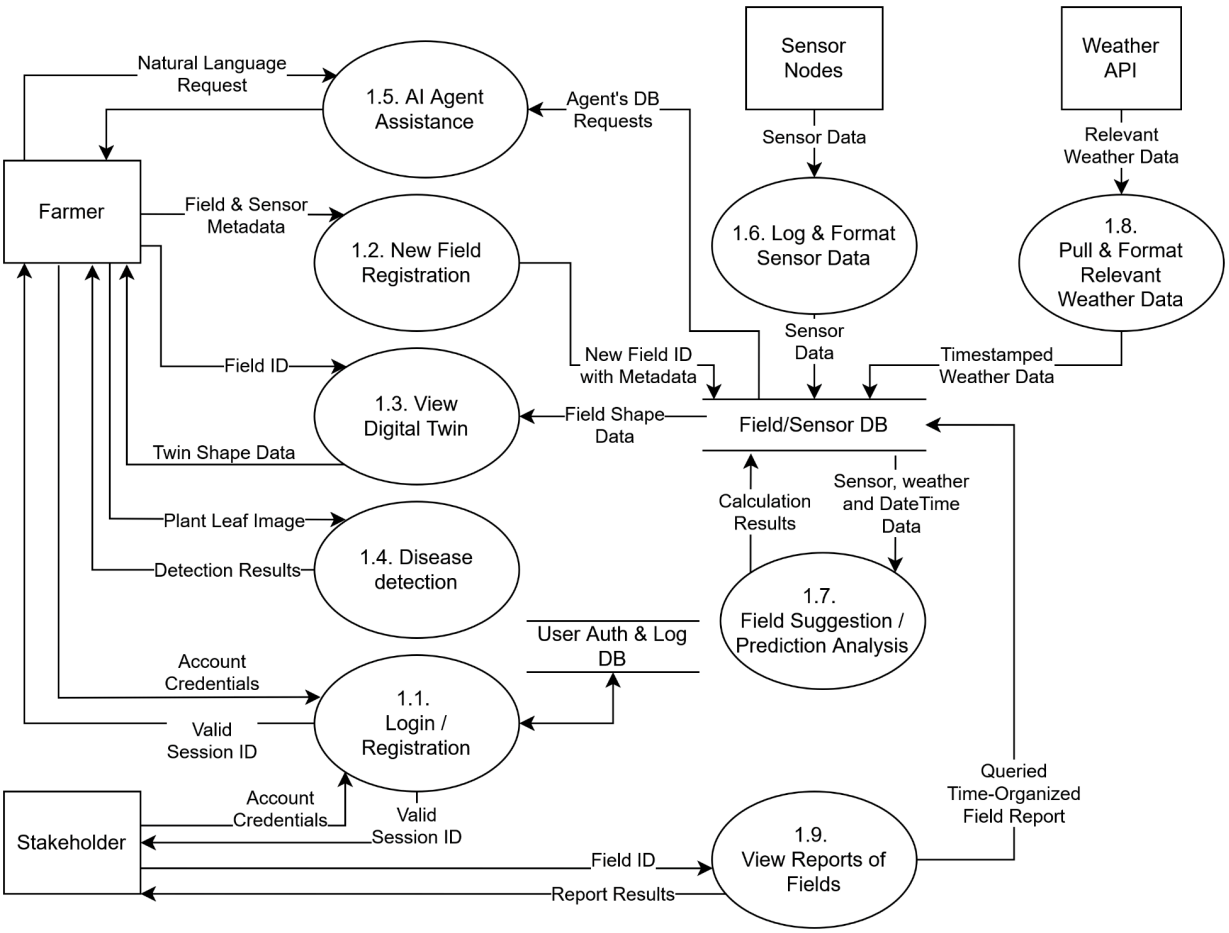
2.4.2. Functional Modelling(DFD)

2.4.2.1. DFD Diagrams

2.4.2.1.1. Level 0 - Context Diagram



2.4.2.1.2. Level 1 DFD



2.5. Conclusion

This Software Requirements Specification (SRS) defines the architectural and functional scope of TARAS and provides a strong foundation for system implementation. The document outlines how the platform bridges complex data analysis with practical agricultural decision-making to support irrigation and plant health management.

The proposed design integrates IoT sensing, machine-learning models, and digital-twin visualization within a unified cloud-based architecture, while addressing real-world agricultural constraints such as intermittent connectivity through buffering and synchronization mechanisms.

Overall, this SRS establishes a shared technical understanding among stakeholders and serves as a comprehensive blueprint for developing TARAS as a scalable, secure, and user-friendly precision agriculture system.

2.6. References

[1] Espressif Systems, “ESP32-C6 Technical Reference Manual,” 2023.

[2] “Systematic review of machine learning applications in sustainable agriculture: Insights on soil health and crop improvement,” PHYTON, vol. 94, 61375, 2025.
[3] PostgreSQL Global Development Group, PostgreSQL Documentation
[4] Amazon Web Services, Amazon Cognito – Developer Guide, AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/cognito/
[5] Timescale Inc., “TimescaleDB Documentation – Continuous Aggregates,” 2024.
[6] PostGIS Project, PostGIS Manual
[7] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1 - Success Criterion 1.4.3: Contrast (Minimum),” W3C Recommendation, 2018.
[8] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1 - Success Criterion 2.5.5: Target Size,” W3C Recommendation, 2018.
[9] ISO, ISO 14064-1: Greenhouse Gases - Carbon Footprint Quantification, International Organization for Standardization, 2018.

3. System Architecture & Methodology

3.1. Introduction

3.1.1. Purpose

This document defines the system architecture of the TARAS precision agriculture platform. It presents the high-level structural design of the system by describing its core architectural components, logical layers, and their interactions within the overall platform. The architectural decisions described in this document are derived directly from the functional and non-functional requirements defined in the Software Requirements Specification (SRS).

In addition to the architectural design, the document outlines the methodological approach adopted for data acquisition, preprocessing, and irrigation decision logic, explaining how agronomic principles and sensor-driven analytics are translated into operational system behavior. It also describes the role of auxiliary AI-assisted advisory components, which support explainability and user understanding without performing core decision-making.

It further explains how data flows across sensing, processing, analytics, and application layers, and how system components communicate to support scalable, reliable, and intelligent agricultural operations. The document serves as an architectural and methodological reference for guiding system implementation, integration, and future evolution, ensuring a consistent and maintainable realization of the TARAS platform.

3.1.2. Definitions and Acronyms

Term	Definition
Accuracy	The ratio of correctly classified instances (both positive and negative) to the total number of instances. It represents the overall correctness of the model's predictions.
Precision	The proportion of correctly predicted positive instances among all instances predicted as positive. It indicates how reliable the positive predictions of the model are.
Recall	The proportion of correctly predicted positive instances among all actual positive instances. It measures the model's ability to identify all relevant positive cases.
F1-Score	The harmonic mean of precision and recall. It provides a balanced measure that considers both false positives and false negatives, especially useful when class distributions are imbalanced.
Agronomy	The branch of agricultural science that focuses on crop production and soil management, examining the interactions between crops, soil, climate, and agricultural practices in order to optimize yield and resource efficiency.
Evapotranspiration (ET_0)	ET_0 is a standardized measure of atmospheric water demand, representing the evapotranspiration rate from a hypothetical reference surface under given meteorological conditions. It reflects the effect of weather conditions on water loss independently of crop type.
Crop Coefficients(Kc)	Kc is a dimensionless coefficient used to adjust reference evapotranspiration (ET_0) to account for crop type and growth-stage characteristics, enabling estimation of crop-specific water demand.
JWT(JSON Web Token)	JWT is a compact and secure token format used for authentication and authorization, allowing the system to verify user identity and access rights between the client and backend services.
ML	ML refers to computational techniques that enable systems to learn patterns from data and make predictions or decisions without being explicitly programmed, and is used in the TARAS system primarily for plant disease detection and auxiliary analysis tasks.
CV Models	CV models are computational models designed to analyze and interpret visual data such as images, enabling tasks like object recognition, classification, and pattern detection. In the TARAS system, they are used for plant leaf disease detection.
Cloud Services	Cloud services refer to on-demand computing resources such as storage, databases, and backend services provided over the internet, used in the TARAS system to support data storage, processing, and system scalability.
Client	The client is the user-facing component of a client–server architecture that initiates requests to the server and presents system functionality

	and data to the user. In the TARAS system, the mobile application acts as the client.
Server	The server is the backend component of a client–server architecture that receives requests from clients, processes them, manages data storage and system logic, and returns responses. In the TARAS system, the server hosts backend services, databases, and analysis modules.
Storage Bucket	A storage bucket is a logical container in cloud storage systems used to store and organize unstructured data such as images and files. In the TARAS system, storage buckets are used to store uploaded plant images and related assets.
Classification	Classification is a ML task in which input data are assigned to one of several predefined categories or classes. In the TARAS system, classification is used to identify plant diseases from leaf images.
Processing Pipeline	The process of collecting raw data from heterogeneous sources and transforming it into a consistent, reliable, and analysis-ready format. This includes acquiring environmental sensor data, historical records, metadata, and image data, followed by preprocessing steps such as timestamp alignment, unit normalization, quality control, outlier detection, and missing data handling to ensure data integrity before model-based analysis.
Deep Learning	A subfield of ML that utilizes neural networks with multiple layers to automatically learn hierarchical feature representations from data. It is particularly effective for complex tasks such as image recognition, natural language processing, and speech analysis.
CNN	A deep learning architecture designed to automatically extract features from image data using convolutional layers. It is widely used in tasks such as image classification and object recognition.
Overfitting	Overfitting refers to a modeling phenomenon where a ML model learns training data too closely, resulting in high training performance but poor generalization to unseen data.
Pre-trained weights	Model parameters obtained by training a neural network on a large-scale dataset prior to being applied to a target task. They are used to accelerate training and improve generalization performance.
Finetune	The process of retraining some or all layers of a pre-trained model on a task-specific dataset using a low learning rate, allowing the model to adapt to the new problem domain.
TensorFlow	An open-source ML framework developed by Google for building, training, and deploying ML and deep learning models.
Keras	A high-level deep learning API that runs on top of TensorFlow, enabling fast and efficient development of neural network models with a user-friendly and modular structure.
WCAG	WCAG is a set of international guidelines that define standards for

(Web Content Accessibility Guidelines)	making digital interfaces accessible to users with diverse abilities, focusing on principles such as perceivability, operability, understandability, and robustness.
Cognitive Load	Cognitive load refers to the amount of mental effort required to process information and interact with a system. In the TARAS interface design, reducing cognitive load aims to make the application easier to understand and use, especially for users with limited technical experience.
DAP	DAP refers to the number of days elapsed since a crop was planted and is used to represent crop growth stages for time-dependent analyses such as crop coefficient (Kc) estimation and irrigation planning.

3.2. System Overview

The TARAS (Tarım Analiz ve Simülasyonu) platform is a precision agriculture decision-support system designed to assist farmers in irrigation planning, crop health monitoring and sustainability assessment. The project initially explored a wide range of smart agriculture features, which were evaluated based on impact, feasibility, usability, cost and sustainability. Based on literature analysis and comparisons with existing solutions, the system scope was refined to focus on the most effective and implementable components. The final design centers on integrated field sensing, AI-driven analysis, digital twin visualization and a mobile application interface, ensuring a practical, scalable and user-oriented solution.

3.3. System Architecture

3.3.1. Architectural Design

The architectural design of the TARAS follows a hybrid architecture that combines Layered Architecture and Client–Server Architecture principles. This approach supports modular development, scalability and clear separation of responsibilities across system components.

3.3.1.1. Layered Architecture

3.3.1.1.1. Presentation Layer

Provides the mobile application interface through which farmers interact with the system. This layer presents sensor readings, irrigation recommendations, disease detection results, digital twin visualizations, sustainability metrics, and AI-assisted advisory explanations in an accessible and user-friendly manner. The advisory interface translates analytical outputs into clear, context-aware guidance, supporting user understanding without replacing farmer decision-making.

3.3.1.1.2. Application & Analytics Layer

Implements the core system logic, including data processing, irrigation prediction, disease analysis, decision-support workflows and user interaction management. ML and computer vision models operate within this layer to transform raw data into actionable insights.

3.3.1.1.3. Data & Cloud Services Layer

Handles data storage, retrieval and synchronization between field devices and the mobile application. This layer manages time-series sensor data, historical records, model outputs and user inputs using a centralized cloud-based backend.

3.3.1.1.4. Device & Sensing Layer

Includes IoT sensor nodes deployed in the field, responsible for collecting soil moisture, temperature and environmental data. This layer abstracts hardware-level interactions and ensures reliable data acquisition.

3.3.1.1.5. Communication Layer

Manages data transmission between sensor nodes, gateway devices and cloud services using long-range and internet-based communication protocols. It ensures reliable, low-power and fault-tolerant data flow across the system.

3.3.1.2. Client–Server Architecture

The TARAS mobile application operates as a client that interacts with the cloud-based backend services. Field sensor data, user inputs, and system outputs, such as irrigation recommendations, disease analysis results and sustainability metrics are exchanged through request–response communication mechanisms. This separation between client and server components enables loose coupling, supports scalability and simplifies maintenance and future extensions.

3.3.2. Decomposition Description

3.3.2.1. Field Sensing & Data Acquisition Subsystem

- Soil Moisture Sensing Module
- Environmental Data Collection (Temperature, Humidity)

- Sensor Calibration & Validation Module
- Gateway Communication Module

3.3.2.2. Analytics & Decision Support Subsystem

- Data Storage Module
- Irrigation Prediction Module
- Disease Detection Module
- Sustainability & Carbon Footprint Calculation Module
- Data Processing & Model Management Module
- LLM advisory & Explainability Module

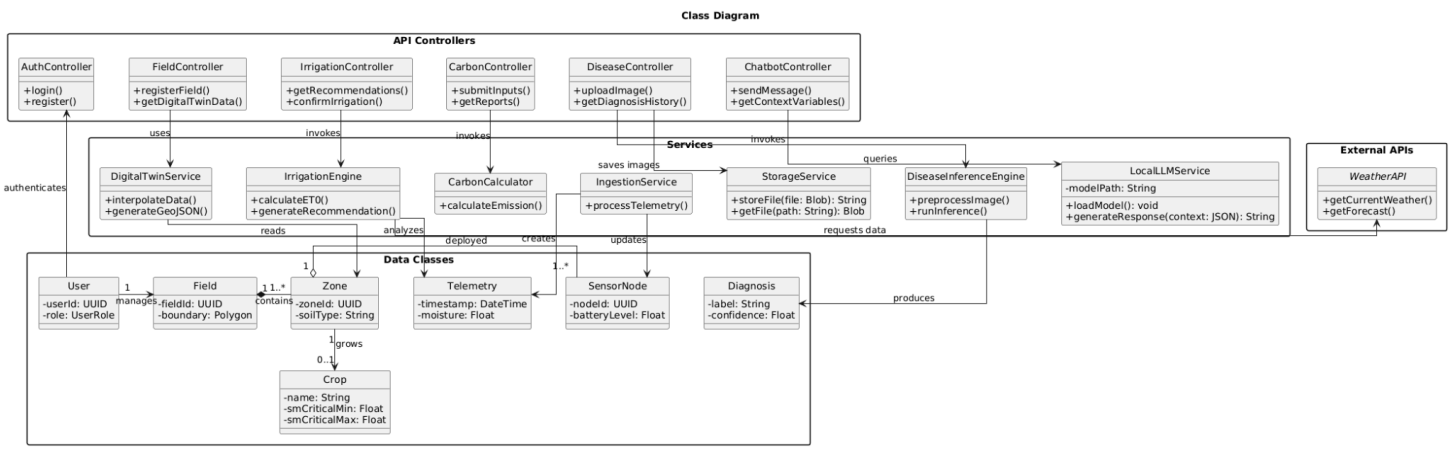
3.3.2.3. Visualization & User Interaction Subsystem

- Mobile Application Interface
- Digital Twin Visualization Module
- Recommendation & Alert Presentation
- User Input & Configuration Management

Each module in the TARAS platform interacts with other components through clearly defined interfaces, encapsulating internal logic and exposing only essential operations such as retrieving sensor measurements, generating irrigation recommendations, performing disease analysis, or updating visualization data. This approach supports abstraction, modular refinement, and reusability in line with core software engineering design principles.

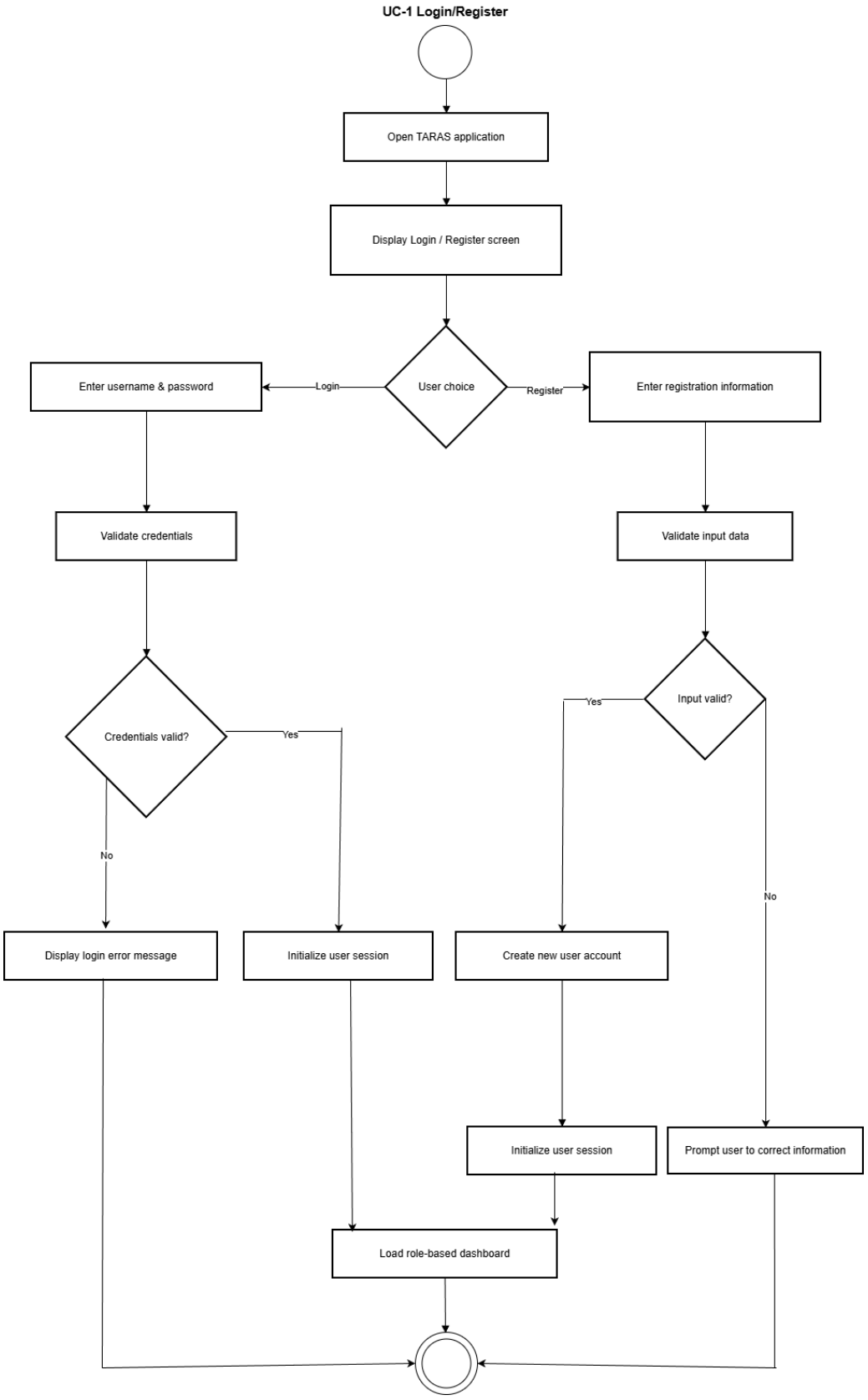
The modular system structure simplifies independent development and testing of components and allows future extensions, such as integrating new sensor types, improving prediction models or expanding sustainability metrics without impacting the overall system architecture.

3.3.2.4. Class Diagram

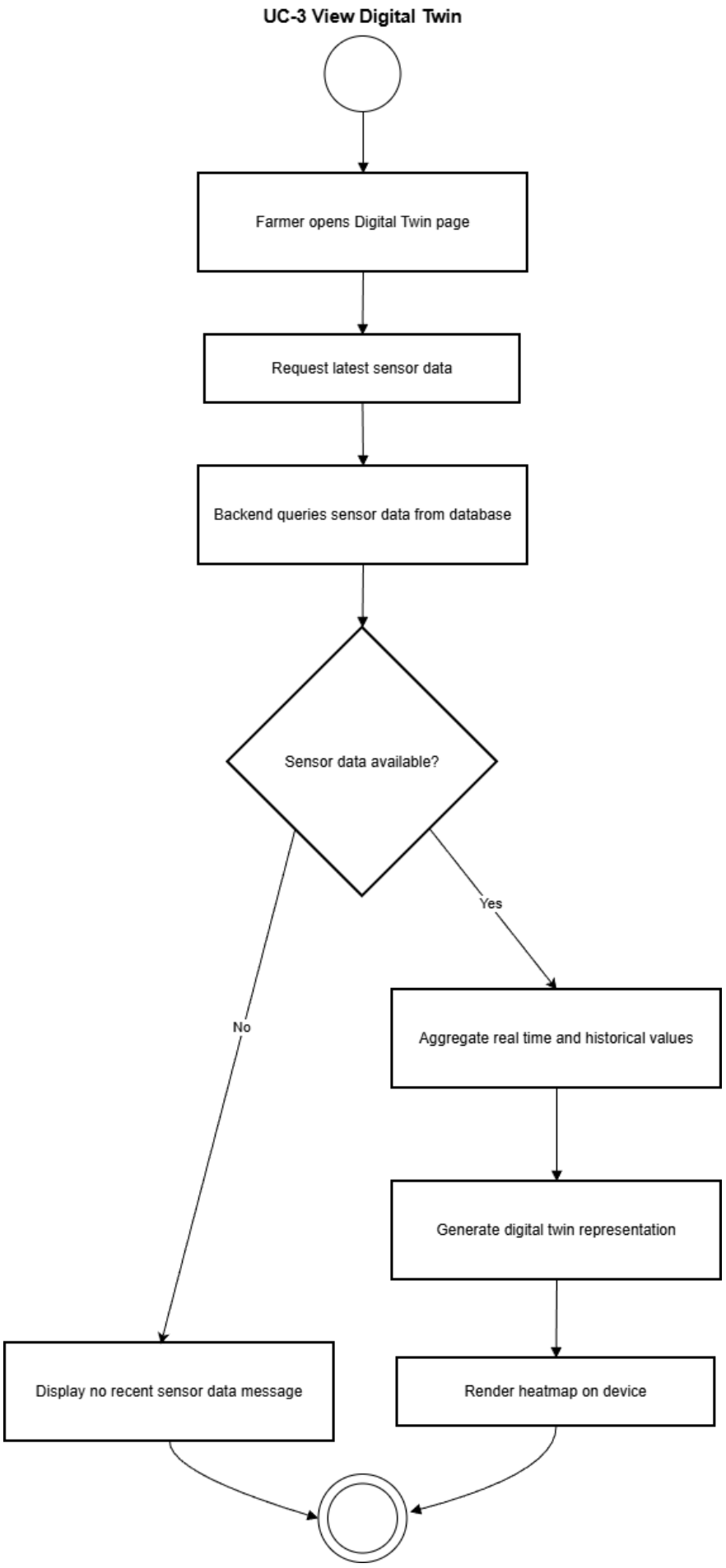


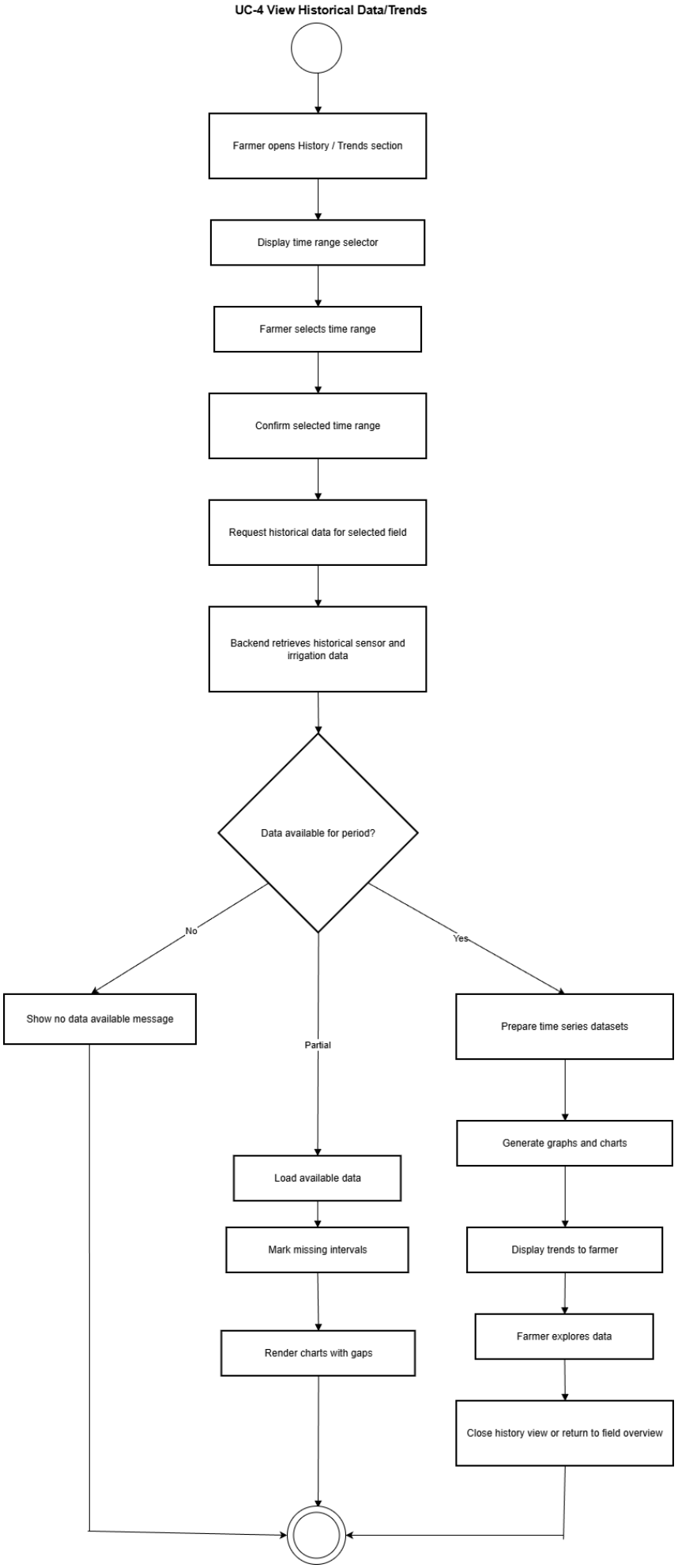
3.3.3. System Modeling

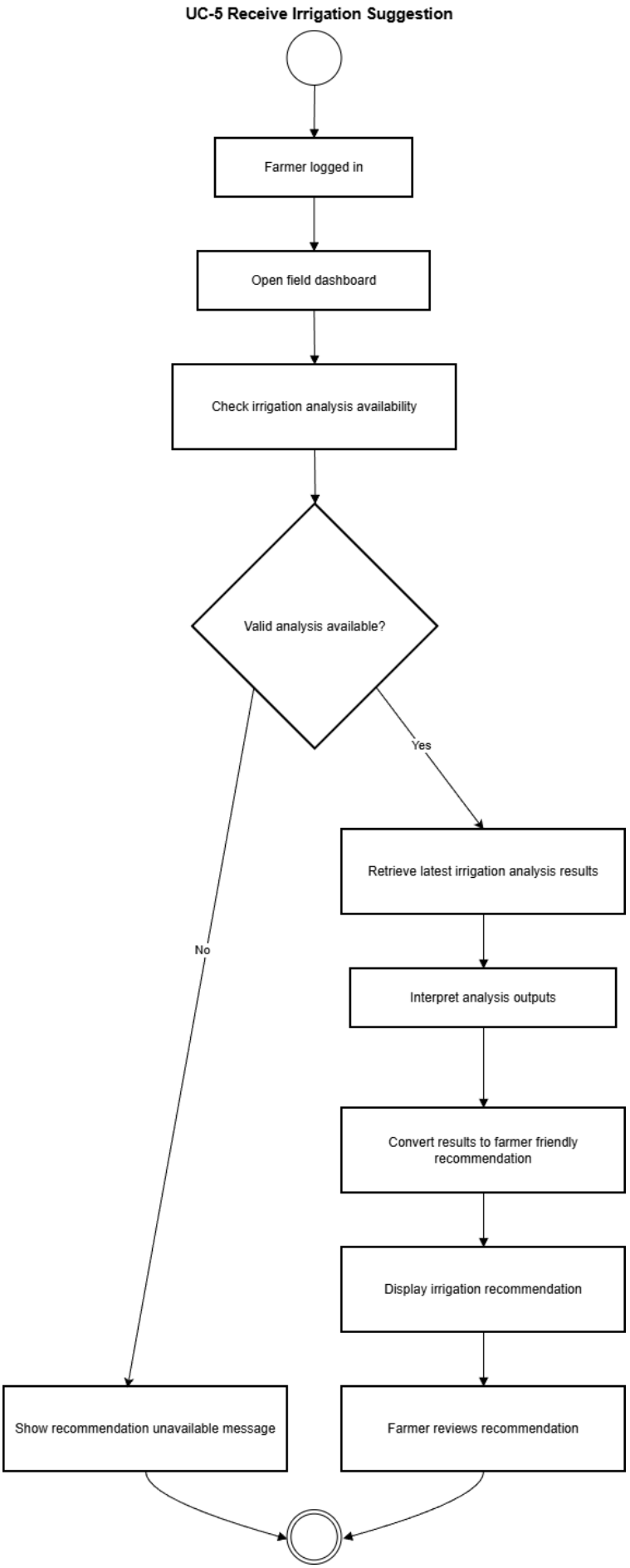
3.3.3.1. Activity Diagrams

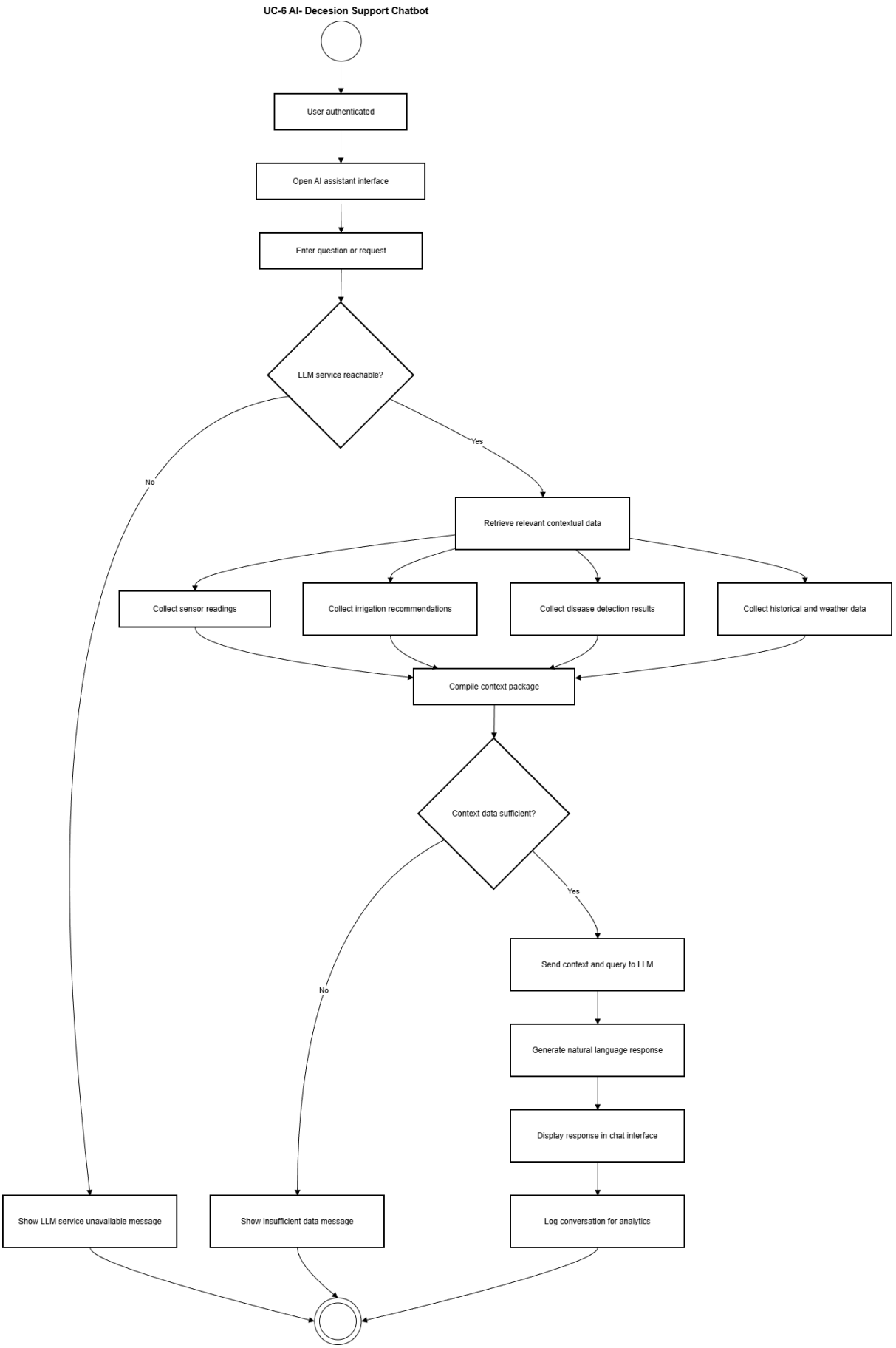


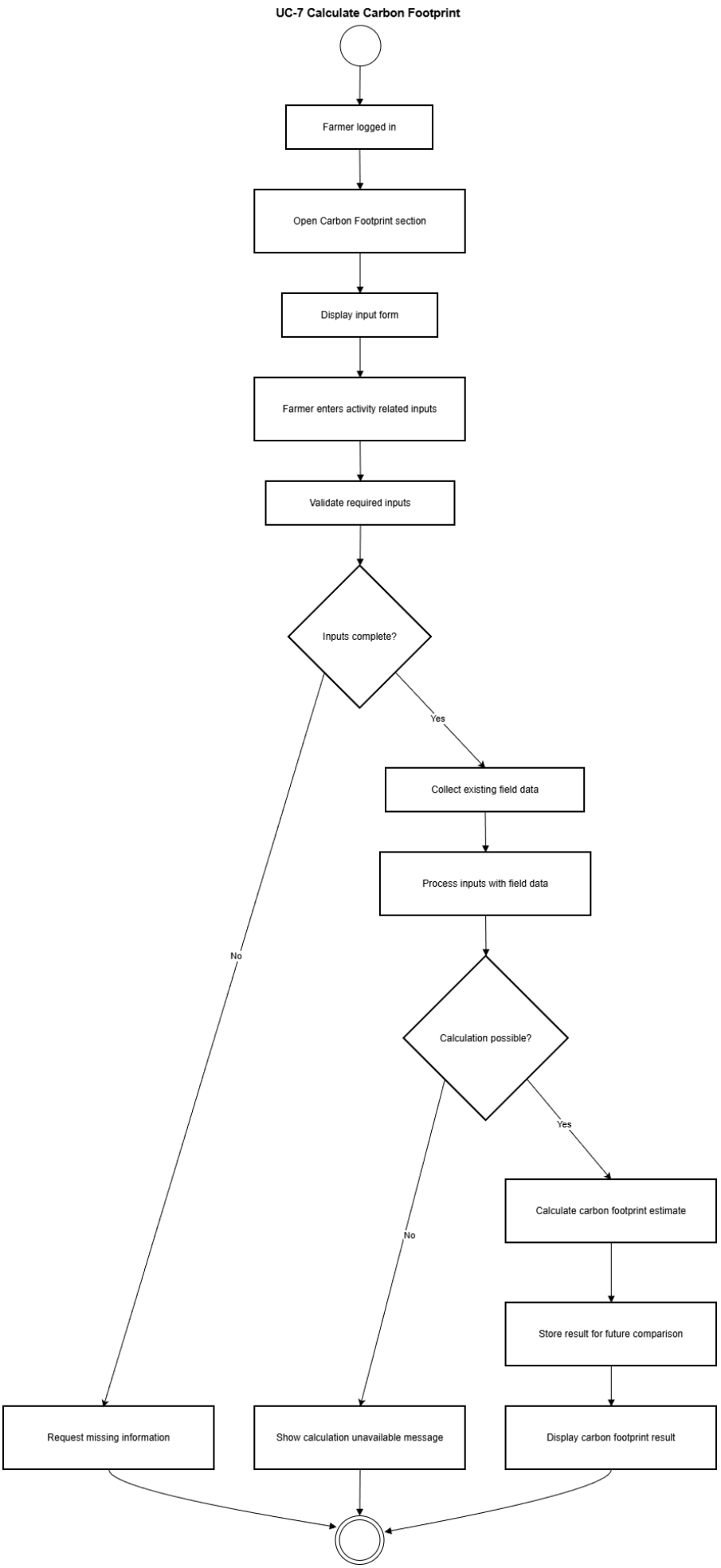
UC-2 New Field Registration
Farmer logged in
Open New Field Registration page
Scan for nearby sensor nodes
or retrieve available node IDs
Sensor nodes detected?
No
Yes
Notify farmer: No sensor nodes found
Display available sensor node IDs
Farmer selects sensor nodes
Validate selected node IDs
IDs valid and unique?
No
Yes
Request correction from farmer
Create new field record
Link selected sensor nodes to field
Confirm field registration

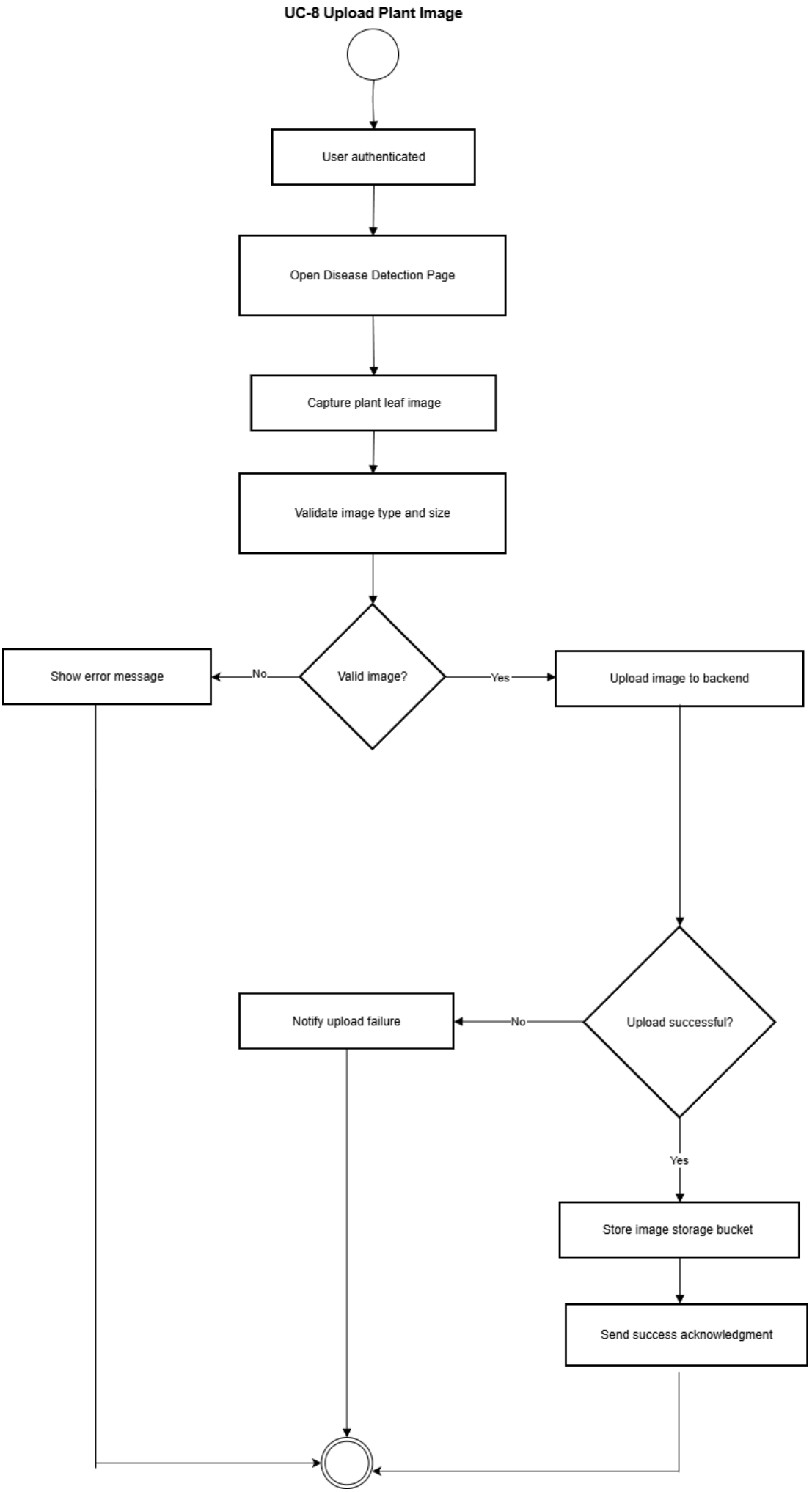

UC-3 View Digital Twin
Farmer opens Digital Twin page
Request latest sensor data
Backend queries sensor data from database
Sensor data available?
Yes
No
Aggregate real time and historical values
Generate digital twin representation
Display no recent sensor data message
Render heatmap on device


UC-4 View Historical Data/Trends
Farmer opens History / Trends section
Display time range selector
Farmer selects time range
Confirm selected time range
Request historical data for selected field
Backend retrieves historical sensor and irrigation data
Data available for period?
No
Partial
Yes
Show no data available message
Load available data
Mark missing intervals
Render charts with gaps
Prepare time series datasets
Generate graphs and charts
Display trends to farmer
Farmer explores data
Close history view or return to field overview

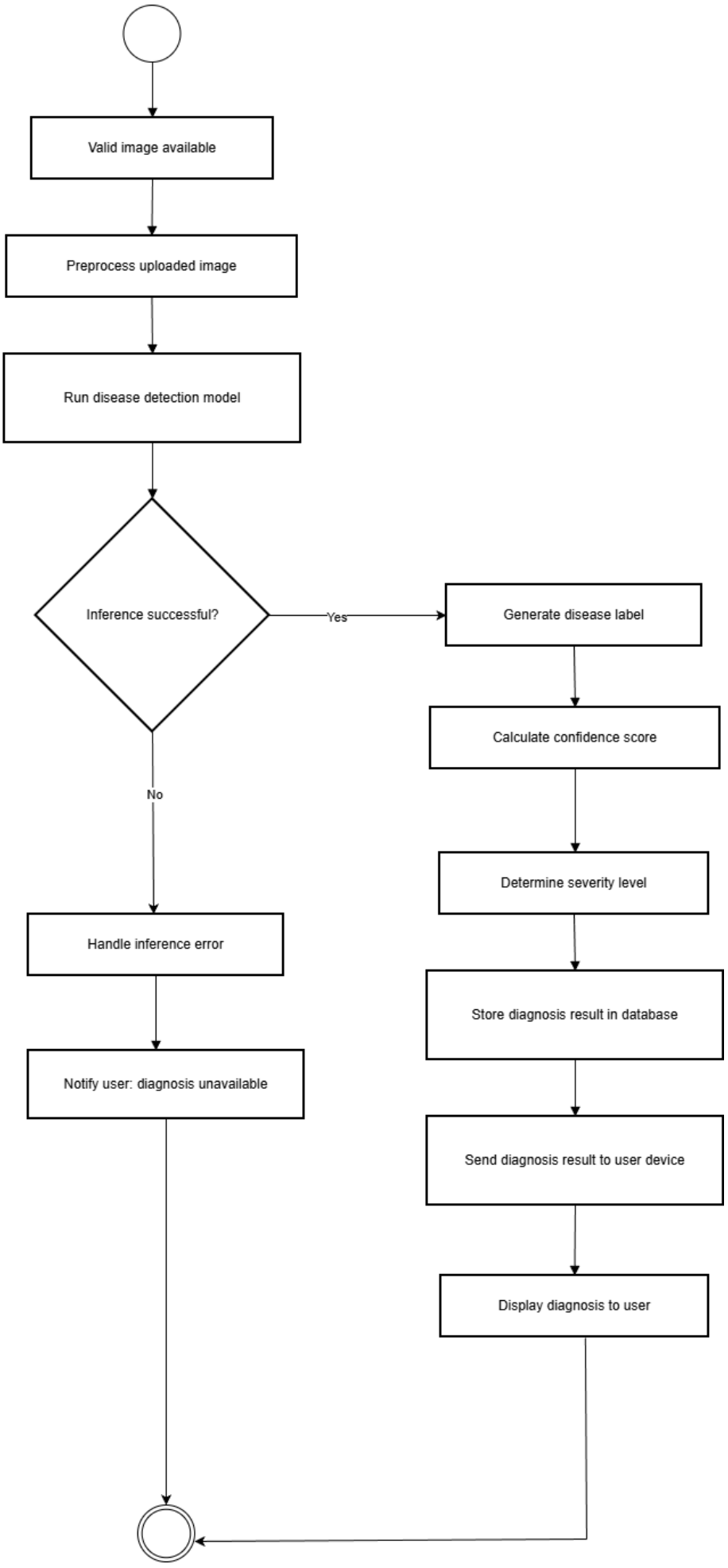

UC-5 Receive Irrigation Suggestion
Farmer logged in
Open field dashboard
Check irrigation analysis availability
Valid analysis available?
Yes
No
Retrieve latest irrigation analysis results
Interpret analysis outputs
Convert results to farmer friendly recommendation
Display irrigation recommendation
Farmer reviews recommendation
Show recommendation unavailable message

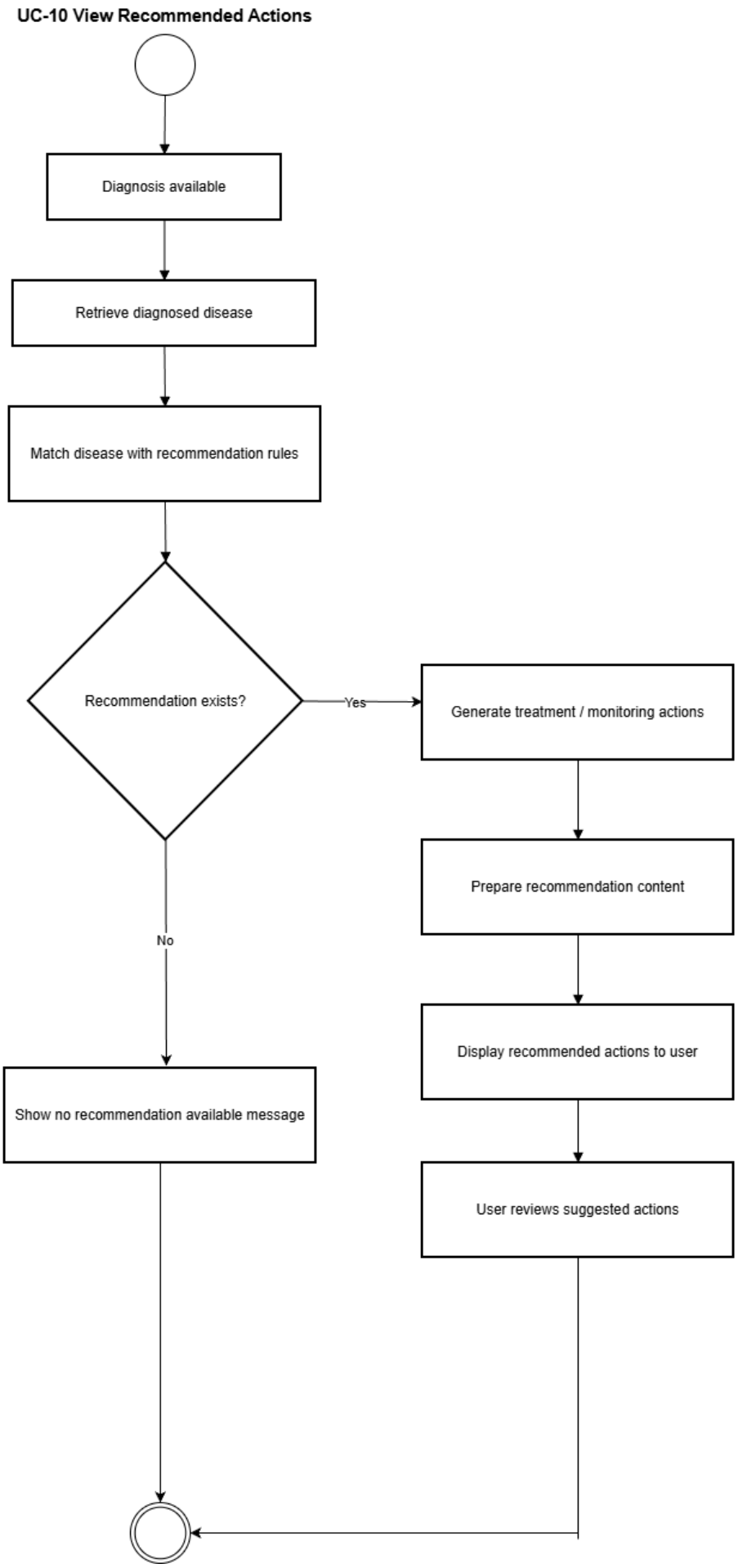

UC-6 AI- Decesion Support Chatbot
User authenticated
Open AI assistant interface
Enter question or request
LLM service reachable?
Yes
No
Retrieve relevant contextual data
Collect sensor readings
Collect irrigation recommendations
Collect disease detection results
Collect historical and weather data
Compile context package
Context data sufficient?
Yes
No
Send context and query to LLM
Generate natural language response
Display response in chat interface
Log conversation for analytics
Show LLM service unavailable message
Show insufficient data message

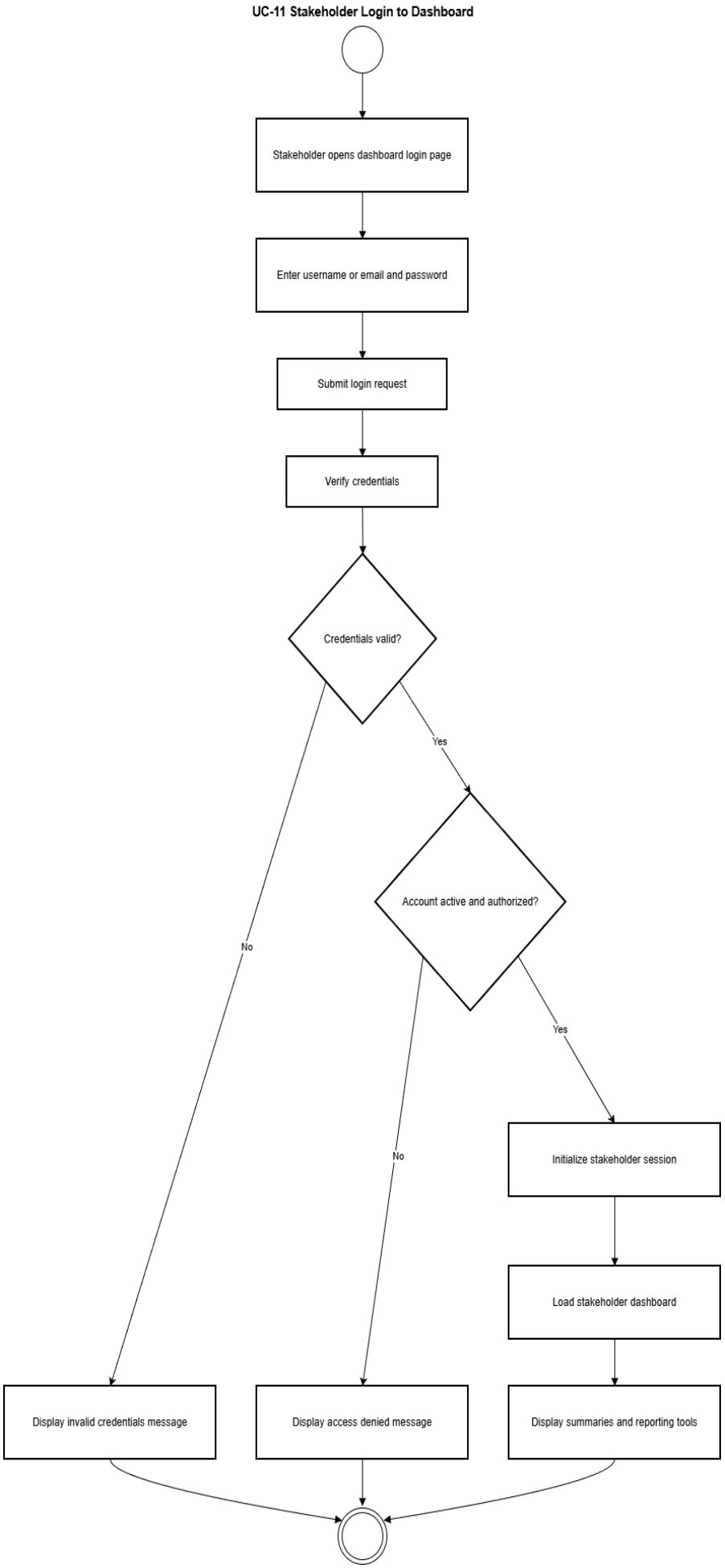

UC-7 Calculate Carbon Footprint
Farmer logged in
Open Carbon Footprint section
Display input form
Farmer enters activity related inputs
Validate required inputs
Inputs complete?
Yes
No
Collect existing field data
Process inputs with field data
Calculation possible?
Yes
No
Calculate carbon footprint estimate
Store result for future comparison
Request missing information
Show calculation unavailable message
Display carbon footprint result

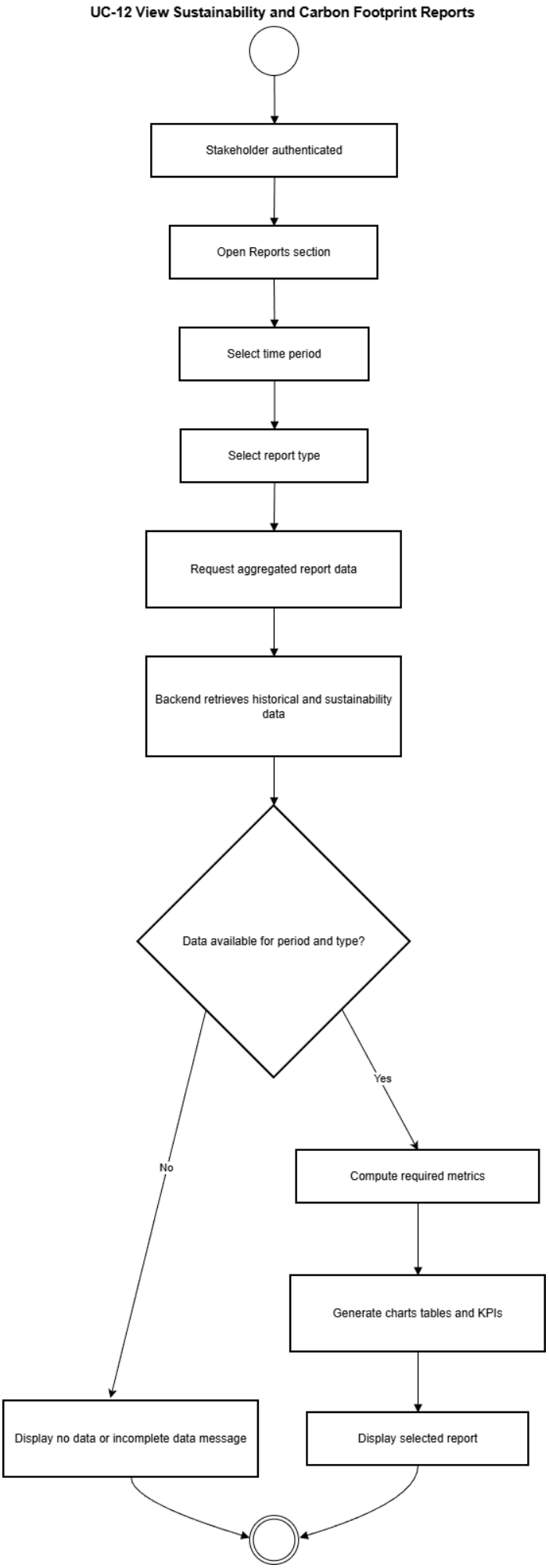

UC-8 Upload Plant Image
User authenticated
Open Disease Detection Page
Capture plant leaf image
Validate image type and size
Valid image?
No
Show error message
Yes
Upload image to backend
Upload successful?
No
Notify upload failure
Yes
Store image storage bucket
Send success acknowledgment

UC-9 Receive Disease Diagnosis

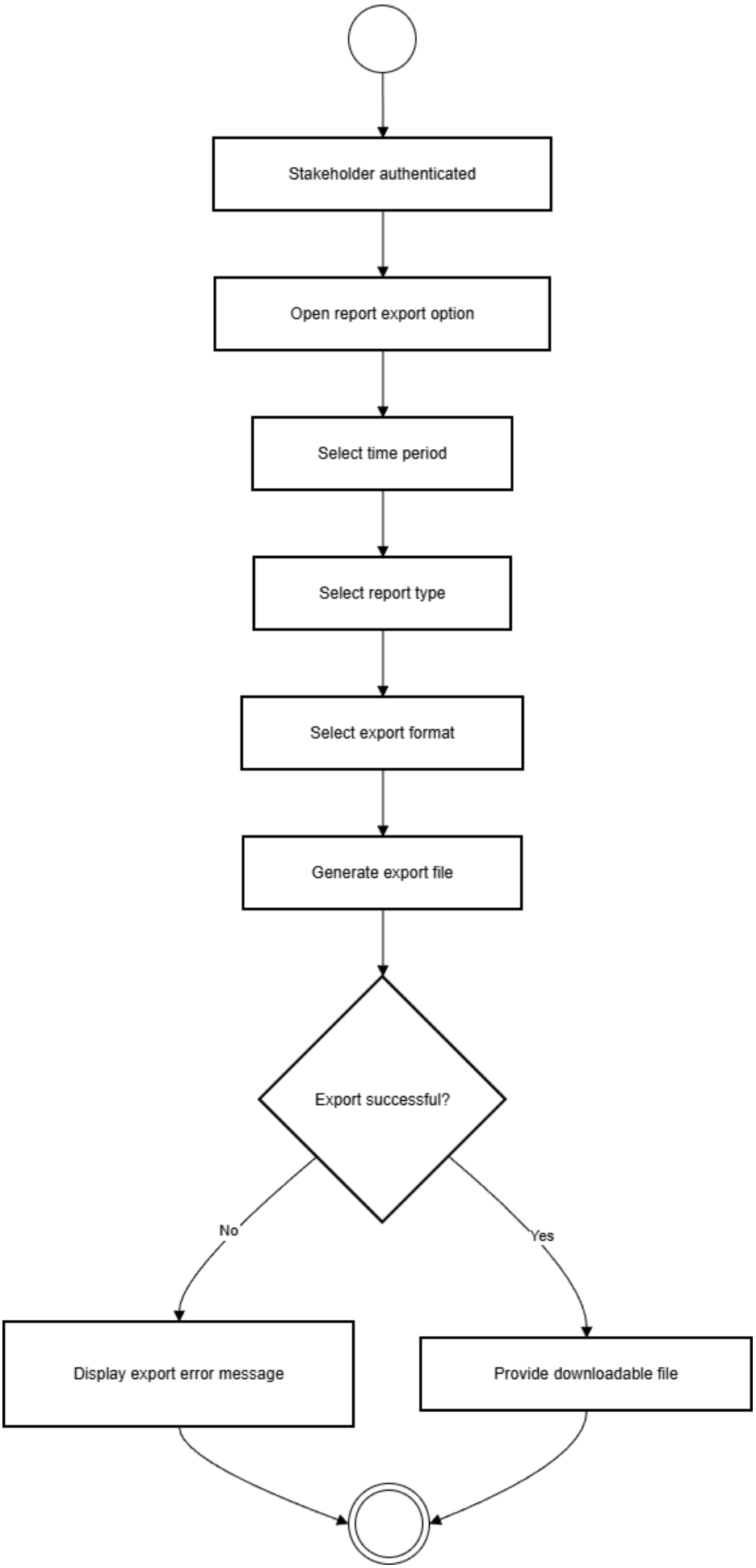


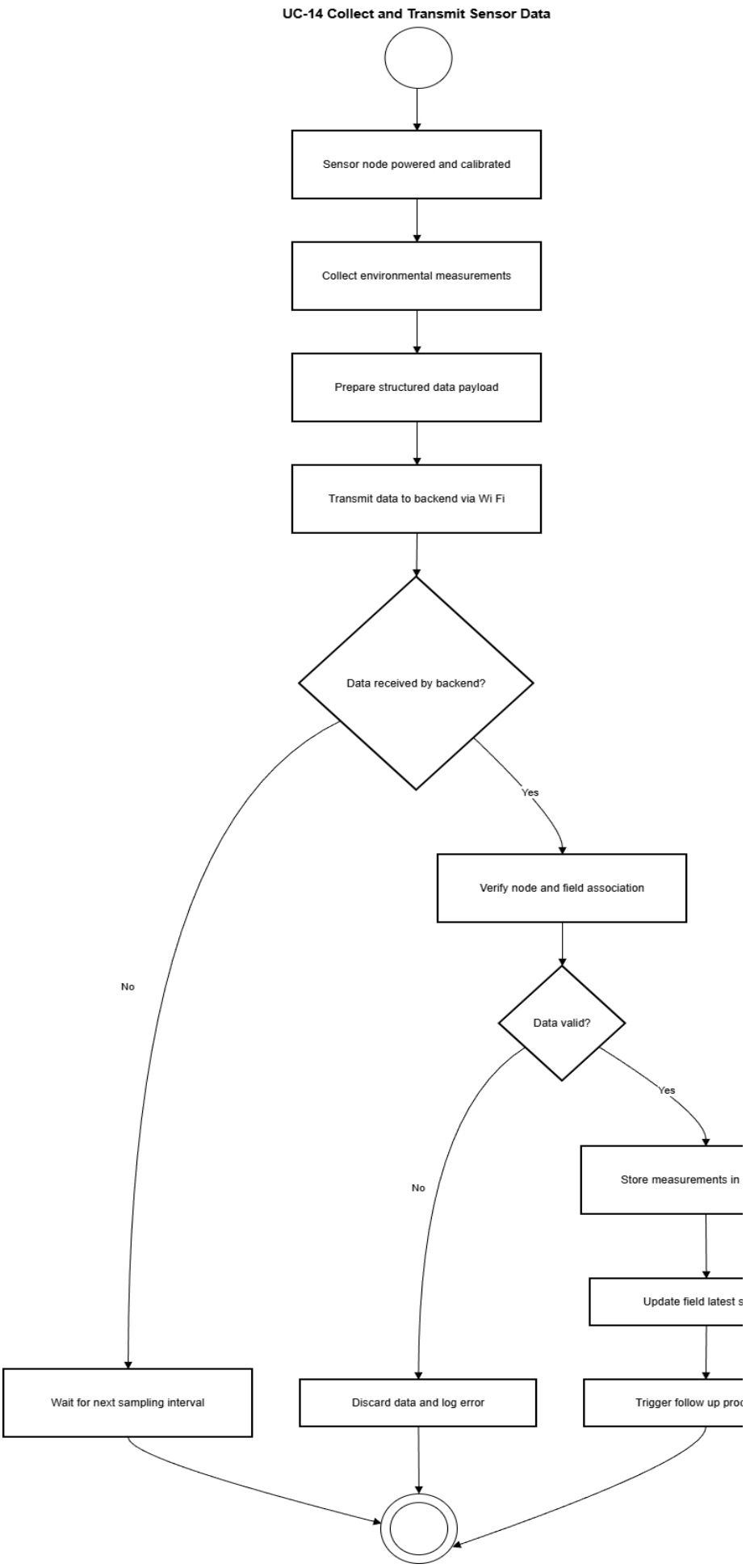

UC-10 View Recommended Actions
Diagnosis available
Retrieve diagnosed disease
Match disease with recommendation rules
Recommendation exists?
Yes
No
Generate treatment / monitoring actions
Prepare recommendation content
Display recommended actions to user
User reviews suggested actions
Show no recommendation available message

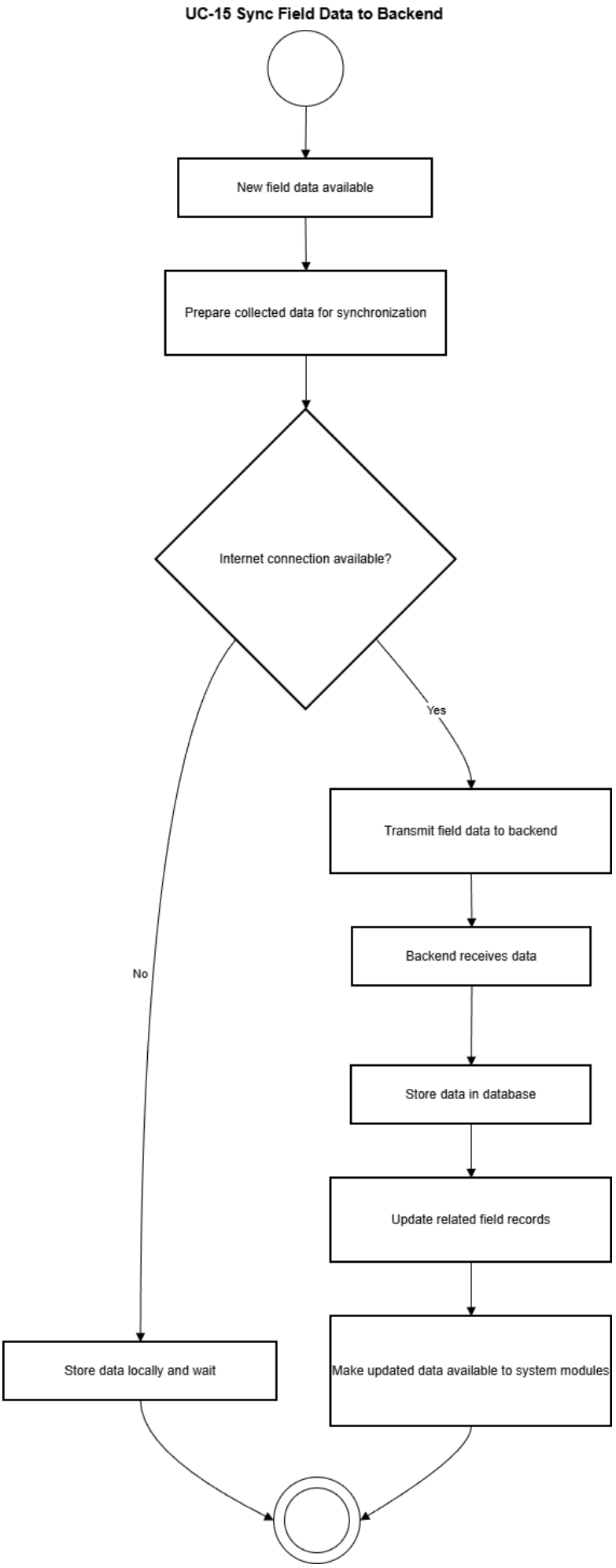

UC-11 Stakeholder Login to Dashboard
Stakeholder opens dashboard login page
Enter username or email and password
Submit login request
Verify credentials
Credentials valid?
Yes
No
Account active and authorized?
Yes
No
Initialize stakeholder session
Load stakeholder dashboard
Display invalid credentials message
Display access denied message
Display summaries and reporting tools

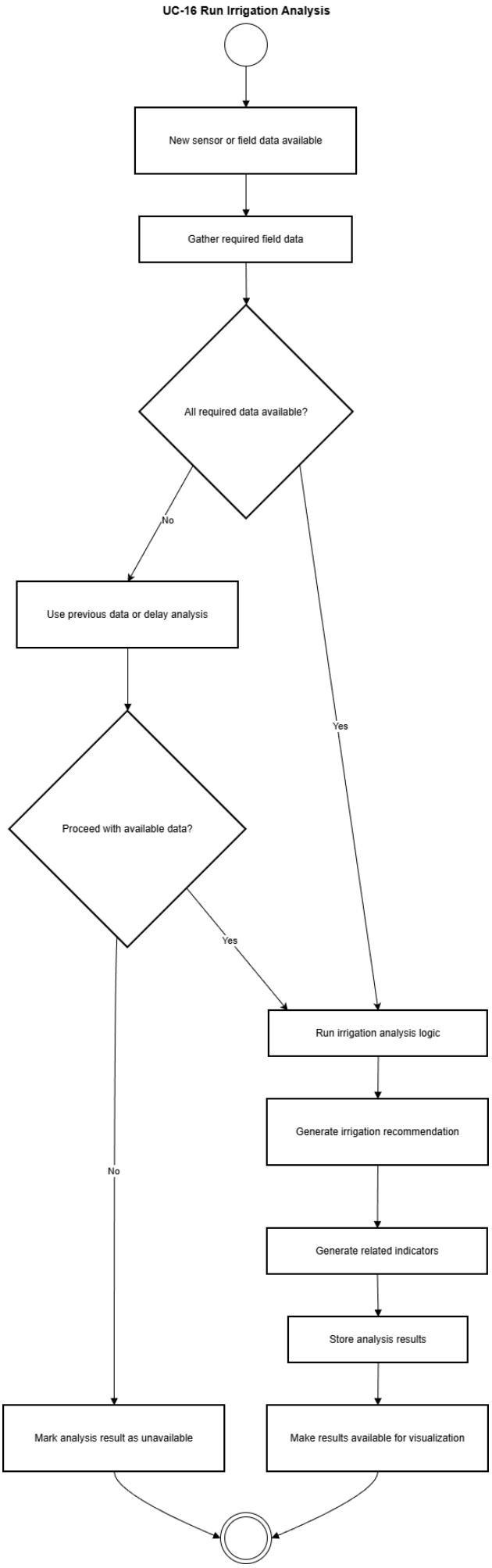

UC-12 View Sustainability and Carbon Footprint Reports
Stakeholder authenticated
Open Reports section
Select time period
Select report type
Request aggregated report data
Backend retrieves historical and sustainability data
Data available for period and type?
Yes
No
Compute required metrics
Generate charts tables and KPIs
Display no data or incomplete data message
Display selected report

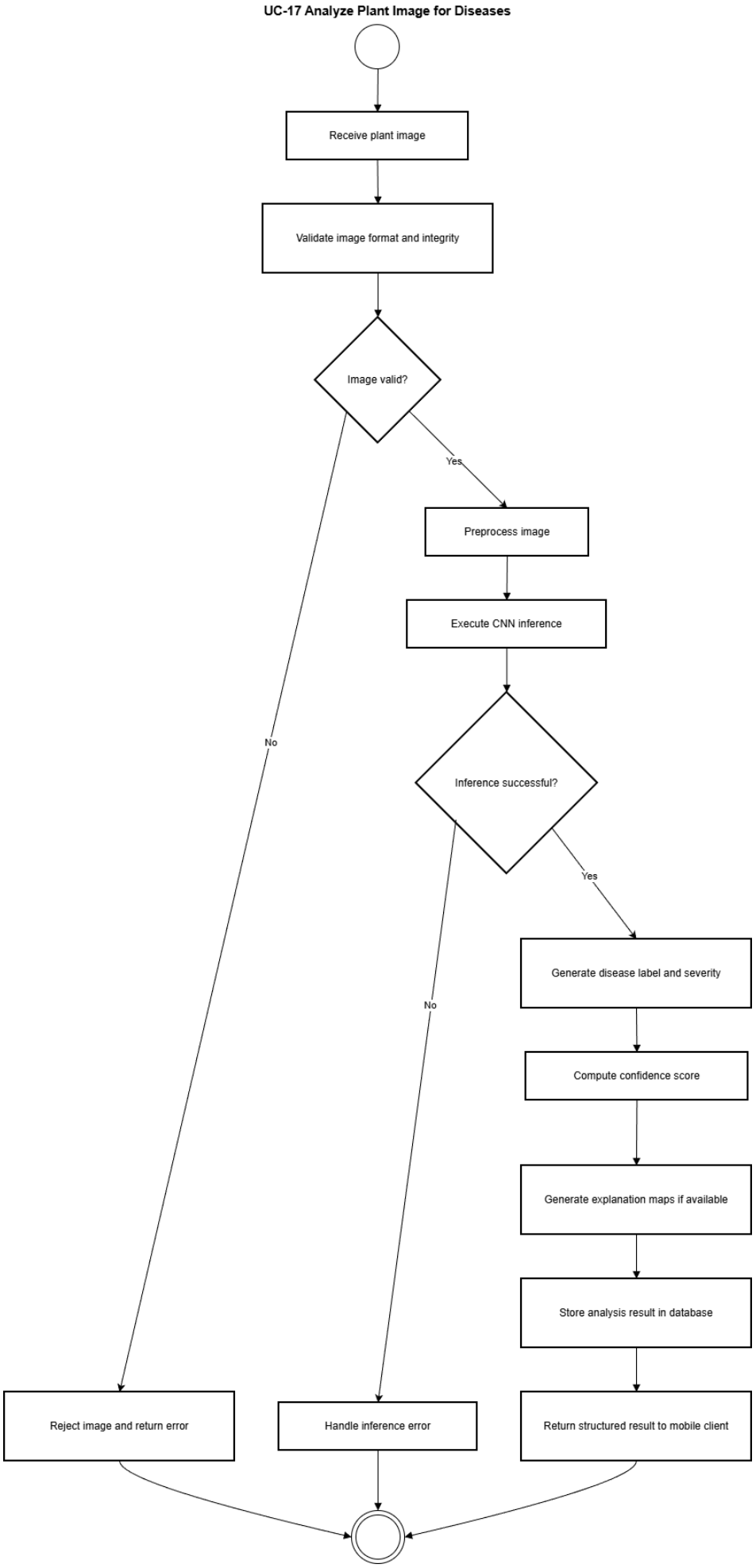
UC-13 Export Sustainability and Carbon Footprint Reports



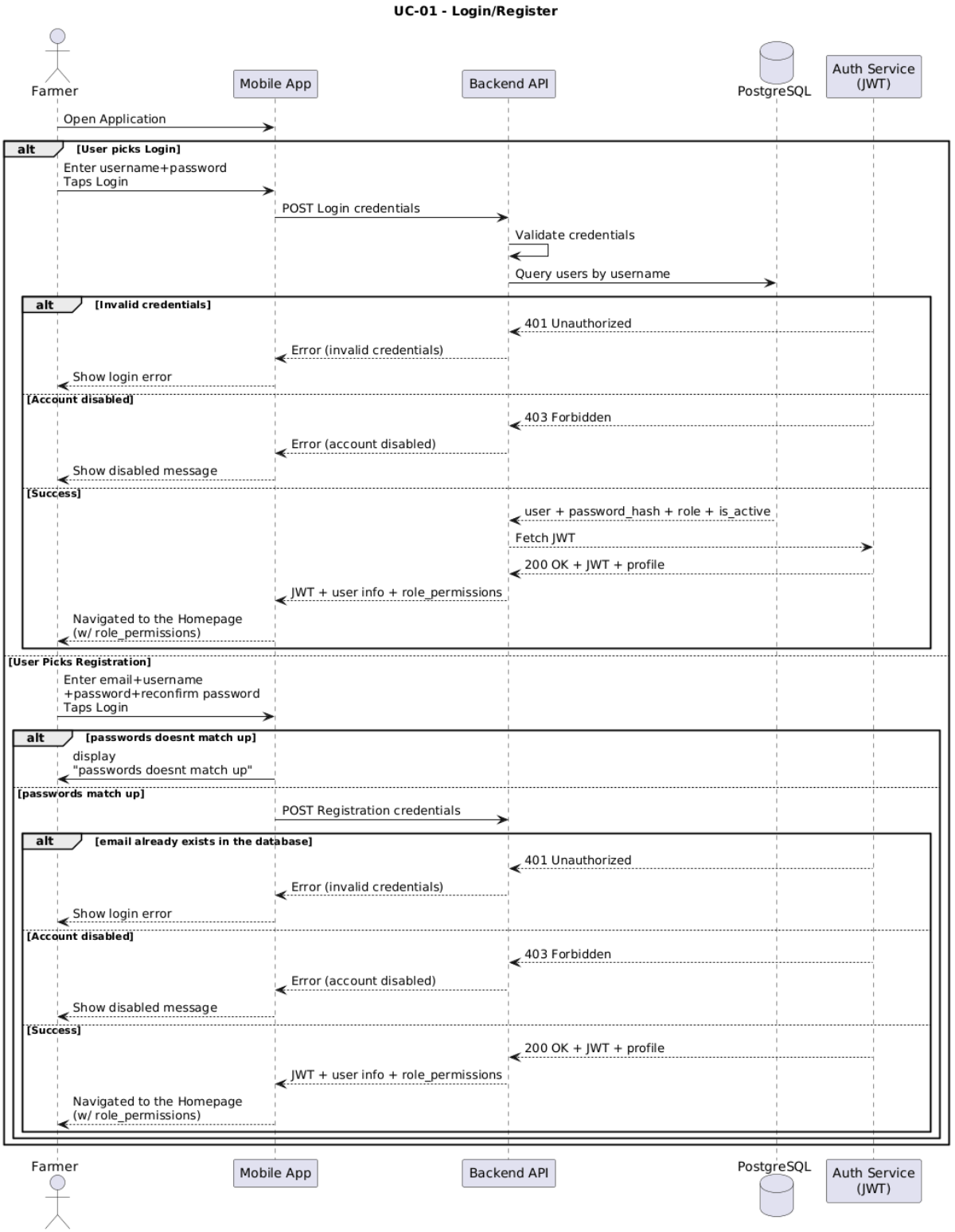

UC-14 Collect and Transmit Sensor Data
Sensor node powered and calibrated
Collect environmental measurements
Prepare structured data payload
Transmit data to backend via Wi Fi
Data received by backend?
Yes
No
Verify node and field association
Data valid?
Yes
No
Store measurements in
Update field latest s
Wait for next sampling interval
Discard data and log error
Trigger follow up proc

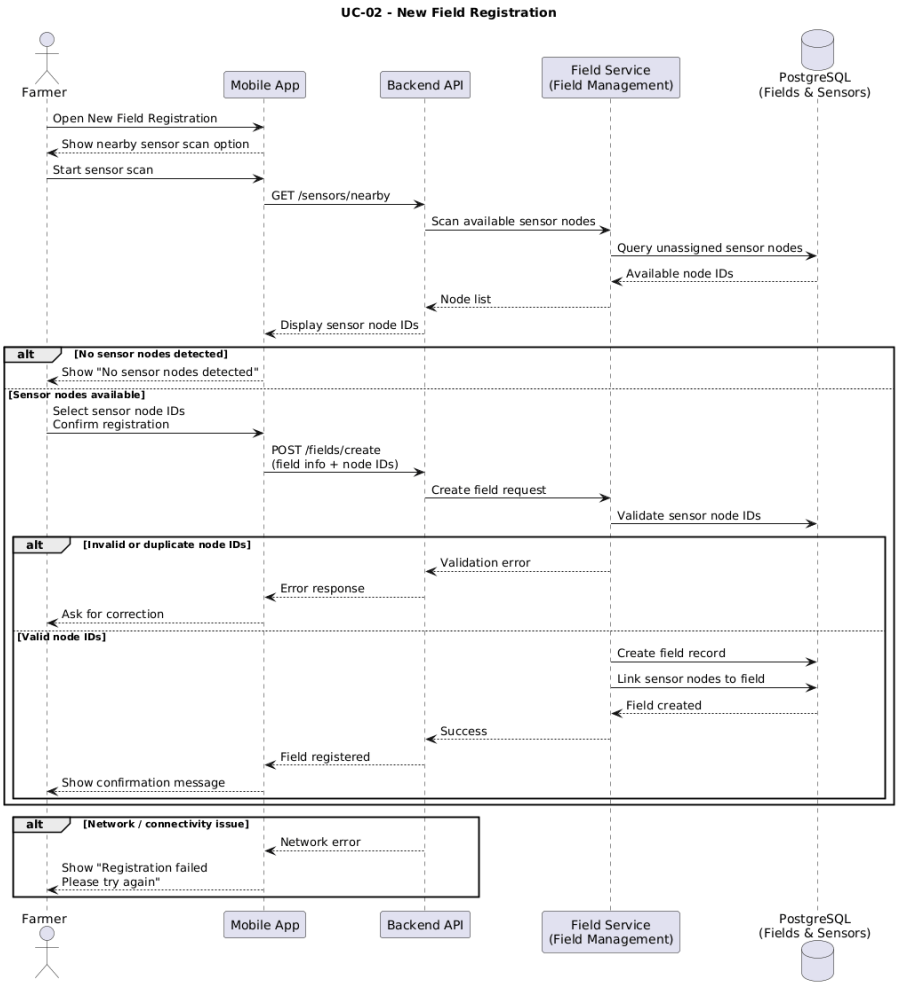

UC-15 Sync Field Data to Backend
New field data available
Prepare collected data for synchronization
Internet connection available?
Yes
No
Transmit field data to backend
Backend receives data
Store data in database
Update related field records
Make updated data available to system modules
Store data locally and wait

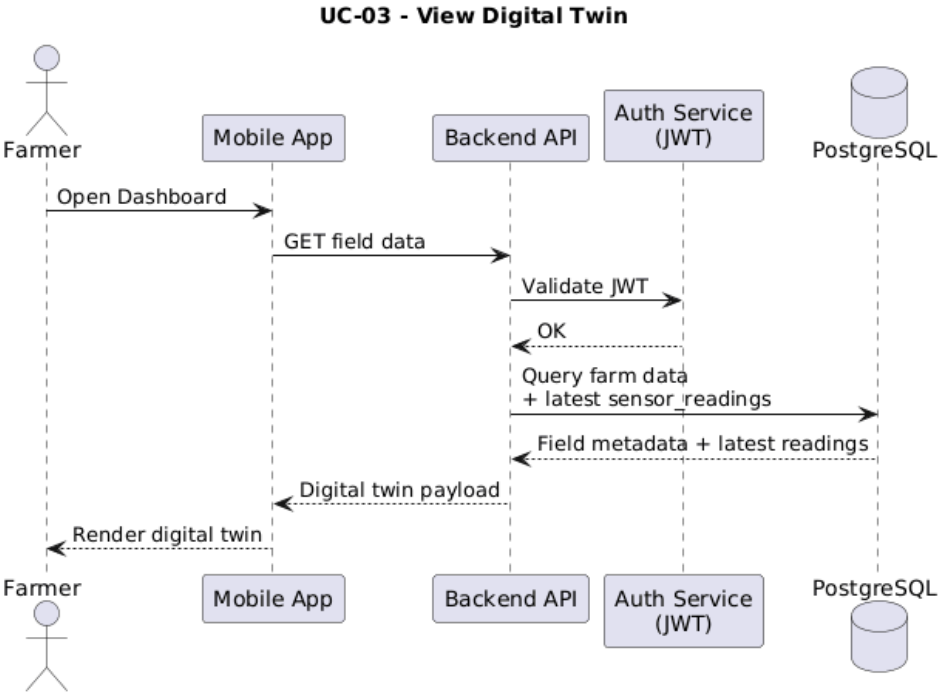

UC-16 Run Irrigation Analysis
New sensor or field data available
Gather required field data
All required data available?
No
Yes
Use previous data or delay analysis
Proceed with available data?
Yes
No
Run irrigation analysis logic
Generate irrigation recommendation
Generate related indicators
Store analysis results
Mark analysis result as unavailable
Make results available for visualization

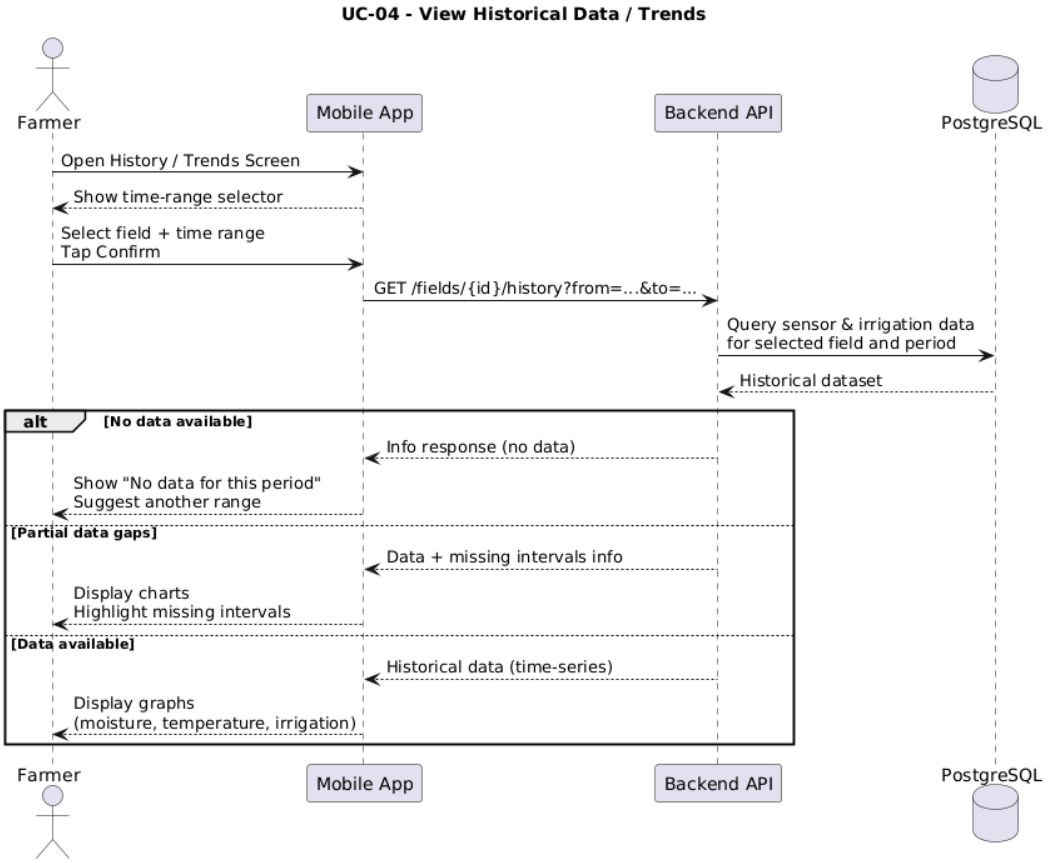

UC-17 Analyze Plant Image for Diseases
Receive plant image
Validate image format and integrity
Image valid?
Yes
No
Preprocess image
Execute CNN inference
Inference successful?
Yes
No
Generate disease label and severity
Compute confidence score
Generate explanation maps if available
Store analysis result in database
Reject image and return error
Handle inference error
Return structured result to mobile client

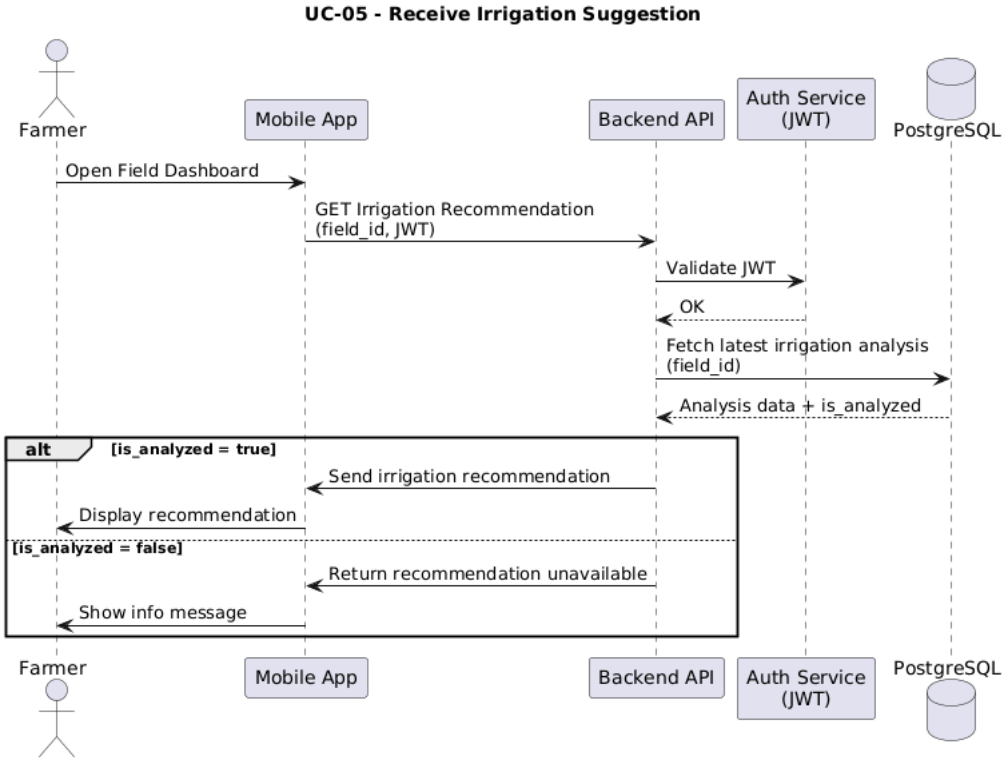
3.3.3.2. Sequence Diagrams

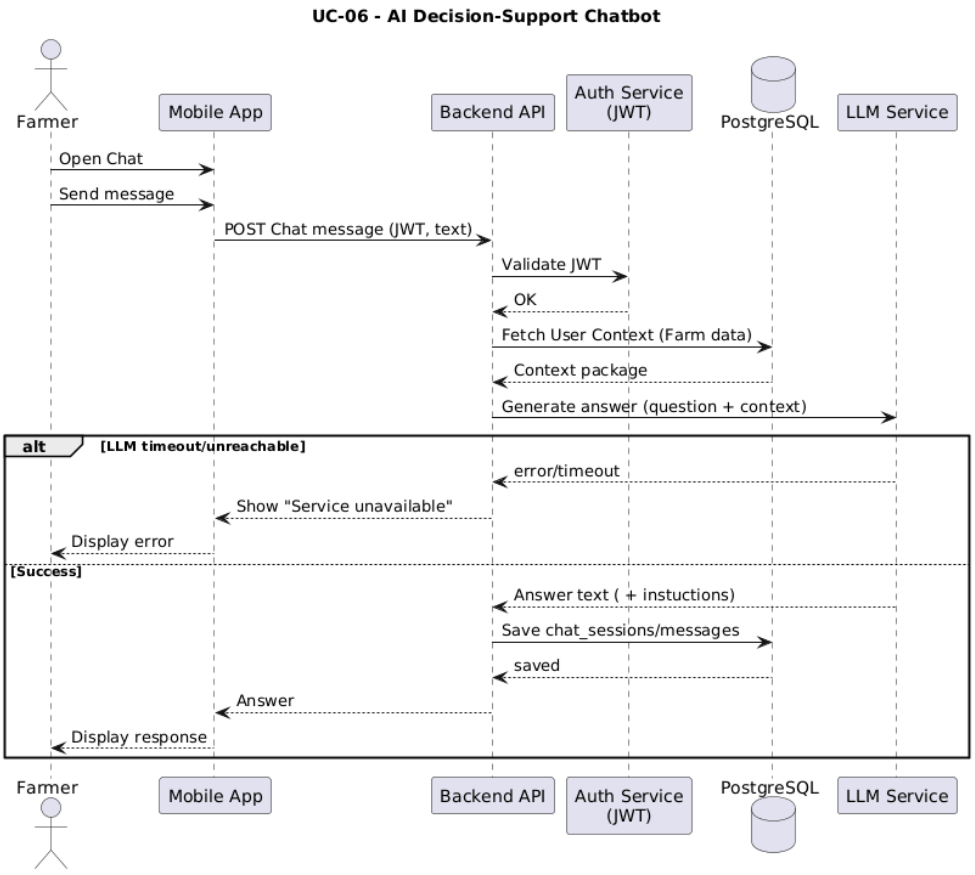


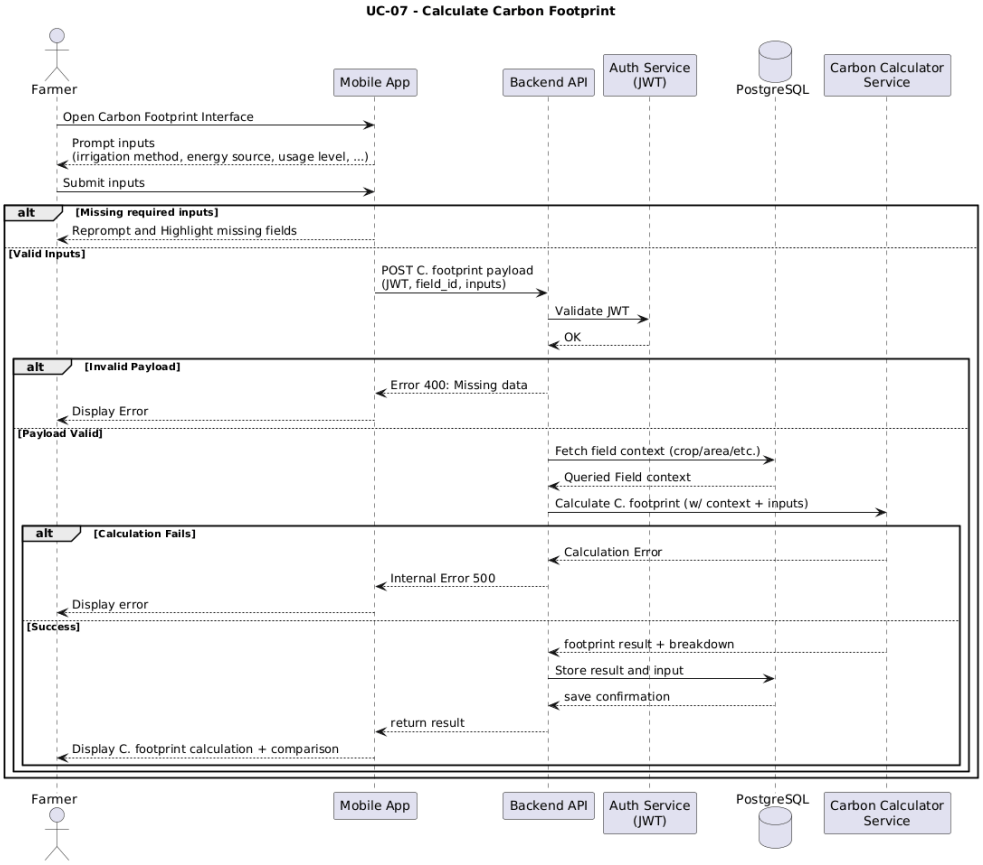

UC-02 - New Field Registration
Farmer
Mobile App
Backend API
Field Service
(Field Management)
PostgreSQL
(Fields & Sensors)
Open New Field Registration
Show nearby sensor scan option
Start sensor scan
GET /sensors/nearby
Scan available sensor nodes
Query unassigned sensor nodes
Available node IDs
Node list
Display sensor node IDs
alt
[No sensor nodes detected]
Show "No sensor nodes detected"
[Sensor nodes available]
Select sensor node IDs
Confirm registration
POST /fields/create
(field info + node IDs)
Create field request
Validate sensor node IDs
alt
[Invalid or duplicate node IDs]
Validation error
Error response
Ask for correction
[Valid node IDs]
Create field record
Link sensor nodes to field
Field created
Success
Field registered
Show confirmation message
alt
[Network / connectivity issue]
Network error
Show "Registration failed
Please try again"

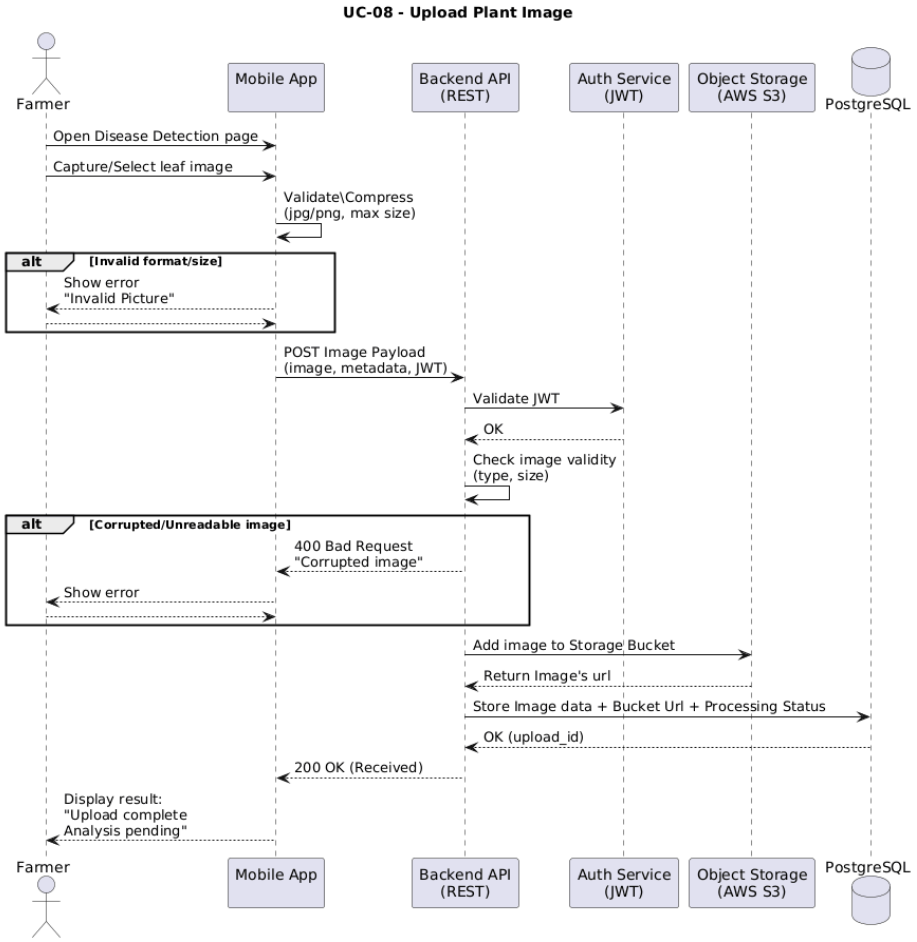

UC-03 - View Digital Twin
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL
Open Dashboard
GET field data
Validate JWT
OK
Query farm data
+ latest sensor readings
Field metadata + latest readings
Digital twin payload
Render digital twin

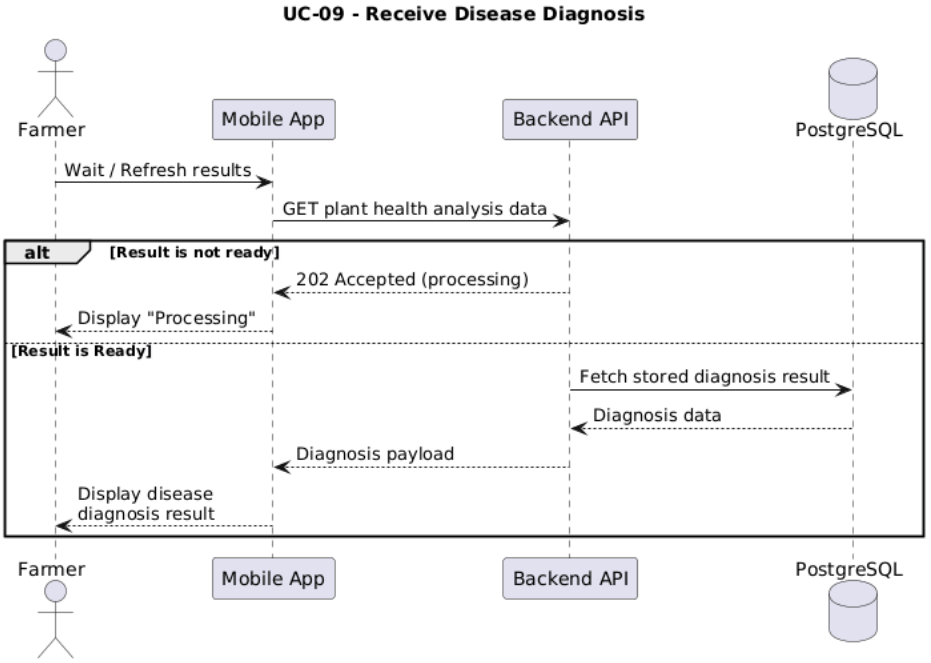

UC-04 - View Historical Data / Trends
Farmer
Mobile App
Backend API
PostgreSQL
Open History / Trends Screen
Show time-range selector
Select field + time range
Tap Confirm
GET /fields/{id}/history?from=...&to=...
Query sensor & irrigation data
for selected field and period
Historical dataset
alt
[No data available]
Info response (no data)
Show "No data for this period"
Suggest another range
[Partial data gaps]
Data + missing intervals info
Display charts
Highlight missing intervals
[Data available]
Historical data (time-series)
Display graphs
(moisture, temperature, irrigation)
Farmer
Mobile App
Backend API
PostgreSQL

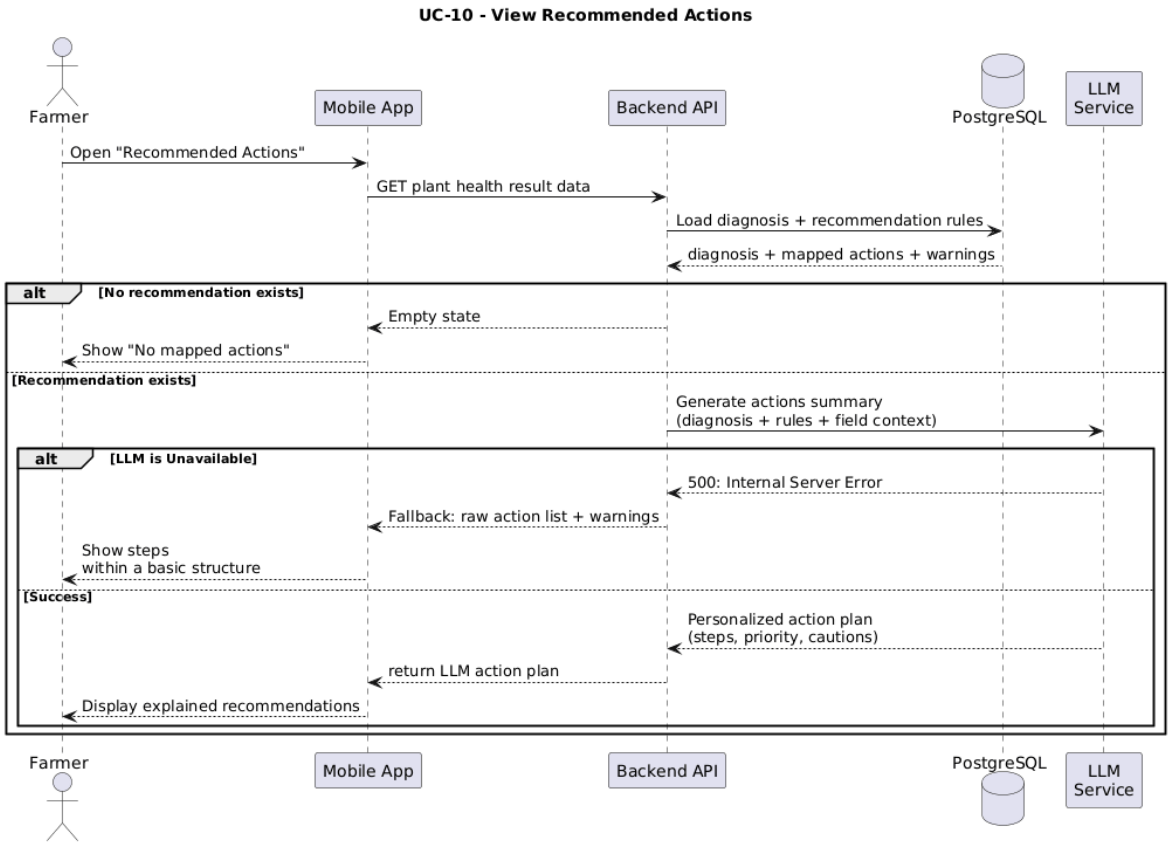

UC-05 - Receive Irrigation Suggestion
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL
Open Field Dashboard
GET Irrigation Recommendation
(field_id, JWT)
Validate JWT
OK
Fetch latest irrigation analysis
(field_id)
Analysis data + is_analyzed
alt
[is_analyzed = true]
Send irrigation recommendation
Display recommendation
[is_analyzed = false]
Return recommendation unavailable
Show info message
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL

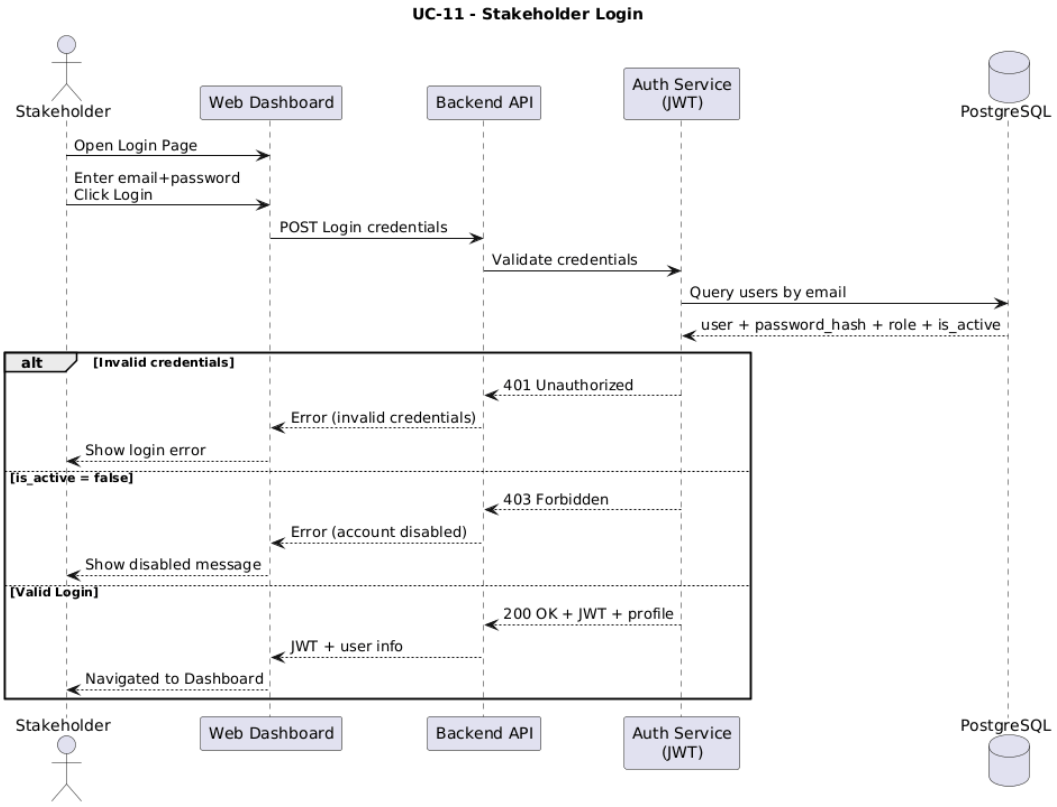

UC-06 - AI Decision-Support Chatbot
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
LLM Service
Open Chat
Send message
POST Chat message (JWT, text)
Validate JWT
OK
Fetch User Context (Farm data)
Context package
Generate answer (question + context)
alt
[LLM timeout/unreachable]
error/timeout
Show "Service unavailable"
Display error
[Success]
Answer text (+ instuctions)
Save chat_sessions/messages
saved
Answer
Display response

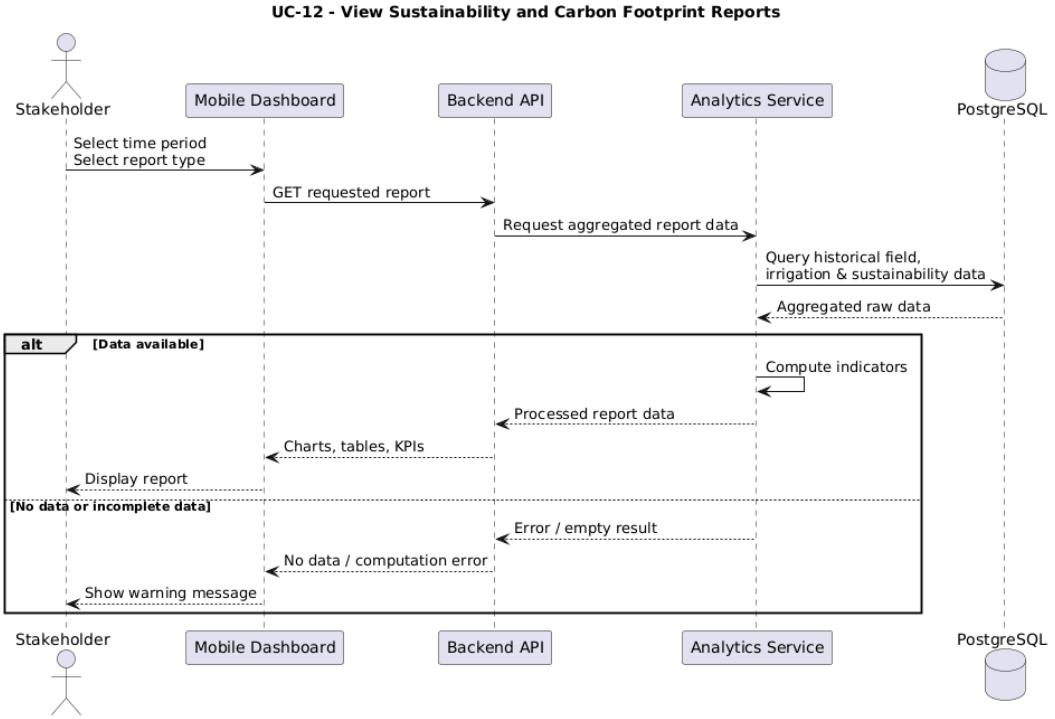

UC-07 - Calculate Carbon Footprint
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
Carbon Calculator Service
Open Carbon Footprint Interface
Prompt inputs
(irrigation method, energy source, usage level, ...)
Submit inputs
alt
[Missing required inputs]
Reprompt and Highlight missing fields
[Valid Inputs]
POST C. footprint payload
(JWT, field_id, inputs)
Validate JWT
OK
alt
[Invalid Payload]
Error 400: Missing data
Display Error
[Payload Valid]
Fetch field context (crop/area/etc.)
Queried Field context
Calculate C. footprint (w/ context + inputs)
alt
[Calculation Fails]
Calculation Error
Internal Error 500
Display error
[Success]
footprint result + breakdown
Store result and input
save confirmation
return result
Display C. footprint calculation + comparison

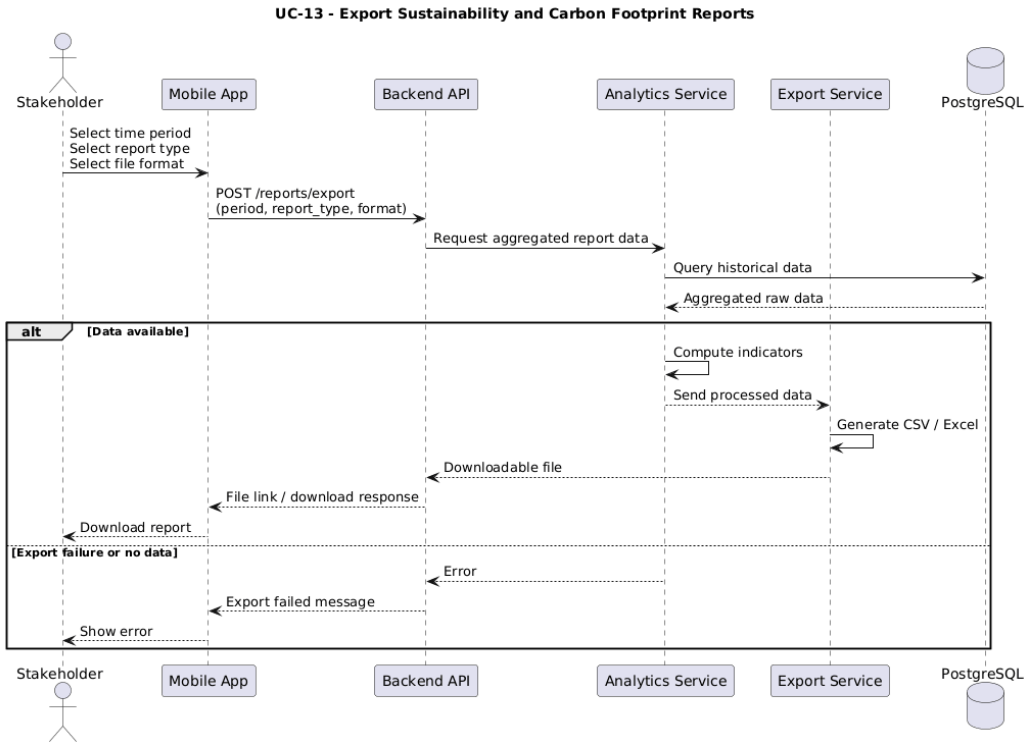

UC-08 - Upload Plant Image
Farmer
Mobile App
Backend API (REST)
Auth Service (JWT)
Object Storage (AWS S3)
PostgreSQL
Open Disease Detection page
Capture/Select leaf image
Validate\Compress (jpg/png, max size)
alt
[Invalid format/size]
Show error "Invalid Picture"
POST Image Payload (image, metadata, JWT)
Validate JWT
OK
Check image validity (type, size)
alt
[Corrupted/Unreadable image]
400 Bad Request "Corrupted image"
Show error
Add image to Storage Bucket
Return Image's url
Store Image data + Bucket Url + Processing Status
OK (upload_id)
200 OK (Received)
Display result: "Upload complete Analysis pending"

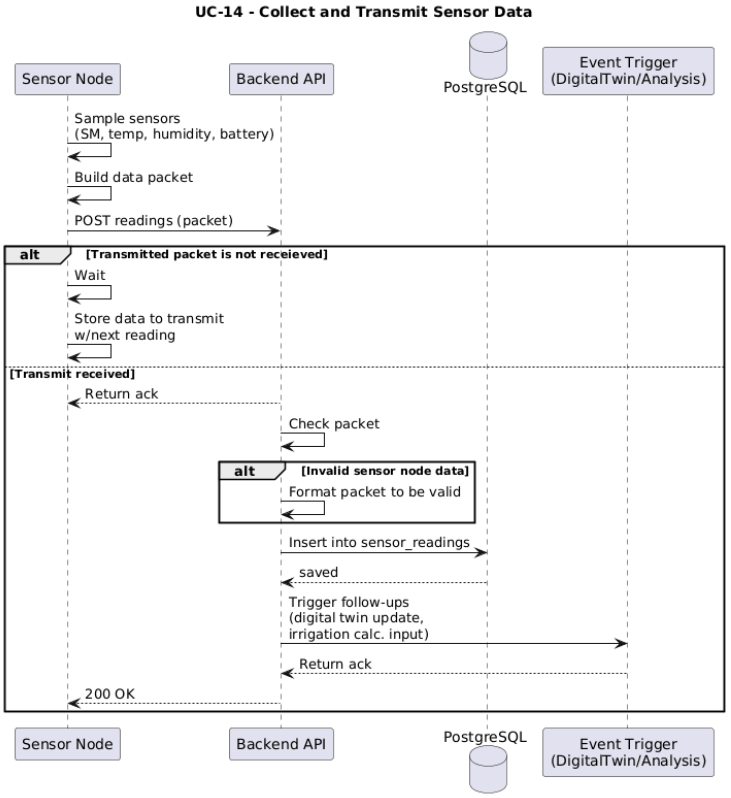

UC-09 - Receive Disease Diagnosis
Farmer
Mobile App
Backend API
PostgreSQL
Wait / Refresh results
GET plant health analysis data
alt
[Result is not ready]
202 Accepted (processing)
Display "Processing"
[Result is Ready]
Fetch stored diagnosis result
Diagnosis data
Diagnosis payload
Display disease diagnosis result

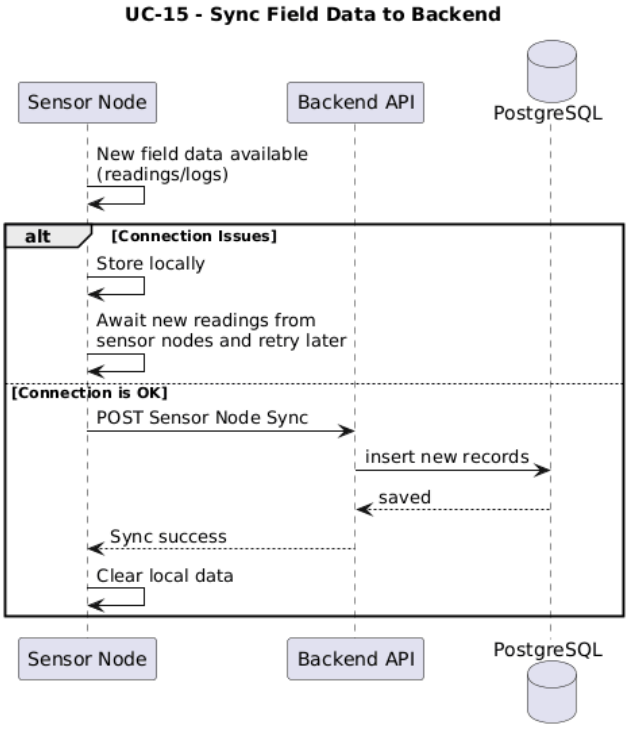

UC-10 - View Recommended Actions
Farmer
Mobile App
Backend API
PostgreSQL
LLM Service
Open "Recommended Actions"
GET plant health result data
Load diagnosis + recommendation rules
diagnosis + mapped actions + warnings
alt
[No recommendation exists]
Empty state
Show "No mapped actions"
[Recommendation exists]
Generate actions summary
(diagnosis + rules + field context)
alt
[LLM is Unavailable]
500: Internal Server Error
Fallback: raw action list + warnings
Show steps
within a basic structure
[Success]
Personalized action plan
(steps, priority, cautions)
return LLM action plan
Display explained recommendations

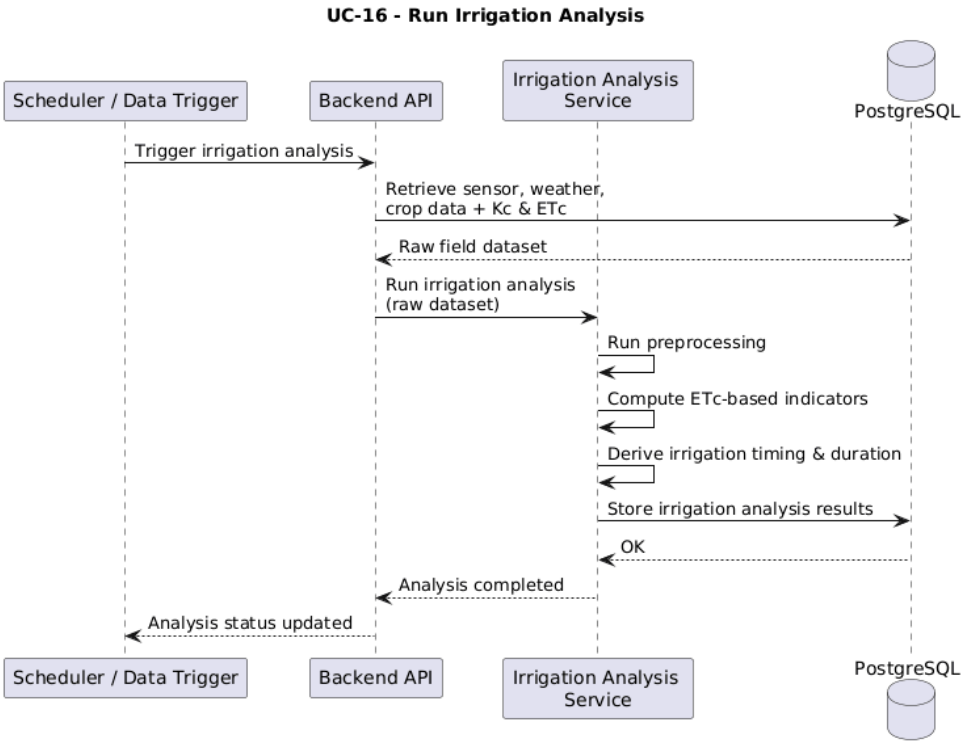

UC-11 - Stakeholder Login
Stakeholder
Web Dashboard
Backend API
Auth Service
(JWT)
PostgreSQL
Open Login Page
Enter email+password
Click Login
POST Login credentials
Validate credentials
Query users by email
user + password_hash + role + is_active
alt
[Invalid credentials]
401 Unauthorized
Error (invalid credentials)
Show login error
[is_active = false]
403 Forbidden
Error (account disabled)
Show disabled message
[Valid Login]
200 OK + JWT + profile
JWT + user info
Navigated to Dashboard

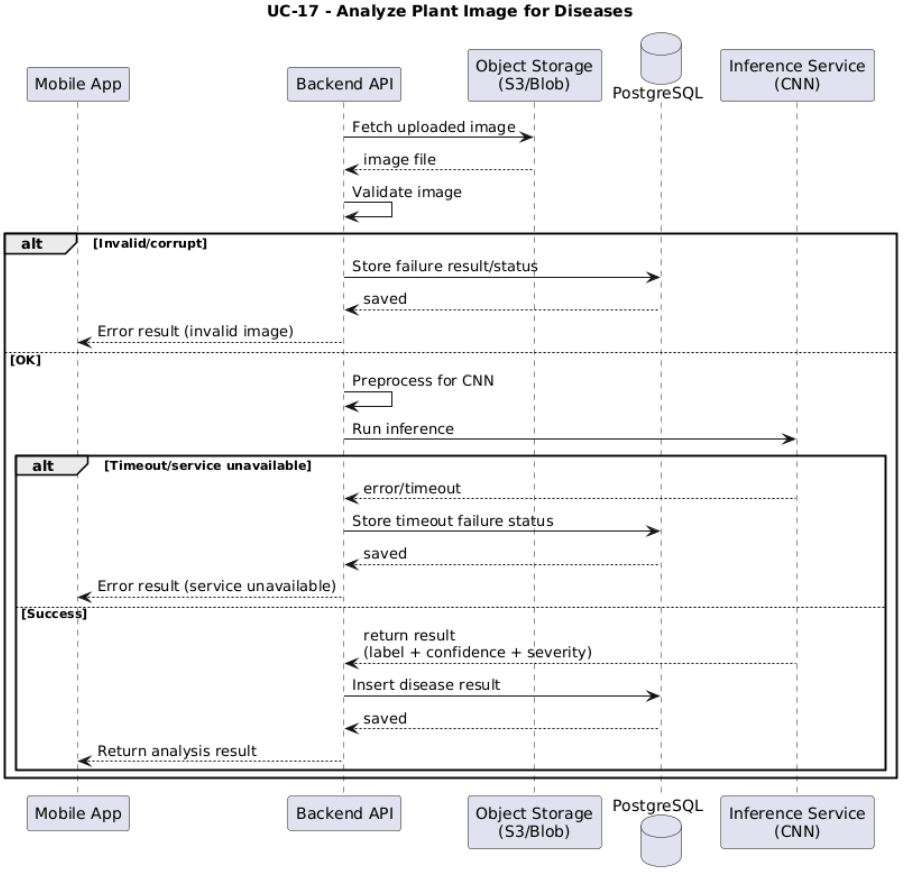

UC-12 - View Sustainability and Carbon Footprint Reports
Stakeholder
Mobile Dashboard
Backend API
Analytics Service
PostgreSQL
Select time period
Select report type
GET requested report
Request aggregated report data
Query historical field,
irrigation & sustainability data
Aggregated raw data
alt
[Data available]
Compute indicators
Processed report data
Charts, tables, KPIs
Display report
[No data or incomplete data]
Error / empty result
No data / computation error
Show warning message
Stakeholder
Mobile Dashboard
Backend API
Analytics Service
PostgreSQL


UC-13 - Export Sustainability and Carbon Footprint Reports
Stakeholder
Mobile App
Backend API
Analytics Service
Export Service
PostgreSQL
Select time period
Select report type
Select file format
POST /reports/export
(period, report_type, format)
Request aggregated report data
Query historical data
Aggregated raw data
alt
[Data available]
Compute indicators
Send processed data
Generate CSV / Excel
Downloadable file
File link / download response
Download report
[Export failure or no data]
Error
Export failed message
Show error
Stakeholder
Mobile App
Backend API
Analytics Service
Export Service
PostgreSQL


UC-14 - Collect and Transmit Sensor Data
Sensor Node
Backend API
PostgreSQL
Event Trigger
(DigitalTwin/Analysis)
Sample sensors
(SM, temp, humidity, battery)
Build data packet
POST readings (packet)
alt
[Transmitted packet is not receieved]
Wait
Store data to transmit
w/next reading
[Transmit received]
Return ack
Check packet
alt
[Invalid sensor node data]
Format packet to be valid
Insert into sensor_readings
saved
Trigger follow-ups
(digital twin update,
irrigation calc. input)
Return ack
200 OK
Sensor Node
Backend API
PostgreSQL
Event Trigger
(DigitalTwin/Analysis)


UC-15 - Sync Field Data to Backend
Sensor Node
Backend API
PostgreSQL
New field data available
(readings/logs)
alt
[Connection Issues]
Store locally
Await new readings from
sensor nodes and retry later
[Connection is OK]
POST Sensor Node Sync
insert new records
saved
Sync success
Clear local data
Sensor Node
Backend API
PostgreSQL


UC-16 - Run Irrigation Analysis
Scheduler / Data Trigger
Backend API
Irrigation Analysis Service
PostgreSQL
Trigger irrigation analysis
Retrieve sensor, weather, crop data + Kc & ETc
Raw field dataset
Run irrigation analysis (raw dataset)
Run preprocessing
Compute ETc-based indicators
Derive irrigation timing & duration
Store irrigation analysis results
OK
Analysis completed
Analysis status updated
Scheduler / Data Trigger
Backend API
Irrigation Analysis Service
PostgreSQL


UC-17 - Analyze Plant Image for Diseases
Mobile App
Backend API
Object Storage (S3/Blob)
PostgreSQL
Inference Service (CNN)
Fetch uploaded image
image file
Validate image
alt
[Invalid/corrupt]
Store failure result/status
saved
Error result (invalid image)
[OK]
Preprocess for CNN
Run inference
alt
[Timeout/service unavailable]
error/timeout
Store timeout failure status
saved
Error result (service unavailable)
[Success]
return result (label + confidence + severity)
Insert disease result
saved
Return analysis result
Mobile App
Backend API
Object Storage (S3/Blob)
PostgreSQL
Inference Service (CNN)

3.4. Methodology

3.4.1. Irrigation Prediction

This study does not employ a data-driven ML model for irrigation recommendation. Instead, the methodology is based on a physics- and agronomy-driven approach that combines reference evapotranspiration (ET_0), crop coefficients (Kc), and field sensor observations. Optional data-driven calibration steps (e.g., irrigation gain tuning) are treated as parameter adjustment rather than predictive ML.

3.4.1.1. Problem Formulation

The irrigation recommendation problem is formulated as a timing and duration estimation task. Given weather conditions, crop characteristics and soil moisture dynamics, the objective is to determine when irrigation should be initiated, based on estimated crop water depletion derived from ETc, and how long irrigation should be applied, by converting the required water amount into pump operation time using irrigation gain parameters.

The core formulation relies on estimating crop evapotranspiration (ETc), defined as the actual water consumption of a crop under current atmospheric conditions and growth stage.

3.4.1.2. Data Acquisition and Preprocessing

Data required for irrigation estimation are collected from heterogeneous sources:

- Meteorological data (temperature, humidity, wind speed or equivalent variables) used to compute or directly obtain ET_0.
- Field sensor data, primarily soil moisture, supported by air temperature and humidity.
- Crop metadata, including crop type and coefficient (Kc).
- Historical irrigation records, including irrigation events and pump duration or flow-rate information for calibration and validation.

Prior to irrigation estimation, all acquired data undergo a structured preprocessing pipeline:

- Timestamp standardization and alignment across all data sources.
- Unit normalization and physical range validation.
- Sensor quality control, including noise reduction, spike detection and drift monitoring.

- Missing data analysis with context-aware handling based on data criticality.
- Context-aware outlier detection distinguishing valid irrigation-induced changes from sensor anomalies.

3.4.1.3. Algorithmic Approach

The irrigation recommendation algorithm follows a deterministic and interpretable workflow:

- **ET_0 Estimation:** Reference evapotranspiration is obtained directly or computed from meteorological inputs.
- **Kc Application:** Crop-specific Kc values are generated as a time series based on crop type and growth stage and calibrated through the process.
- **ETc Computation:** Crop evapotranspiration is calculated as **$ETc = ET_0 \times Kc$** [1].
- **Water Depletion Assessment:** Cumulative ETc is evaluated over a rolling time window to estimate soil water depletion.
- **Irrigation Timing Estimation:** Irrigation start time is determined when depletion exceeds predefined or calibrated thresholds.
- **Irrigation Duration Estimation:** Required water depth is converted into irrigation duration using irrigation gain.

3.4.1.4. System Evaluation Metrics

System performance is evaluated using domain-relevant and operational metrics rather than classification accuracy:

- Consistency between predicted irrigation timing and observed soil moisture recovery
- Deviation between estimated and observed irrigation duration
- Reduction in over-irrigation or under-irrigation events

Where historical irrigation data are available, predicted and actual irrigation durations are compared for validation.

3.4.1.5. Implementation Details

The irrigation estimation logic is implemented as a modular service within the system architecture. ET_0 computation, Kc profile generation, ETc calculation and irrigation duration estimation are

implemented as separate functional modules to ensure maintainability and extensibility.

3.4.2. Disease Detection

3.4.2.1. Problem Formulation

This module is designed as an image processing and ML component of the TARAS system, focusing on the analysis of plant leaf images for disease identification. The primary objective is to enable users to capture or upload a leaf image, after which the system automatically detects the presence of plant disease and classifies its type.

Formally, the task is modeled as a multi-class image classification problem. Each input image **X** is mapped to a disease label **Y**, selected from a predefined set of categories representing both healthy and diseased plant conditions. The model aims to achieve high classification accuracy while ensuring robust generalization performance on previously unseen data.

3.4.2.2. Data Acquisition and Preprocessing

3.4.2.2.1. Data Acquisition

Plant leaf images are obtained either by direct capture using camera-enabled devices or by user upload through the TARAS system interface. This approach enables the module to operate under real field conditions as well as controlled analysis environments. The acquired images constitute the raw input of the image processing pipeline.

3.4.2.2.2. Preprocessing

All input images are converted to RGB format to ensure consistency across different devices and imaging conditions. The images are then resized to a fixed resolution of 256 × 256 pixels to enable uniform processing and computational efficiency. Pixel values are normalized using the EfficientNetV2 preprocessing function to align the input distribution with ImageNet-pretrained weights.

3.4.2.3. Algorithmic Approach

The disease detection module employs a deep learning–based image classification approach to identify plant diseases from leaf images. Convolutional neural networks (CNNs) are used to automatically learn discriminative visual features from preprocessed images, including texture patterns, color distributions, and structural irregularities that are commonly associated with diseased plant leaves.

Input images are processed through a CNN backbone that transforms raw pixel data into high-level feature representations. These features enable the model to distinguish between healthy and diseased leaf conditions as well as different disease categories. The use of pretrained CNN architectures allows the system to leverage general visual knowledge learned from large-scale datasets, improving robustness under varying imaging conditions such as illumination changes, background noise, and leaf orientation.

The final classification is performed by a task-specific output layer that maps the extracted features to predefined disease classes. The predicted disease label is selected based on the highest class probability, enabling reliable and automated disease identification within the TARAS system. This approach ensures accurate detection while maintaining adaptability to real-world agricultural scenarios.

3.4.2.4. Model Selection and Comparative Evaluation

To identify the most suitable architecture for the proposed system, multiple CNN models with varying complexity levels were systematically evaluated, including MobileNetV3-Small, EfficientNet-B0, ResNet50, EfficientNet-B7, and EfficientNetV2-S. All models were trained and tested on the same dataset using identical preprocessing pipelines and evaluation protocols to ensure a fair and consistent comparison.

The quantitative performance comparison of the evaluated architectures on the independent test set is summarized in Table 1. As shown in Table 1, EfficientNetV2-S achieved the highest performance across all major evaluation metrics. Specifically, the model reached an accuracy of 96.55%, a macro F1-score of 96.30%, and a weighted F1-score of 96.54%, outperforming all other evaluated architectures.

Although EfficientNet-B7 is a deeper and more parameter-rich architecture compared to EfficientNetV2-S, its performance on the independent test set was inferior, as summarized in Table 1. This outcome suggests that the increased model complexity did not yield improved generalization for the given dataset and may have led to overfitting effects. In contrast, the lightweight MobileNetV3-Small model achieved competitive accuracy with substantially lower computational cost; however, it did not demonstrate the same level of robustness and consistency as EfficientNetV2-S across precision, recall, and F1-score metrics.

Based on these findings, EfficientNetV2-S was selected as the core classification model due to its superior balance between classification accuracy, generalization performance, and computational efficiency, making it well suited for deployment within the TARAS system.

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)	Precision (Weighted)	Recall (Weighted)	F1 (Weighted)
EfficientNetV2-S(Base Model)	**0.9654**	0.9662	0.9610	0.9629	0.9659	0.9654	0.9653
EfficientNetB0	0.8564	0.8473	0.8192	0.8276	0.8574	0.8564	0.8547
MobileNetV3Small	0.9313	0.9286	0.9069	0.9155	0.9311	0.9313	0.9302
ResNet-50	0.7916	0.7738	0.7564	0.7621	0.7927	0.7916	0.7894
EfficientNetB7	0.9055	0.8958	0.8736	0.8818	0.9066	0.9055	0.9046

Table 1. Test-set performance comparison of the evaluated CNN models using accuracy, precision, recall, and F1-score metrics.

3.4.2.5. System Evaluation Metrics

Model performance is evaluated using multiple complementary metrics to capture different aspects of classification quality:

- **Accuracy**: Overall proportion of correctly classified images
- **Precision**: Ability of the model to avoid false disease predictions
- **Recall**: Ability of the model to correctly identify diseased samples
- **F1-score**: Harmonic mean of precision and recall

3.4.2.6. Implementation Details

The plant disease detection module is implemented using the TensorFlow and Keras deep learning frameworks. Model training and evaluation are conducted in an offline environment, while the trained model is deployed for inference as part of the TARAS image processing pipeline. This separation ensures efficient development while enabling real-time disease classification during system operation.

Within the TARAS system, the module receives preprocessed leaf images as input and performs inference using the trained

EfficientNetV2-S model. The model outputs a probability distribution over predefined disease classes, and the class with the highest probability is selected as the predicted disease label. This output is then forwarded to higher-level TARAS components to support decision-making processes such as disease monitoring and agricultural analysis.

The implementation supports modular integration, allowing the disease detection component to operate independently from other system modules. This design facilitates future updates, including model retraining, architecture replacement, or deployment on different computational platforms without affecting the overall system structure.

3.4.3. LLM-Based Advisory & Decision Support

3.4.3.1. Problem Formulation

The objective of the LLM-based advisory component is to provide context-aware, human-readable decision support by interpreting outputs generated by the irrigation prediction and disease detection modules. While these modules produce numerical predictions or categorical outputs, end users such as farmers may lack the technical expertise to directly interpret these results. Therefore, the problem is formulated as a natural language generation and reasoning task, where structured system outputs and contextual data are transformed into actionable recommendations and explanations.

Formally, given a set of structured inputs C (input string) representing environmental data, model predictions, and user context, the LLM generates a natural language response R (output string) that explains system outputs and provides advisory guidance aligned with agricultural best practices.

3.4.3.2. Input Representation and Context Construction

The input to the LLM is constructed as a structured context that combines multiple information sources, including irrigation predictions, detected disease classes, crop type, growth stage and recent environmental conditions.

Context construction follows a deterministic process in which numerical outputs are normalized, categorical variables are mapped to domain-specific terminology and temporal information is summarized. This structured context is embedded within a prompt that constrains the LLM to generate responses grounded in system data, thereby minimizing hallucinations and ensuring domain relevance.

3.4.3.3. Response Generation and Explainability

The LLM generates clear and actionable natural-language advisory responses by explaining underlying system outputs and explicitly referencing key input factors such as soil moisture, growth stage, or visual symptoms. This approach improves transparency and user trust by clarifying not only what is recommended, but also why the recommendation is made.

3.4.3.4. System Evaluation and Limitations

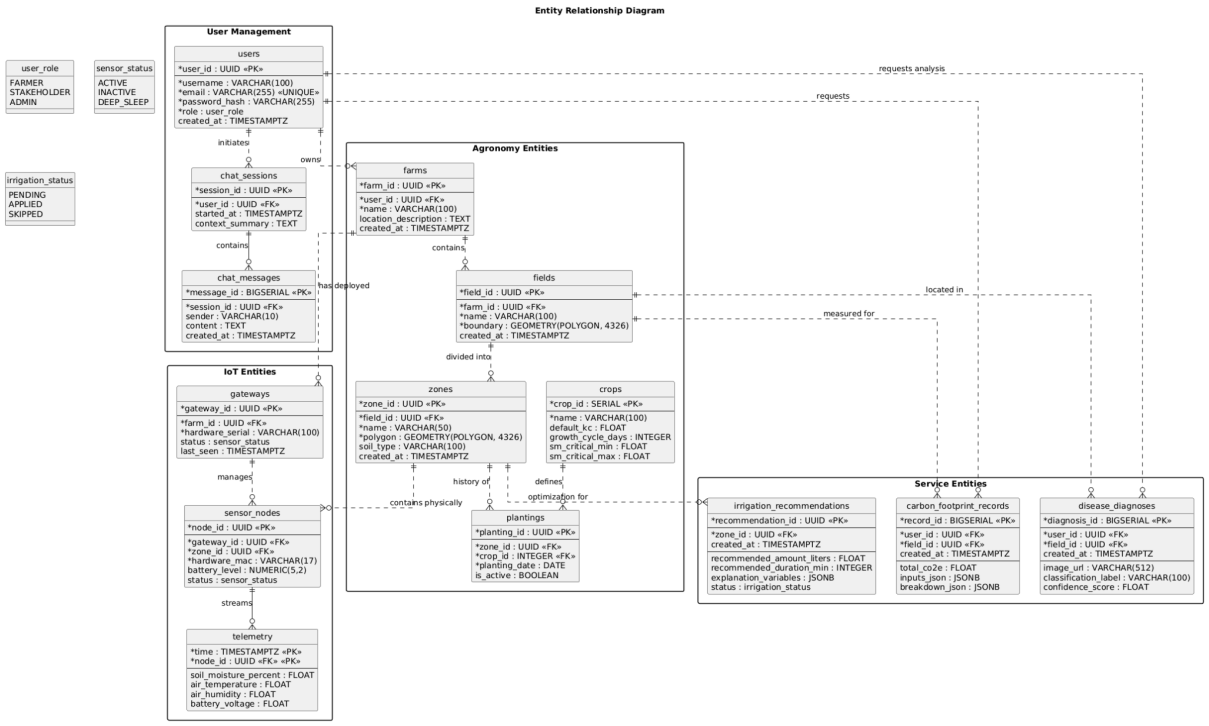
The LLM-based advisory system is evaluated qualitatively based on explanation coherence, consistency with system outputs and usability from an end-user perspective. As it does not perform prediction tasks, the evaluation focuses on the correctness and domain alignment of generated explanations. The system is inherently dependent on the quality of upstream model outputs and prompt design, and its reliability may be reduced for unseen crops or disease types; therefore, the advisory component is intended strictly as a decision-support aid and does not replace expert agronomists.

3.4.3.5. Implementation Details

The LLM-based advisory module is implemented as a cloud-based service integrated with the TARAS backend. It receives structured inputs via an API interface and returns generated responses to the user interface layer

3.5. Data Design

3.5.1. Entity-Relationship Diagram



3.5.2. Data Pipeline

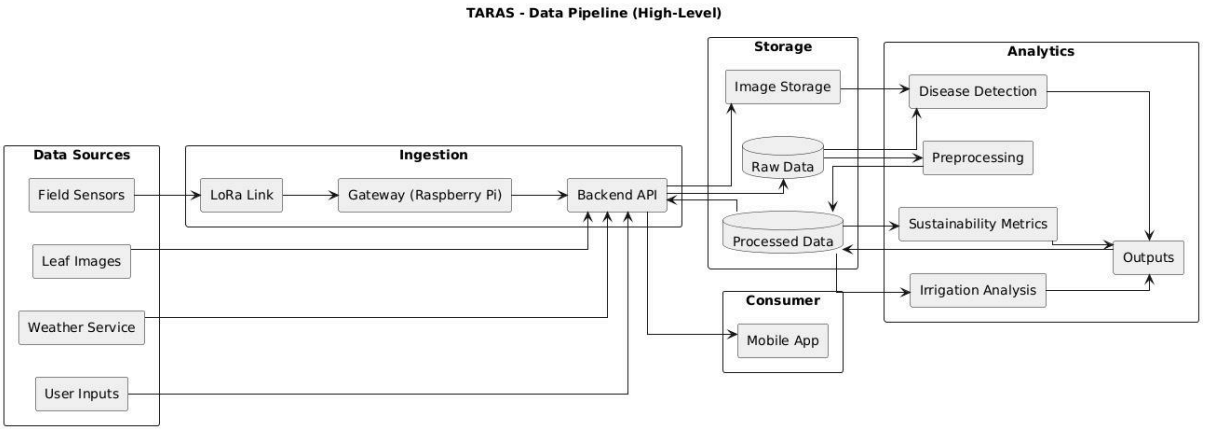
The TARAS data pipeline defines how heterogeneous data collected from field sensors, user inputs, and image uploads are transformed into reliable and analysis-ready information. The pipeline is designed to support real-time monitoring, historical analysis, and decision-support operations while maintaining data consistency and traceability.

Field sensor measurements such as soil moisture and environmental conditions are transmitted to gateway devices and forwarded to cloud services. In parallel, meteorological data, crop metadata, irrigation parameters and plant images are received through backend interfaces. All incoming data are timestamped and associated with field and device identifiers to ensure coherent integration.

Before storage, the data undergo preprocessing steps including timestamp alignment, unit normalization, basic quality checks, and context-aware filtering to handle missing values and abnormal sensor readings. Processed numerical data are stored in a centralized database, while image data are stored in dedicated storage buckets with linked metadata.

The processed data are then consumed by analytical modules responsible for irrigation estimation, disease detection, and sustainability analysis. Derived outputs such as irrigation recommendations,

classification results and environmental indicators are stored as processed records and delivered to the mobile application through backend APIs. This structured pipeline enables consistent data flow across sensing, analytics and visualization layers and provides a scalable foundation for the TARAS platform.



3.6. User Interface Design

3.6.1. Interface Hierarchy

The interface follows a hierarchical structure that prioritizes critical agricultural information over details. Core metrics such as soil moisture, temperature, and field status are positioned at the highest level of the interface hierarchy and are intended to be presented through large, clearly separated data cards. This structure enables users to access essential information with minimal navigation and reduces cognitive effort during decision-making.

The hierarchy is intentionally simplified to accommodate the diverse demographic profile of farmers, particularly older individuals with limited technical experience. By minimizing nested menus and reducing visual competition between interface elements, the interface supports rapid identification of actionable insights without overwhelming the user.

3.6.2. Visual Presentation

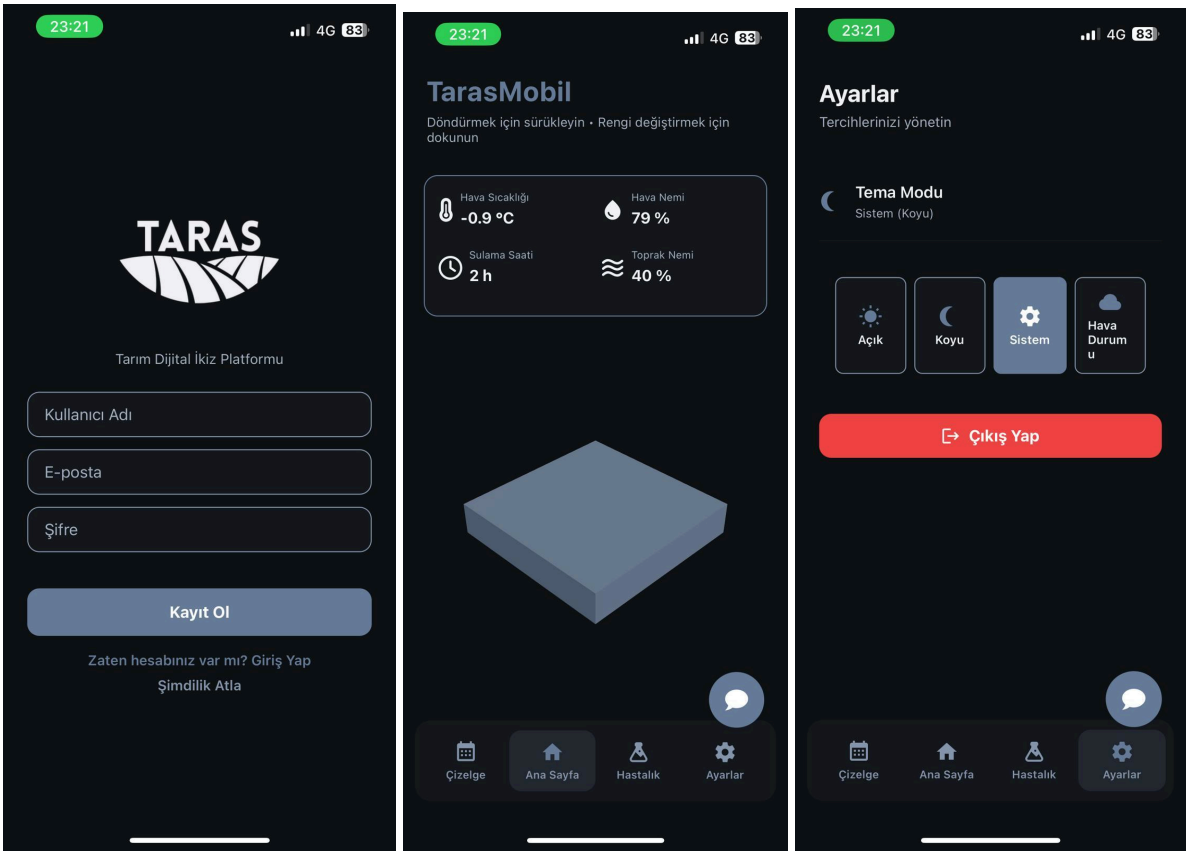
A balanced and minimalistic color palette was adopted as the foundation of the visualization and interaction layer. The palette is deliberately kept simple so that digital twin visualizations remain visually distinct from the interface background across different contexts.

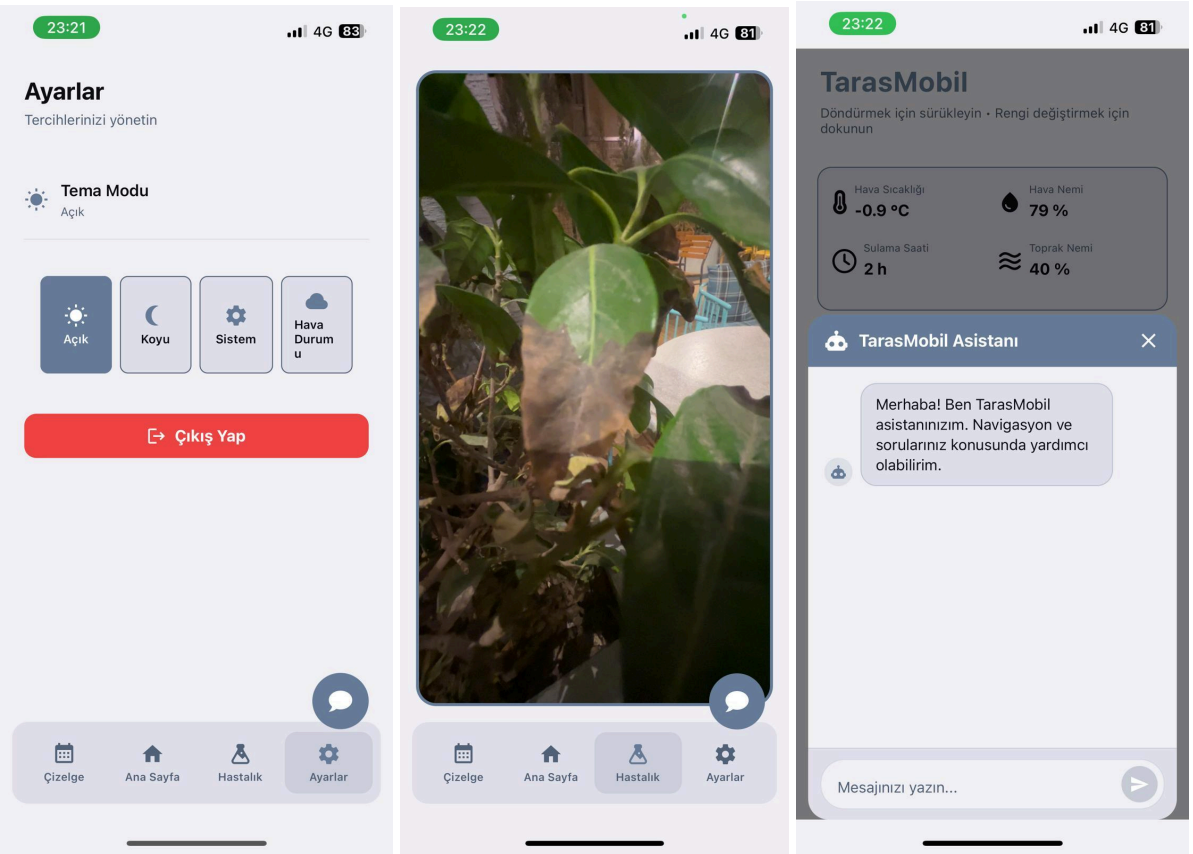
Light and dark modes are defined to assign inverse roles to background and text elements, preserving foreground and background separation, and visual comfort under varying lighting conditions.

3.6.3. Accessibility and Responsivity

The interface design adheres to web accessibility and readability principles, including WCAG contrast recommendations, to support usability across varying lighting conditions and display settings. Text sizes, spacing, and contrast levels are specified to compensate for age-related declines in visual acuity and cognitive flexibility.

To further reduce cognitive load, essential information is structured to be presented in a concise and visually separated manner. In addition, an AI-powered conversational assistant is planned as a core interaction mechanism to support users with varying levels of technological literacy [5].





3.7. Conclusion

This document outlined the system architecture and methodology of the TARAS precision agriculture platform, focusing on its layered structure, modular subsystems and data-driven decision logic. By combining field sensing, agronomy-based irrigation analysis, AI-supported disease detection and user-centered interface design, TARAS provides a scalable and maintainable foundation for intelligent agricultural support. The proposed architecture enables future extensions while ensuring reliable and accessible system operation.

3.8. References

[1]*Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998).*

Crop evapotranspiration – Guidelines for computing crop water requirements.

FAO Irrigation and Drainage Paper No. 56.

Food and Agriculture Organization of the United Nations.

https://www.fao.org/4/x0490e/x0490e00.htm

[2]Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016).
Using deep learning for image-based plant disease detection.

https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2016.01419/full

[3] Tan, M., & Le, Q. (2019).
EfficientNetV2: Smaller Models and Faster Training
Proceedings of the 36th International Conference on Machine Learning (ICML)
https://proceedings.mlr.press/v139/tan21a/tan21a.pdf

[4]*Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014).*

How transferable are features in deep neural networks?

Advances in Neural Information Processing Systems (NeurIPS).

https://arxiv.org/abs/1411.1792

[5] Czaja, S. J., & Lee, C. C. (2007).

The impact of aging on access to technology.

Universal Access in the Information Society, 5(4), 341–349.

https://www.researchgate.net/publication/220606716_The_impact_of_aging_on_access_to_technologdr